

FarmVendor

A Role- Based Digital Platform for Farmer-Vendor Coordination
with Demand Forecasting and vendor optimization

Student Name : Maneesha Eeshwara

Student ID : 300392759

Course : CSIS 4495 Applied Research Project

Section : 03

GitHub Repository : https://github.com/maneeshaprs-rgb/W26_4495_S3_ManeeshaE.git

Contents

1. Introduction.....	4
1.1 Background and Domain Context.....	4
1.2 Problem Definition	4
1.3 Research Questions.....	5
1.4 Literature Overview and Research Gap.....	5
Literature Overview	5
Research Gap	6
1.5 Hypotheses and Expected Benefits	6
2. Summary Of Research Project	7
Architecture Diagram.....	9
Database Structure.....	9
3. Changes to the Proposal.....	10
4. Project Completion Timeline	12
Description of timeline.....	13
Milestones and Deliverables	15
Team Responsibilities	15
5. Implemented Feature	16
Feature 1: Demand Forecasting using ML.NET SSA.....	16
Feature 2: Vendor Recommendation, Relationship Score, and Distance-Based Matching	41
Feature 3: Realtime chatmodule	57
Feature 3: Forecast Export Functionality	75
Feature 4: Role-Based Authentication (Registration & Login).....	79
Feature 5: Farmer Inventory Management	91
Feature 6: Vendor Demand Request System	113
Feature 6: Dispatch Management Module.....	121
Feature 7: Dashboard Analytics and Status Badge System.....	131
6. Evaluation Technique	133
Forecasting model evaluation	133

Comparison between ML.NET SSA model and Moving Average model.....	134
Recommendation Logic Evaluation	137
Functional and Integration Testing	137
Usability Evaluation	138
7. Reflections/Discussions.....	139
Challenges Faced	139
Lessons Learned	139
Most Satisfying Parts.....	139
Future Improvements	139
8. AI Use Section.....	140
Summary of AI use	144
Appendix – Prompt History	145
9. Work Date/Hours logs for student	149
10.Concluding Remarks	159
11.References.....	160
12.Appendix A: Installation Guide	161
13.Appendix B: User Guide	164

1. Introduction

1.1 Background and Domain Context

Local and small-scale agricultural supply chains continue to struggle with challenges such as demand uncertainty, inventory mismatch, food waste, overproduction, under-supply, communication gap between producer and vendor. These challenges are basically visible in British Columbia as many farmers and buyers still depend on informal communication like phone calls, text messages or word of mouth to coordinate with their business partners. This lack of communication may result in delays, inefficient dispatch planning, and poor visibility of demand and supply status.

With increasing interest in digital transformation, there is a clear opportunity to develop a lightweight web-based platform that supports farmers and vendors through structured workflows, shared information, and decision-support features. Many currently available systems are either too enterprise-focused, too expensive, or not designed for the needs of small and medium-scale agricultural stakeholders. This project addresses that gap by designing and implementing FarmVendor, a role-based digital platform that supports farmer–vendor coordination, demand forecasting, vendor recommendation, and real-time messaging.

1.2 Problem Definition

This research addresses the lack of a structured, secure, and intelligent coordination platform for small agricultural stakeholders. The problem includes the following limitations:

- farmers do not have a clear way to view vendor demand in one place
- vendors do not have a structured method to submit requests and track deliveries
- dispatch planning is not supported by demand forecasting
- decision-making for farmers lacks data-driven vendor recommendations
- communication between farmers and vendors is often informal and fragmented

1.3 Research Questions

Following research questions are supposed to be addressed by the project:

- How can a role-based digital platform improve coordination between farmers and vendors?
- How can simulated and user-generated data support early-stage demand forecasting?
- How can machine-learning-based forecasting improve dispatch planning?
- How can farmers be supported with vendor recommendations based on predicted demand and available inventory?
- Can a real-time chat feature improve coordination and usability within the supply chain workflow?

1.4 Literature Overview and Research Gap

Literature Overview

Research in agriculture supply chain management highlights the importance of demand forecasting, inventory visibility, and coordination between farmers and vendors. Prior studies show that even basic forecasting techniques prioritised with time and structured communication should reduce food waste, improve supply planning and increase efficiency in agricultural operations. Digital platforms are increasingly used to support these goals by enabling data sharing, record keeping and making decisions on data.

There are several farm management tools and other platforms that partially support those challenges. Here is background research and limitations on existing applications.

- **AgriWebb:** Farm management mobile application that focus on helping farmers manage production records, operational planning and inventory. offer limited direct coordination with vendors such as grocery stores and restaurants
- **FarmLogs:** largely farmer-centric mobile application with good internal visibility. limited vendor supportive.
- **CropIn:** web based Agri-intelligence and AI driven platform for enterprises across the agriculture value chain. Designed only for enterprise level Agri businesses
- **AgriDigital:** Provide cloud-based grain supply chain and management platform. Use blockchain technology to secure settlement while digitize grain storage,

trading, logistics and financing. Less suitable for small and medium scale farmers where developed models need extensive historical data and long-term adoption.

A key limitation across both academic literature and existing mobile applications and platforms is the high dependability on large historic datasets. While many optimization and forecasting models assume the availability of long-term, real world transactional data newly deployed systems and small – scale agricultural networks face a cold-start problem, where little or no historical data exists.

Most of systems ignore human decision making while fully automated recommendations prioritized where agriculture grow with trust, long-term relationships and farmer experience is vital in decision making and planning.

Research Gap

Below gap remains based on existing research and platforms,

- Limited availability of lightweight, role-based platforms designed specifically for local farmers and vendors.
- Insufficient support for incremental data collection, where forecasting improves gradually over time.
- Lack of optimization models that combines distance, requested quantity and historical relationships.
- Minimal focus on human-in-the-loop decision support rather than full automation.
- Overdependence on real-world, large-scale datasets that are impractical for early-stage systems.

1.5 Hypotheses and Expected Benefits

The project is based on the following assumptions:

- structured demand communication reduces uncertainty for farmers
- forecasting can improve dispatch planning
- recommendation logic can support better vendor selection
- real-time communication can reduce coordination friction

- simulated and user-generated data can support meaningful early-stage research and evaluation

Expected benefits include:

- better visibility of demand and supply
- improved dispatch planning
- reduced communication barriers
- better farmer decision support

2. Summary Of Research Project

The FarmVendor project is a full-stack, role-based web application designed to enhance coordination between farmers and vendors by integrating demand management, forecasting, and intelligent decision support into a unified platform. The system enables farmers to manage inventory, monitor product availability, and make dispatch decisions, while vendors can submit demand requests specifying required quantities and delivery timelines.

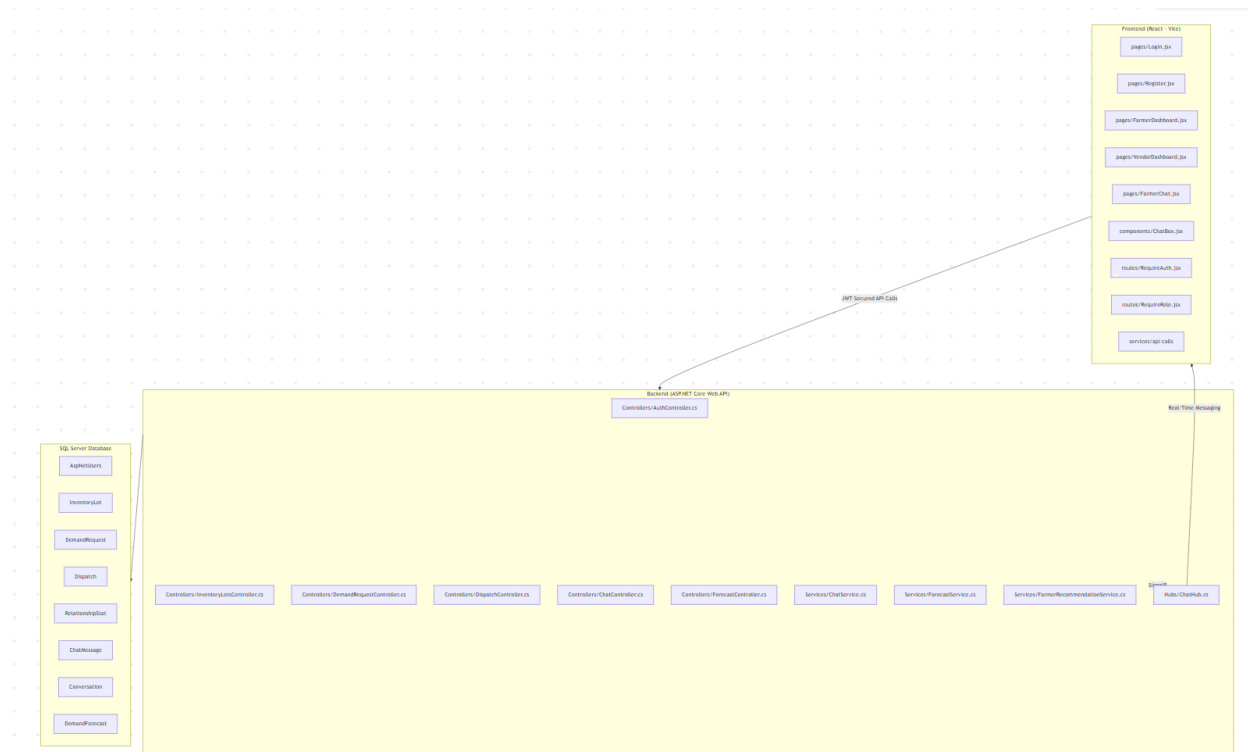
A central feature of the project is the integration of machine learning–based demand forecasting using the ML.NET Singular Spectrum Analysis (SSA) algorithm. This forecasting component analyzes historical demand request data to generate future demand predictions for specific vendor–product combinations. These predictions support proactive planning by helping farmers anticipate demand, reduce produce waste, and optimize inventory and dispatch strategies.

Building on the forecasting capability, the system incorporates an intelligent vendor recommendation module that identifies the most suitable vendors for dispatching. This module evaluates multiple factors, including forecasted demand, available farmer inventory, geographical distance between farmers and vendors, and historical relationship scores derived from past interactions. A weighted scoring mechanism is applied to rank vendors, allowing farmers to make informed, data-driven decisions when selecting vendors for fulfillment.

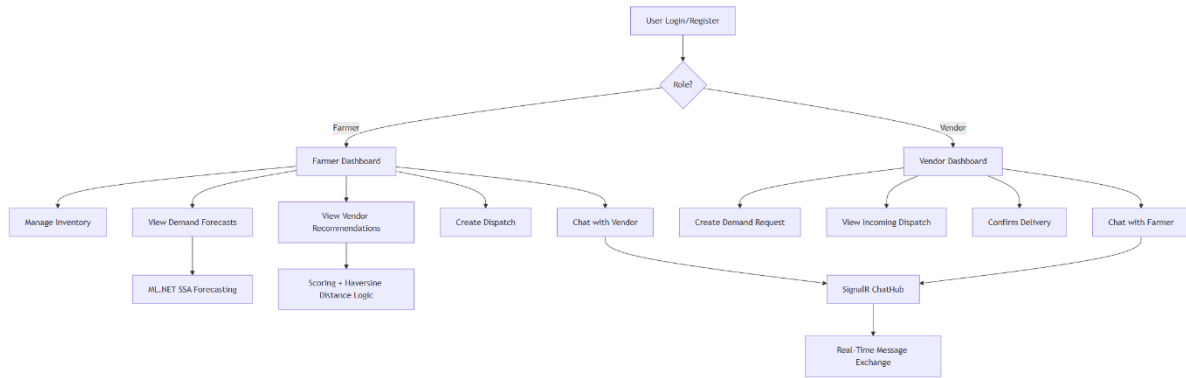
To enhance collaboration and operational efficiency, the platform also includes a real-time communication module implemented using SignalR, enabling direct and instant messaging between farmers and vendors. This feature supports timely coordination, negotiation, and clarification during the dispatch process.

The application is developed using a modern and scalable technology stack, with React (Vite) for the frontend, ASP.NET Core (.NET 8) for backend services, and SQL Server for data persistence. The system follows a layered architectural approach, clearly separating the presentation layer, business logic, and data access components to improve modularity, maintainability, and scalability.

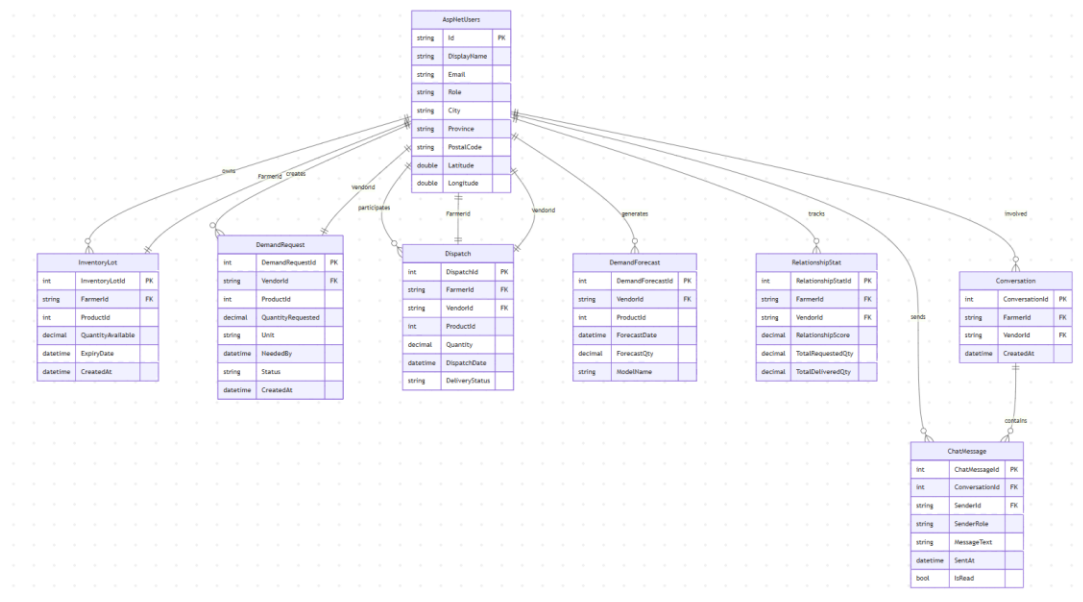
Overall, the FarmVendor project transforms a traditional transactional system into an intelligent, data-driven agricultural supply chain platform by integrating forecasting, recommendation algorithms, and real-time communication. The project demonstrates how emerging technologies can be applied to improve efficiency, reduce uncertainty, and support informed decision-making in agricultural supply chain management.



Architecture Diagram



Database Structure



3. Changes to the Proposal

During the implementation phase of the FarmVendor project, several modifications were introduced to enhance system functionality, usability, and alignment with real-world agricultural requirements. These changes were informed by iterative testing, technical feasibility analysis, and the need to support intelligent decision-making for non-technical users. Overall, the refinements strengthened the original proposal by improving scalability, interpretability, and practical applicability.

Change Area	Description	Justification
Forecast Model	The initially proposed simple logic-based demand estimation and Moving Average algorithm was replaced with a machine learning-based forecasting model using ML.NET Singular Spectrum Analysis (SSA).	The SSA model provides more accurate, data-driven, and scalable demand forecasts by analyzing historical time-series data, enabling better production planning and dispatch decisions.
Forecasting Integration (New Addition)	ML-based demand forecasting was fully embedded into the operational workflow and used as a core input for vendor recommendations.	Integrating forecasting directly into decision support improved system intelligence and ensured that recommendations were driven by future demand rather than static historical data.
Dashboard Architecture	The dashboard was upgraded from a static UI design to a fully dynamic, API-driven architecture supporting real-time data updates.	This change improved system responsiveness and enabled real-time visibility of inventory, requests, forecasts, and recommendations, enhancing user decision-making.
Authentication and Security	Role-based authentication was implemented using ASP.NET Identity with JWT-based authorization.	Secure role separation was required to protect sensitive data and enforce access control between farmer and vendor functionalities.

Recommendation Logic	The original optimization-based approach was simplified into a weighted scoring-based recommendation system.	The scoring approach improved interpretability and usability, making recommendation logic easier for non-technical agricultural users to understand and trust by prioritizing demand match, distance and relationship score which calculated score by weighted formula
Relationship Tracking Enhancement	A new relationship-tracking entity was introduced to record historical farmer–vendor interactions and compute a relationship score.	Including relationship history improved recommendation accuracy by accounting for prior collaboration performance and trust which mainly focused on Total Requested Quantity, Total Delivered Quantity, Relationship Score and finally calculating relationship Stats using EF Core aggregation queries
Location Feature	Latitude and longitude fields were added to user profiles to support geospatial calculations.	Geolocation data is required to compute distances between farmers and vendors, which is essential for distance-based ranking and logistics optimization. Here I used Haversine formula with fields Longitude and Latitude
Data Generation Strategy	The initial simulated dataset approach was enhanced to include multi-period historical demand, inventory, and dispatch data.	More realistic historical data was necessary to support reliable machine learning forecasting and to reflect real-world agricultural scenarios. Here I used SQL scripts to generate multi-week historical data. Data seeded into DemandRequest, InventoryLot, Dispatch tables
User Experience Improvements	Searchable dropdowns, improved navigation through a shared sidebar layout, hover-based details, and enhanced input validation were introduced.	These changes significantly improved usability, reduced user errors, and enhanced overall interaction efficiency for both farmer and vendor roles.

Chat Module	A real-time communication module was added using SignalR.	Real-time messaging was required to support timely coordination, negotiations, and clarification during dispatch and fulfillment processes. Here Implementation use SignalR Hub with WebSocket transport. Client uses @microsoft/signalr to establish connection
Image Integration	Product images were integrated using the Unsplash API to dynamically display images in dropdowns and dashboards.	This enhancement improves visual clarity, user engagement, and usability. It eliminates the need for manual image uploads while providing high-quality, scalable visual content. Integrated via Unsplash REST API

4. Project Completion Timeline

Activity	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	Week 13
Requirements & Literature													
System Design & Architecture													
Backend Development													
API Development & Integration													
Milestone: progress report 1													
Frontend Integration													
Data Modeling & Simulation													
Milestone: Midterm Report and Implementation													
ML Model Training & Evaluation													
Dispatch module implementation													
Milestone: progress report 2													
Forecasting logic improvement													
Integration testing													
UI/UX polish													
Milestone: progress report 3													
Bug fixing & performance tuning													
Chat Module													
Milestone: progress report 4													
Milestone: progress report 5													
Image Integration and UI polish													
Research analysis & evaluation													
Evaluation & Documentation													
Milestone: Final Report and Implementation													

Description of timeline

The project was executed over a 13-week period, following a structured and incremental development approach that combined research, system design, implementation, testing, and evaluation.

The development process began with requirements gathering and literature review (Weeks 1–2), where the problem domain, research objectives, and system requirements were defined. This was followed by system design and architecture planning (Weeks 2–3), where the overall system structure, database schema, and API design were finalized.

The backend development and API integration phase (Weeks 2–10) formed the core of the system implementation. During this phase, key modules such as authentication, demand request management, inventory tracking, dispatch handling, and forecasting services were developed using ASP.NET Core and Entity Framework.

Parallel to backend work, frontend integration (Weeks 2–10) was carried out using React, enabling dynamic interaction with backend APIs and real-time data visualization.

The data modeling and simulation phase (Weeks 3–9) focused on generating realistic historical datasets for demand, inventory, and dispatch operations. This was essential to support machine learning forecasting.

The machine learning phase (Weeks 6–10) involved implementing and evaluating forecasting models. Initially, a Moving Average approach was used, which was later enhanced by integrating the ML.NET SSA model to improve prediction accuracy.

Several key milestones were achieved during the project:

- Progress Report 1 (Week 4) – Initial design and backend setup
- Midterm Report & Implementation (Week 5–7) – Core system functionality completed
- Progress Report 2 (Week 7–8) – Forecasting and dispatch modules
- Progress Report 3 (Week 9) – UI/UX and integration progress
- Progress Report 4 (Week 10) – Chat module and performance improvements
- Progress Report 5 (Week 11) – Final system enhancements

Advanced features such as the chat module using SignalR (Weeks 9–11) and image integration using the Unsplash API (Weeks 11–12) were implemented in later stages to enhance usability and system interaction.

The final stages of the project included:

- Integration testing (Weeks 9–12)
- UI/UX improvements (Weeks 9–13)
- Bug fixing and performance tuning (Weeks 9–12)
- Research analysis and evaluation (Weeks 11–13)
- Final documentation and report preparation (Weeks 12–13)

Milestones and Deliverables

Milestone	Deliverable
Progress Report 1	Requirements, system design, initial backend setup
Midterm Report	Functional backend + partial frontend
Progress Report 2	Dispatch + forecasting modules
Progress Report 3	UI improvements + integration
Progress Report 4	Chat module + performance optimization
Progress Report 5	Final system features completed
Final Report	Full system + evaluation + documentation

Team Responsibilities

This project was completed individually by **Maneesha Eeshwara**, who was responsible for all aspects of the system, including:

- Backend development using ASP.NET Core
- Database design and Entity Framework integration
- Machine learning model implementation (ML.NET SSA)
- Frontend development using React
- API integration and authentication (JWT)
- Real-time chat implementation using SignalR
- UI/UX design improvements
- Testing, debugging, and performance optimization
- Final report preparation and documentation

5. Implemented Feature

Feature 1: Demand Forecasting using ML.NET SSA

This is one of the most significant features of the final system. It predicts future vendor demand using historical demand data and supports proactive dispatch planning.

This feature includes:

Forecast generation using Moving Average

Forecast generation using ML.NET SSA

Forecast results table

Forecast line charts

Multi-vendor comparison chart

Filtered vendor/product selection

The design consists of:

- Auto-loaded chart view
- Product and vendor filters
- Forecast generation controls
- CSV export
- Top vendor comparison visualization

Implementation

Frontend

- FarmerForecast.jsx
- ForecastLineChart.jsx

- MultiVendorForecastLineChart.jsx
- forecastApi.js

Backend

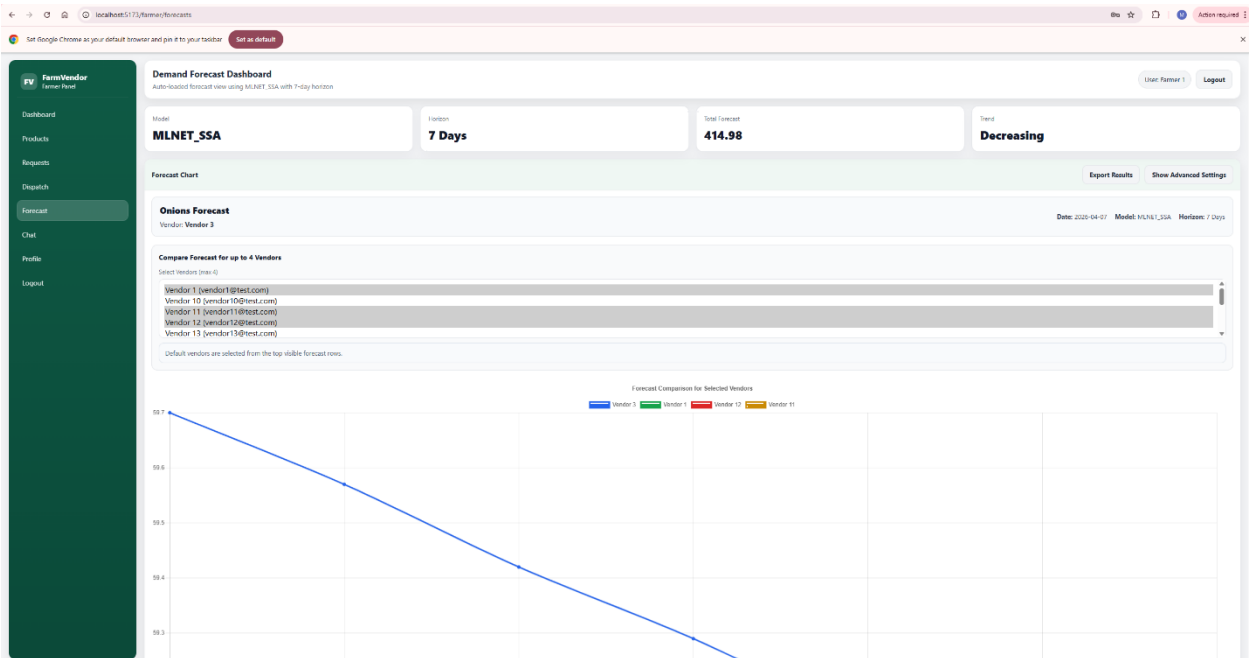
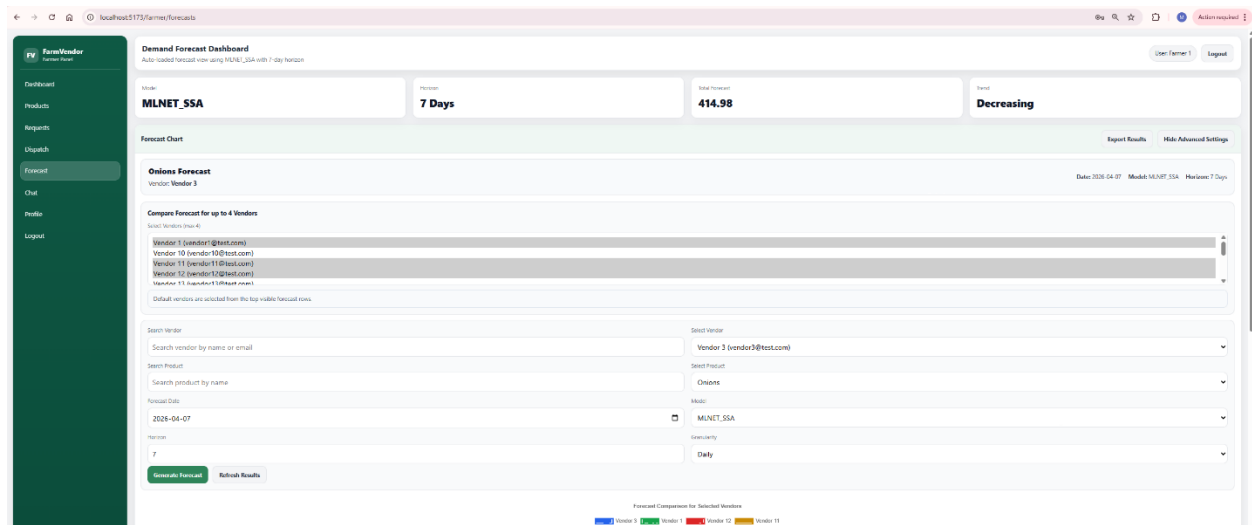
- DemandForecastController.cs
- DemandForecastService.cs
- MLDemandForecastingEngine.cs
- filtered vendor/product API endpoints for the logged-in farmer

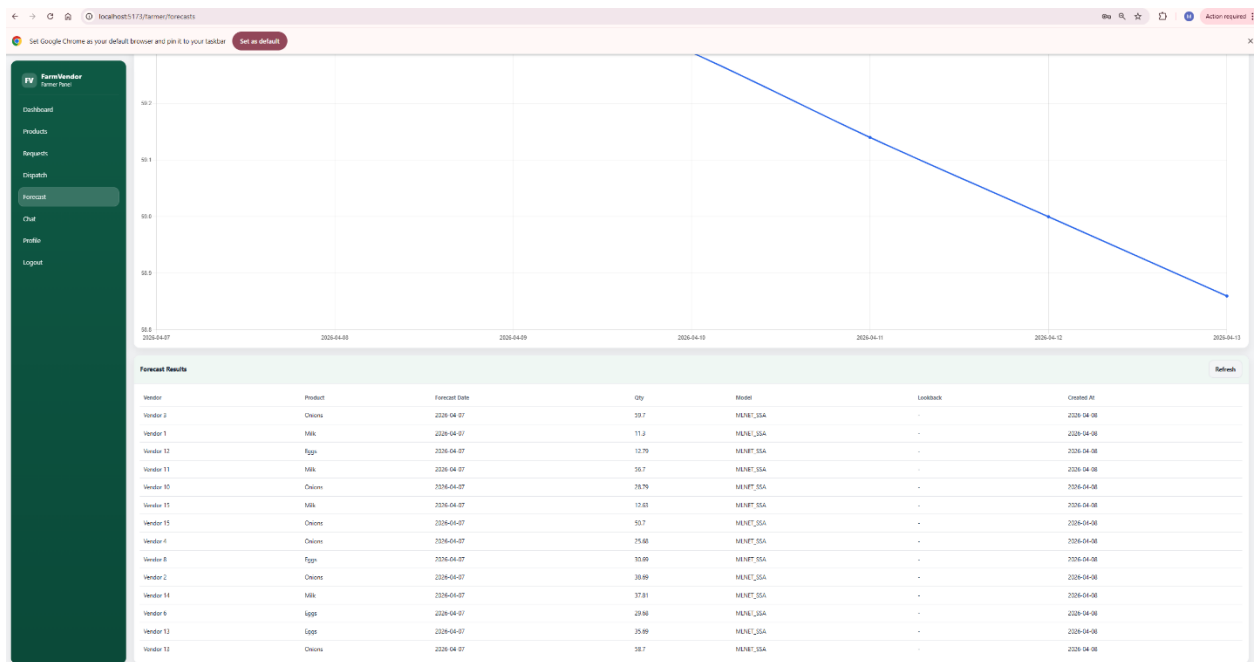
Database

- DemandForecast table storing:
 - VendorId
 - ProductId
 - ForecastDate
 - ForecastQty
 - ModelName
 - LookbackPeriods
 - CreatedAt

Technical Methods

- Moving Average based on recent demand history
- ML.NET SSA time-series forecasting
- historical demand grouped by VendorId + ProductId
- chart comparison using shared time axis





Frontend

- *FarmerForecast.jsx*

```
1 import React, { useEffect, useLocalStorage, useNavigate, useState } from "react";
2 import { useNavigate } from "react-router-dom";
3 import "../styles/dashboard.css";
4 import {
5   generateForecasts,
6   getForecasts,
7   getForecastModels,
8   getForecastChartData,
9   getForecastVendors,
10  getForecastProducts,
11 } from "../assets/forecastApi";
12 import ForecastLineChart from "../assets/components/ForecastLineChart";
13 import MultiVendorForecastLineChart from "../assets/components/MultiVendorForecastLineChart";
14 import FarmerSidebar from "../assets/components/FarmerSidebar";
15
16 export default function FarmerAnalytics() {
17   const navigate = useNavigate();
18   const token = localStorage.getItem("token");
19   const displayName = localStorage.getItem("displayName") || "Farmer";
20
21   const [error, setError] = useState("");
22   const [success, setSuccess] = useState("");
23   const [loading, setLoading] = useState(false);
24   const [loadingRows, setloadingRows] = useState(false);
25   const [loadingChart, setLoadingChart] = useState(false);
26
27   const [rows, setRows] = useState([]);
28   const [models, setModels] = useState([]);
29   const [chartData, setChartData] = useState([]);
30
31   const [vendors, setVendors] = useState([]);
32   const [products, setProducts] = useState([]);
33
34   const [vendorSearch, setVendorSearch] = useState("");
35   const [productSearch, setProductSearch] = useState("");
36
37   const [advancedOpen, setAdvancedOpen] = useState(false);
38
39   const [form, setForm] = useState({
40     forecastDate: new Date().toISOString().slice(0, 10),
41     modelName: "MLNET_SSA",
42     lookbackPeriods: 3,
43     horizon: 7,
44     granularity: "Daily",
45     vendorId: "",
46   });
47
48   const [chartForm, setChartForm] = useState({
49     vendorId: "",
50     productId: "",
51     forecastDate: new Date().toISOString().slice(0, 10),
52   });
53
54   //for multi line charts
55   const [selectedVendorIds, setSelectedVendorIds] = useState([]);
56   const [multiVendorChartRows, setMultiVendorChartRows] = useState([]);
57   const [loadingMultiChart, setloadingMultiChart] = useState(false);
58
59   const logout = () => {
60     localStorage.clear();
61     navigate("/login");
62   };
63
64   const loadModels = async () => {
65     try {
66       const data = await getForecastModels(token);
67       setModels(data);
68     } catch (error) {
69       console.log(error);
70     }
71   };
72 }
```



```

v FarmerForecast.jsx x MSSQL: Welcome & What's New
4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages >
export default function FarmerAnalytics() {
  const logout = () => {
    navigate("/login");
  };

  const loadModels = async () => {
    try {
      const data = await getForecastModels(token);
      setModels(data);
    } catch {
      setModels([]);
    }
  };

  const loadVendors = async (search = "") => {
    try {
      const data = await getForecastVendors(token, search);
      setVendors(data);
    } catch {
      setVendors([]);
    }
  };

  const loadProducts = async (search = "") => {
    try {
      const data = await getForecastProducts(token, search);
      setProducts(data);
    } catch {
      setProducts([]);
    }
  };

  const loadChart = async (override = null) => {
    const payload = override || chartForm;

    setError("");
    setLoadingChart(true);
    setChartData([]);

    if (!payload.vendorId || !payload.productId) {
      setLoadingChart(false);
      return;
    }

    try {
      const data = await getForecastChartData(
        {
          vendorId: payload.vendorId,
          productId: Number(payload.productId),
          forecastDate: payload.forecastDate,
          modelName: "MLNET_SSA",
        },
        token
      );
      setChartData(data);
    } catch (e) {
      setError(e?.message || "Failed to load chart");
    } finally {
      setLoadingChart(false);
    }
  };

  const applyFirstValidForecastRowToChart = (forecastRows) => {
    if (!forecastRows || forecastRows.length === 0) return;

    const first = forecastRows[0];

```

```

v FarmerForecast.jsx x MSSQL: Welcome & What's New
4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerForecast.jsx
export default function FarmerAnalytics() {
  const loadChart = async (override = null) => {
    };

    const applyFirstValidForecastRowToChart = (forecastRows) => {
      if (!forecastRows || forecastRows.length === 0) return;

      const first = forecastRows[0];

      if (!first?.vendorId || !first?.productId || !first?.forecastDate) return;

      const nextChartForm = {
        vendorId: first.vendorId,
        productId: String(first.productId),
        forecastDate: new Date(first.forecastDate).toISOString().slice(0, 10),
      };

      setChartForm(nextChartForm);
      setForm((prev) => ({
        ...prev,
        vendorId: first.vendorId,
        forecastDate: new Date(first.forecastDate).toISOString().slice(0, 10),
      }));

      loadChart(nextChartForm);
    };

    const loadForecastRows = async (forecastDateArg, modelNameArg) => {
      setLoadingRows(true);
      setError("");
      try {
        const requestedDate = forecastDateArg || form.forecastDate;
        const requestedModel = modelNameArg || form.modelName;

        const data = await getForecasts(
          {
            forecastDate: requestedDate,
            modelName: requestedModel,
          },
          token
        );

        setRows(data);

        if (requestedModel === "MLNET_SSA" && data.length > 0) {
          applyFirstValidForecastRowToChart(data);
        } else if (data.length === 0) {
          setChartData([]);
        }
      } catch (e) {
        setError(e?.message || "Failed to load forecasts");
      } finally {
        setLoadingRows(false);
      }
    };

    const handleGenerate = async () => {
      setLoading(true);
      setError("");
      setSuccess("");

      try {
        const payload =
          form.modelName === "MovingAverage"
            ? {
                forecastDate: form.forecastDate,
                modelName: form.modelName,

```

```

v FarmerForecast.jsx x MSSQL: Welcome & What's New
4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerForecast.jsx > FarmerAnalytics
export default function FarmerAnalytics() {
  const handleGenerate = async () => {

    try {
      const payload =
        form.modelName === "MovingAverage"
        ? {
            forecastDate: form.forecastDate,
            modelName: form.modelName,
            lookbackPeriods: Number(form.lookbackPeriods),
            vendorId: form.vendorId || null,
          }
        : {
            forecastDate: form.forecastDate,
            modelName: form.modelName,
            horizon: Number(form.horizon),
            granularity: form.granularity,
            vendorId: form.vendorId || null,
          };

      const result = await generateForecasts(payload, token);

      setSuccess(
        `Forecast generation completed. Model: ${result.modelName}, Rows: ${result.forecastCount}`
      );

      await loadModels();
      await loadForecastRows(form.forecastDate, form.modelName);
    } catch (e) {
      setError(e?.message || "Failed to generate forecasts");
    } finally {
      setLoading(false);
    }
  };

  const escapeCsv = (value) => {
    if (value === null || value === undefined) return "";
    const str = String(value);
    if (str.includes("'") || str.includes(",") || str.includes("\n")) {
      return `"${str.replace(/"/g, '""')}"`;
    }
    return str;
  };

  const exportForecastResultsCsv = () => {
    if (!rows || rows.length === 0) {
      setError("No forecast rows available to export.");
      return;
    }

    const header = [
      "Vendor",
      "Product",
      "Forecast Date",
      "Qty",
      "Model",
      "Lookback",
      "Created At",
    ];

    const csvRows = rows.map((r) => [
      r.vendorName || r.vendorId,
      r.productName,
      new Date(r.forecastDate).toISOString().slice(0, 10),
      r.forecastQty,
    ]);
  };
}

```

- *ForecastLineChart.jsx*

```

1  import React from "react";
2  import {
3    Chart as ChartJS,
4    CategoryScale,
5    LinearScale,
6    PointElement,
7    LineElement,
8    Title,
9    Tooltip,
10   Legend,
11 } from "chart.js";
12 import { Line } from "react-chartjs-2";
13
14 ChartJS.register(
15   CategoryScale,
16   LinearScale,
17   PointElement,
18   LineElement,
19   Title,
20   Tooltip,
21   Legend
22 );
23
24 export default function ForecastLineChart({ chartPoints }) {
25   if (!chartPoints || chartPoints.length === 0) {
26     return <div>No chart data available.</div>;
27   }
28
29   const allDates = [...new Set(chartPoints.map((p) => p.date))].sort();
30
31   const historicalMap = {};
32   const forecastMap = {};
33
34   chartPoints.forEach((p) => {
35     if (p.series === "Historical") historicalMap[p.date] = p.quantity;
36     if (p.series === "Forecast") forecastMap[p.date] = p.quantity;
37   });
38
39   const historicalData = allDates.map((d) =>
40     historicalMap[d] !== undefined ? historicalMap[d] : null
41   );
42
43   const forecastData = allDates.map((d) =>
44     forecastMap[d] !== undefined ? forecastMap[d] : null
45   );
46
47   const data = {
48     labels: allDates,
49     datasets: [
50       {
51         label: "Historical Demand",
52         data: historicalData,
53         borderColor: "#2f855a",
54         backgroundColor: "#2f855a",
55         tension: 0.3,
56         spanGaps: true,
57       },
58       {
59         label: "Forecast",
60         data: forecastData,
61         borderColor: "#2563eb",
62         backgroundColor: "#2563eb",
63         borderDash: [6, 6],
64         tension: 0.3,
65         spanGaps: true,
66       },
67     ],
68   };
69 }

```

- *MultiVendorForecastLineChart.jsx*

```

1  import React from "react";
2  import {
3    Chart as ChartJS,
4    CategoryScale,
5    LinearScale,
6    PointElement,
7    LineElement,
8    Title,
9    Tooltip,
10   Legend,
11 } from "chart.js";
12 import { Line } from "react-chartjs-2";
13
14 ChartJS.register(
15   CategoryScale,
16   LinearScale,
17   PointElement,
18   LineElement,
19   Title,
20   Tooltip,
21   Legend
22 );
23
24 const COLORS = ["#2563eb", "#16a34a", "#dc2626", "#ca8a04"];
25
26 export default function MultiVendorForecastLineChart({ chartRows = [], vendorNames = [] }) {
27   if (!chartRows || chartRows.length === 0) {
28     return <div>No multi-vendor chart data available.</div>;
29   }
30
31   const labels = chartRows.map((r) => r.date);
32
33   const datasets = vendorNames.map((vendorName, index) => ({
34     label: vendorName,
35     data: chartRows.map((row) =>
36       row[vendorName] !== undefined ? row[vendorName] : null
37     ),
38     borderColor: COLORS[index % COLORS.length],
39     backgroundColor: COLORS[index % COLORS.length],
40     tension: 0.3,
41     spanGaps: true,
42   }));
43
44   const data = {
45     labels,
46     datasets,
47   };
48
49   const options = {
50     responsive: true,
51     plugins: {
52       legend: {
53         position: "top",
54       },
55       title: {
56         display: true,
57         text: "Forecast Comparison for Selected Vendors",
58       },
59     },
60   };
61
62   return <Line data={data} options={options} />;
63 }

```

- ForecastApi.js

```

1  const API_BASE = import.meta.env.VITE_API_URL;
2
3  async function parseResponse(res, defaultMessage) {
4    const text = await res.text();
5
6    if (!res.ok) {
7      throw new Error(text || defaultMessage);
8    }
9
10   if (!text) return null;
11
12   try {
13     return JSON.parse(text);
14   } catch {
15     throw new Error("Invalid JSON response from server");
16   }
17 }
18
19 export async function generateForecasts(payload, token) {
20   const res = await fetch(`${API_BASE}/api/forecasts/generate`, {
21     method: "POST",
22     headers: {
23       "Content-Type": "application/json",
24       Authorization: `Bearer ${token}`,
25     },
26     body: JSON.stringify(payload),
27   });
28   return parseResponse(res, "Failed to generate forecast");
29 }
30
31 export async function getForecasts(params, token) {
32   const url = new URL(`${API_BASE}/api/forecasts`);
33
34   if (params?.forecastDate) {
35     url.searchParams.append("forecastDate", params.forecastDate);
36   }
37
38   if (params?.modelName) {
39     url.searchParams.append("modelName", params.modelName);
40   }
41
42   const res = await fetch(url, {
43     headers: {
44       Authorization: `Bearer ${token}`,
45     },
46   });
47   return parseResponse(res, "Failed to load forecasts");
48 }
49
50 export async function getForecastModels(token) {
51   const res = await fetch(`${API_BASE}/api/forecasts/models`, {
52     headers: {
53       Authorization: `Bearer ${token}`,
54     },
55   });
56   return parseResponse(res, "Failed to load model names");
57 }
58
59 export async function getForecastChartData(params, token) {
60   const url = new URL(`${API_BASE}/api/forecasts/chart`);
61
62   if (params?.vendorId) {
63     url.searchParams.append("vendorId", params.vendorId);
64   }
65
66   if (params?.productId) {
67     url.searchParams.append("productId", String(params.productId));
68   }
69
70   if (params?.forecastDate) {
71     url.searchParams.append("forecastDate", params.forecastDate);
72   }
73
74   if (params?.modelName) {
75     url.searchParams.append("modelName", params.modelName);
76   }
77
78   console.log("Calling chart API:", url.toString());
79
80   const res = await fetch(url, {
81     headers: {
82       Authorization: `Bearer ${token}`,
83     },
84   });
85   return parseResponse(res, "Failed to load chart data");
86 }
87
88 export async function getForecastVendors(token, search = "") {
89   const url = new URL(`${API_BASE}/api/forecasts/vendors`);
90
91   if (search.trim()) {
92     url.searchParams.append("search", search.trim());
93   }
94
95   const res = await fetch(url, {
96     headers: {
97       Authorization: `Bearer ${token}`,
98     },
99   });
100  return parseResponse(res, "Failed to load vendors");
101 }
102
103 export async function getForecastProducts(token, search = "") {
104   const url = new URL(`${API_BASE}/api/forecasts/products`);
105
106   if (search.trim()) {
107     url.searchParams.append("search", search.trim());
108   }
109
110   const res = await fetch(url, {
111     headers: {
112       Authorization: `Bearer ${token}`,
113     },
114   });
115   return parseResponse(res, "Failed to load products");
116 }
117
118 export async function compareForecasts(params, token) {
119   const url = new URL(`${API_BASE}/api/forecasts/compare`);
120
121   if (params?.vendorId) {
122     url.searchParams.append("vendorId", params.vendorId);
123   }
124
125   if (params?.productId) {
126     url.searchParams.append("productId", String(params.productId));
127   }
128
129   if (params?.forecastDate) {
130     url.searchParams.append("forecastDate", params.forecastDate);
131   }
132
133   if (params?.modelName) {
134     url.searchParams.append("modelName", params.modelName);
135   }
136
137   console.log("Calling compare API:", url.toString());
138
139   const res = await fetch(url, {
140     headers: {
141       Authorization: `Bearer ${token}`,
142     },
143   });
144   return parseResponse(res, "Failed to load comparison data");
145 }

```

```

export async function getForecastChartData(params, token) {
  const url = new URL(`${API_BASE}/api/forecasts/chart`);

  if (params?.vendorId) {
    url.searchParams.append("vendorId", params.vendorId);
  }

  if (params?.productId) {
    url.searchParams.append("productId", String(params.productId));
  }

  if (params?.forecastDate) {
    url.searchParams.append("forecastDate", params.forecastDate);
  }

  if (params?.modelName) {
    url.searchParams.append("modelName", params.modelName);
  }

  console.log("Calling chart API:", url.toString());

  const res = await fetch(url, {
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

  return parseResponse(res, "Failed to load chart data");
}

export async function getForecastVendors(token, search = "") {
  const url = new URL(`${API_BASE}/api/forecasts/vendors`);

  if (search.trim()) {
    url.searchParams.append("search", search.trim());
  }

  const res = await fetch(url, {
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

  return parseResponse(res, "Failed to load vendors");
}

export async function getForecastProducts(token, search = "") {
  const url = new URL(`${API_BASE}/api/forecasts/products`);

  if (search.trim()) {
    url.searchParams.append("search", search.trim());
  }

  const res = await fetch(url, {
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

  return parseResponse(res, "Failed to load products");
}

export async function compareForecasts(params, token) {
  const url = new URL(`${API_BASE}/api/forecasts/compare`);

  if (params?.vendorId) {
    url.searchParams.append("vendorId", params.vendorId);
  }

  if (params?.productId) {
    url.searchParams.append("productId", String(params.productId));
  }

  if (params?.forecastDate) {
    url.searchParams.append("forecastDate", params.forecastDate);
  }

  if (params?.modelName) {
    url.searchParams.append("modelName", params.modelName);
  }

  console.log("Calling compare API:", url.toString());

  const res = await fetch(url, {
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

  return parseResponse(res, "Failed to load comparison data");
}

```

Backend

- DemandForecastController

```
1 using FarmVendor.Api.Models.DTOS;
2 using FarmVendor.Api.Services;
3 using Microsoft.AspNetCore.Authorization;
4 using Microsoft.AspNetCore.Mvc;
5 using System.Security.Claims;
6
7 namespace FarmVendor.Api.Controllers;
8
9 [ApiController]
10 [Route("api/forecasts")]
11 [Authorize]
12 public class DemandForecastController : ControllerBase
13 {
14     private readonly DemandForecastService _forecastService;
15
16     public DemandForecastController(DemandForecastService forecastService)
17     {
18         _forecastService = forecastService;
19     }
20
21     private string GetUserId()
22     => User.FindFirstValue(ClaimTypes.NameIdentifier) ?? "";
23
24     // POST: /api/forecasts/generate
25     [HttpPost("generate")]
26     public async Task<IActionResult> GenerateForecasts([FromBody] GenerateForecastDto dto)
27     {
28         var farmerId = GetUserId();
29         if (string.IsNullOrWhiteSpace(farmerId))
30             return Unauthorized();
31
32         if (dto.ForecastDate == default)
33             return BadRequest("ForecastDate is required.");
34
35         if (string.IsNullOrWhiteSpace(dto.ModelName))
36             dto.ModelName = "MLNET_SSA";
37
38         List<FarmVendor.Api.Models.DemandForecast> forecasts;
39
40         if (dto.ModelName.Equals("MovingAverage", StringComparison.OrdinalIgnoreCase))
41         {
42             if (dto.LookbackPeriods <= 0)
43                 return BadRequest("LookbackPeriods must be greater than 0.");
44
45             forecasts = await _forecastService.GenerateMovingAverageForecastsForFarmerAsync(
46                 farmerId,
47                 dto.ForecastDate,
48                 dto.LookbackPeriods
49             );
50         }
51         else if (dto.ModelName.Equals("MLNET_SSA", StringComparison.OrdinalIgnoreCase))
52         {
53             if (dto.Horizon <= 0)
54                 return BadRequest("Horizon must be greater than 0.");
55
56             forecasts = await _forecastService.GenerateMlForecastsForFarmerAsync(
57                 farmerId,
58                 dto.ForecastDate,
59                 dto.Horizon,
60                 dto.Granularity ?? "Daily"
61             );
62         }
63         else
64         {
65             return BadRequest("Unsupported modelName. Use 'MovingAverage' or 'MLNET_SSA'.");
66         }
67     }
68 }
```

```

DemandForecastController.cs x MSSQL: Welcome & What's New
495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > DemandForecastController.cs
{
}

var saved = await _forecastService.SaveForecastsAsync(forecasts);

return Ok(new
{
    message = "Forecast generation completed.",
    modelName = dto.ModelName,
    forecastCount = forecasts.Count,
    savedRows = saved
});

// GET: /api/forecasts?forecastDate=2026-04-05&modelName=MLNET_SSA
[HttpGet]
public async Task<ActionResult<IEnumerable<DemandForecastRowDto>>> GetForecasts(
    [FromQuery] DateTime? forecastDate,
    [FromQuery] string? modelName)
{
    var requestedDate = forecastDate ?? DateTime.UtcNow.Date;
    var requestedModel = string.IsNullOrEmpty(modelName)
        ? "MLNET_SSA"
        : modelName.Trim();

    var rows = await _forecastService.GetForecastRowsAsync(requestedDate, requestedModel);
    return Ok(rows);
}

// GET: /api/forecasts/chart?vendorId=abc&productId=1&forecastDate=2026-04-05&modelName=MLNET_SSA
[HttpGet("chart")]
public async Task<ActionResult> GetForecastChartData(
    [FromQuery] string vendorId,
    [FromQuery] int productId,
    [FromQuery] DateTime? forecastDate,
    [FromQuery] string modelName = "MLNET_SSA")
{
    if (string.IsNullOrEmpty(vendorId))
        return BadRequest("VendorId is required.");

    if (productId <= 0)
        return BadRequest("ProductId is required.");

    if (forecastDate == default)
        return BadRequest("ForecastDate is required.");

    var data = await _forecastService.GetForecastChartDataAsync(
        vendorId,
        productId,
        forecastDate,
        modelName);

    return Ok(data);
}

// GET: /api/forecasts/vendors?search=vendor
// Only vendors relevant to logged-in farmer's available products
[HttpGet("vendors")]
public async Task<ActionResult> GetVendors([FromQuery] string? search = null)
{
    var farmerId = GetUserId();
    if (string.IsNullOrEmpty(farmerId))
        return Unauthorized();

    var vendors = await _forecastService.GetForecastVendorsForFarmerAsync(farmerId, search);
    return Ok(vendors);
}

```

```

DemandForecastController.cs x MSSQL: Welcome & What's New
4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > DemandForecastController.cs
{
    var data = await _forecastService.GetForecastChartDataAsync(
        return Ok(data);
    }
}

// GET: /api/forecasts/vendors?search=vendor
// Only vendors relevant to logged-in farmer's available products
[HttpGet("vendors")]
public async Task<ActionResult> GetVendors([FromQuery] string? search = null)
{
    var farmerId = GetUserId();
    if (string.IsNullOrEmpty(farmerId))
        return Unauthorized();

    var vendors = await _forecastService.GetForecastVendorsForFarmerAsync(farmerId, search);
    return Ok(vendors);
}

// GET: /api/forecasts/products?search=milk
// Only products available in logged-in farmer inventory
[HttpGet("products")]
public async Task<ActionResult> GetProducts([FromQuery] string? search = null)
{
    var farmerId = GetUserId();
    if (string.IsNullOrEmpty(farmerId))
        return Unauthorized();

    var products = await _forecastService.GetForecastProductsForFarmerAsync(farmerId, search);
    return Ok(products);
}

// GET: /api/forecasts/models
[HttpGet("models")]
public IActionResult GetModelNames()
{
    return Ok(new[] { "MLNET_SSA", "MovingAverage" });
}

// GET: /api/forecasts/debug-history
[HttpGet("debug-history")]
public async Task<ActionResult> DebugHistory()
{
    var farmerId = GetUserId();
    if (string.IsNullOrEmpty(farmerId))
        return Unauthorized();

    var pairs = await _forecastService.GetEligiblePairsForFarmerAsync(farmerId, DateTime.UtcNow);

    return Ok(new
    {
        farmerId,
        eligiblePairCount = pairs.Count,
        pairs
    });
}
}

```

- *DemandForecastService.cs*

```

1  using FarmVendor.Api.Data;
2  using FarmVendor.Api.Models;
3  using FarmVendor.Api.Models.DTOS;
4  using FarmVendor.Api.Services.Forecasting;
5  using Microsoft.EntityFrameworkCore;
6
7  namespace FarmVendor.Api.Services;
8
9  public class DemandForecastService
10 {
11     private readonly AppDbContext _db;
12     private readonly MLDemandForecastingEngine _mlEngine;
13
14     public DemandForecastService(AppDbContext db)
15     {
16         _db = db;
17         _mlEngine = new MLDemandForecastingEngine();
18     }
19
20     // =====
21     // 0) GET ELIGIBLE FORECAST PAIRS FOR A LOGGED-IN FARMER
22     // Only products available in farmer inventory
23     // Only vendors who requested those products
24     // =====
25     public async Task<List<EligibleForecastPairDto>> GetEligiblePairsForFarmerAsync(string farmerId, DateTime forecastDate)
26     {
27         var availableProductIds = await _db.InventoryLot
28             .AsNoTracking()
29             .Where(i =>
30                 i.FarmerId == farmerId &&
31                 i.QuantityAvailable > 0 &&
32                 i.Product.IsActive)
33             .Select(i => i.ProductId)
34             .Distinct()
35             .ToListAsync();
36
37         if (availableProductIds.Count == 0)
38             return new List<EligibleForecastPairDto>();
39
40         var eligiblePairs = await _db.DemandRequest
41             .AsNoTracking()
42             .Where(r =>
43                 availableProductIds.Contains(r.ProductId) &&
44                 r.CreatedAt < forecastDate)
45             .Select(r => new EligibleForecastPairDto
46             {
47                 VendorId = r.VendorId,
48                 ProductId = r.ProductId
49             })
50             .Distinct()
51             .ToListAsync();
52
53         return eligiblePairs;
54     }
55
56     // =====
57     // 1) MOVING AVERAGE FORECAST
58     // FILTERED FOR FARMER'S AVAILABLE PRODUCTS
59     // =====
60     public async Task<List<DemandForecast>> GenerateMovingAverageForecastsForFarmerAsync(
61         string farmerId,
62         DateTime forecastDate,
63         int lookbackPeriods = 3)
64     {
65         if (lookbackPeriods <= 0)
66             throw new ArgumentException("LookbackPeriods must be greater than 0.");
67

```



```

56 // =====
57 // 1) MOVING AVERAGE FORECAST
58 //   FILTERED FOR FARMER'S AVAILABLE PRODUCTS
59 // =====
60 public async Task<List<DemandForecast>> GenerateMovingAverageForecastsForFarmerAsync(
61     string farmerId,
62     DateTime forecastDate,
63     int lookbackPeriods = 3)
64 {
65     if (lookbackPeriods <= 0)
66         throw new ArgumentException("LookbackPeriods must be greater than 0.");
67
68     var eligiblePairs = await GetEligiblePairsForFarmerAsync(farmerId, forecastDate);
69
70     if (eligiblePairs.Count == 0)
71         return new List<DemandForecast>();
72
73     var groupedRequests = await _db.DemandRequest
74         .AsNoTracking()
75         .Where(r => r.CreatedAt < forecastDate)
76         .OrderByDescending(r => r.CreatedAt)
77         .GroupBy(r => new { r.VendorId, r.ProductId })
78         .Select(g => new
79         {
80             g.Key.VendorId,
81             g.Key.ProductId,
82             Requests = g.OrderByDescending(x => x.CreatedAt)
83                 .Take(lookbackPeriods)
84                 .Select(x => x.QuantityRequested)
85                 .ToList()
86         })
87         .ToListAsync();
88
89     var eligibleSet = eligiblePairs
90         .Select(x => $"{x.VendorId}_{x.ProductId}")
91         .ToHashSet();
92
93     var forecasts = new List<DemandForecast>();
94
95     foreach (var group in groupedRequests)
96     {
97         var key = $"{group.VendorId}_{group.ProductId}";
98         if (!eligibleSet.Contains(key)) continue;
99         if (group.Requests.Count == 0) continue;
100
101         var avgQty = group.Requests.Average();
102
103         forecasts.Add(new DemandForecast
104         {
105             VendorId = group.VendorId,
106             ProductId = group.ProductId,
107             ForecastDate = forecastDate.Date,
108             ForecastQty = Math.Round(avgQty, 2),
109             ModelName = "MovingAverage",
110             LookbackPeriods = lookbackPeriods,
111             CreatedAt = DateTime.UtcNow
112         });
113     }
114
115     return forecasts;
116 }
117

```

```

117 // =====
118 // 2) ML.NET FORECAST
119 // FILTERED FOR FARMER'S AVAILABLE PRODUCTS
120 // =====
121 public async Task<List<DemandForecast>> GenerateMLForecastsForFarmerAsync(
122     string farmerId,
123     DateTime forecastStartDate,
124     int horizon = 7,
125     string granularity = "Daily")
126 {
127     if (horizon <= 0)
128     {
129         throw new ArgumentException("Horizon must be greater than 0.");
130     }
131     var eligiblePairs = await GetEligiblePairsForFarmerAsync(farmerId, forecastStartDate);
132
133     if (eligiblePairs.Count == 0)
134     {
135         return new List<DemandForecast>();
136     }
137     var allForecasts = new List<DemandForecast>();
138
139     foreach (var pair in eligiblePairs)
140     {
141         var series = await LoadAggregatedDemandSeriesAsync(
142             pair.ProductId,
143             pair.VendorId,
144             forecastStartDate,
145             granularity);
146
147         if (series.Count < 3)
148         {
149             Console.WriteLine(
150                 $"Skipped ML forecast for Vendor={pair.VendorId}, Product={pair.ProductId}, Reason=Not enough history, SeriesCount={
151                 continue;
152             }
153
154         var orderedValues = series
155             .OrderBy(x => x.PeriodDate)
156             .Select(x => (float)x.TotalQuantity)
157             .ToList();
158
159         try
160         {
161             var safeHorizon = Math.Min(horizon, Math.Max(1, orderedValues.Count / 2));
162             var mlResult = _mlEngine.TrainAndForecast(orderedValues, safeHorizon);
163
164             if (mlResult.Forecast == null || mlResult.Forecast.Count == 0)
165             {
166                 Console.WriteLine(
167                     $"ML forecast returned no results for Vendor={pair.VendorId}, Product={pair.ProductId}");
168                 continue;
169             }
170
171             for (int i = 0; i < mlResult.Forecast.Count; i++)
172             {
173                 DateTime nextDate =
174                     granularity.Equals("Weekly", StringComparison.OrdinalIgnoreCase)
175                     ? forecastStartDate.Date.AddDays(7 * i)
176                     : forecastStartDate.Date.AddDays(i);
177
178                 decimal qty = Math.Round(
179                     Convert.ToDecimal(Math.Max(0, mlResult.Forecast[i])), 2);
180
181                 allForecasts.Add(new DemandForecast
182                 {
183                     VendorId = pair.VendorId,
184                     ProductId = pair.ProductId,
185                     PeriodDate = nextDate,
186                     Quantity = qty
187                 });
188             }
189         }
190     }
191     return allForecasts;
192 }

```

```

.env DemandForecastService.cs MSSQL: Welcome & What's New
/26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > DemandForecastService.cs
10 {
27 {
39 {
59 {
71 {
77 decimal qty = Math.Round(
78     Convert.ToDecimal(Math.Max(0, mlResult.Forecast[i])), 2);
79
80     allForecasts.Add(new DemandForecast
81     {
82         VendorId = pair.VendorId,
83         ProductId = pair.ProductId,
84         ForecastDate = nextDate.Date,
85         ForecastQty = qty,
86         ModelName = "MLNET_SSA",
87         LookbackPeriods = null,
88         CreatedAt = DateTime.UtcNow
89     });
90     }
91 }
92 catch (Exception ex)
93 {
94     Console.WriteLine(
95         $"ML forecast failed for Vendor={pair.VendorId}, Product={pair.ProductId}. Error={ex.Message}");
96 }
97 }
98
99 Console.WriteLine($"Total ML forecasts generated for farmer {farmerId}: {allForecasts.Count}");
100 return allForecasts;
101 }
102
103 // =====
104 // 3) SAVE FORECASTS
105 // =====
106 public async Task<int> SaveForecastsAsync(List<DemandForecast> forecasts)
107 {
108     if (forecasts.Count == 0) return 0;
109
110     foreach (var fc in forecasts)
111     {
112         var existing = await _db.DemandForecast
113             .Where(x =>
114                 x.ForecastDate == fc.ForecastDate &&
115                 x.VendorId == fc.VendorId &&
116                 x.ProductId == fc.ProductId &&
117                 x.ModelName == fc.ModelName)
118             .ToListAsync();
119
120         if (existing.Count > 0)
121             _db.DemandForecast.RemoveRange(existing);
122     }
123
124     await _db.DemandForecast.AddRangeAsync(forecasts);
125     return await _db.SaveChangesAsync();
126 }
127
128 // =====
129 // 4) FORECAST ROWS FOR UI
130 // =====
131 public async Task<List<DemandForecastRowDto>> GetForecastRowsAsync(DateTime forecastDate, string modelName)
132 {
133     var rows = await _db.DemandForecast
134         .AsNoTracking()
135         .Include(f => f.Product)
136         .Include(f => f.Vendor)
137         .Where(f => f.ForecastDate == forecastDate.Date && f.ModelName == modelName)
138         .OrderByDescending(f => f.CreatedAt)

```

```

env DemandForecastService.cs x MSSQL: Welcome & What's New
26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > DemandForecastService
30 {
31 {
32     var rows = await _db.DemandForecast
33     {
34         DemandForecastId = f.DemandForecastId,
35         VendorId = f.VendorId,
36         VendorName = f.Vendor.DisplayName,
37         ProductId = f.ProductId,
38         ProductName = f.Product.Name,
39         ForecastDate = f.ForecastDate,
40         ForecastQty = f.ForecastQty,
41         ModelName = f.ModelName,
42         LookbackPeriods = f.LookbackPeriods,
43         CreatedAt = f.CreatedAt
44     })
45     .ToListAsync();
46
47     return rows;
48 }
49
50 // =====
51 // 5) CHART DATA
52 // =====
53 public async Task<List<ForecastChartPointDto>> GetForecastChartDataAsync(
54     string vendorId,
55     int productId,
56     DateTime forecastDate,
57     string modelName = "MLNET_SSA")
58 {
59     var history = await _db.DemandRequest
60     .AsNoTracking()
61     .Where(r => r.VendorId == vendorId &&
62         r.ProductId == productId &&
63         r.CreatedAt < forecastDate)
64     .OrderByDescending(r => r.CreatedAt)
65     .Take(10)
66     .Select(r => new ForecastChartPointDto
67     {
68         Date = r.CreatedAt.ToString("yyyy-MM-dd"),
69         Quantity = r.QuantityRequested,
70         Series = "Historical"
71     })
72     .ToListAsync();
73
74     history = history.OrderBy(x => x.Date).ToList();
75
76     var forecasts = await _db.DemandForecast
77     .AsNoTracking()
78     .Where(f => f.VendorId == vendorId &&
79         f.ProductId == productId &&
80         f.ModelName == modelName &&
81         f.ForecastDate >= forecastDate.Date)
82     .OrderBy(f => f.ForecastDate)
83     .Take(10)
84     .Select(f => new ForecastChartPointDto
85     {
86         Date = f.ForecastDate.ToString("yyyy-MM-dd"),
87         Quantity = f.ForecastQty,
88         Series = "Forecast"
89     })
90     .ToListAsync();
91
92     return history.Concat(forecasts).ToList();
93 }
94
95 // =====
96 // 6) FILTERED VENDOR OPTIONS FOR FARMER

```

```

10  {
101
302  // =====
303  // 6) FILTERED VENDOR OPTIONS FOR FARMER
304  // =====
305  public async Task<List<ForecastVendorOptionDto>> GetForecastVendorsForFarmerAsync(string farmerId, string? search = null)
306  {
307      var availableProductIds = await _db.InventoryLot
308          .AsNoTracking()
309          .Where(i =>
310              i.FarmerId == farmerId &&
311              i.QuantityAvailable > 0 &&
312              i.Product.IsActive)
313          .Select(i => i.ProductId)
314          .Distinct()
315          .ToListAsync();
316
317      if (availableProductIds.Count == 0)
318          return new List<ForecastVendorOptionDto>();
319
320      var query = _db.DemandRequest
321          .AsNoTracking()
322          .Where(r => availableProductIds.Contains(r.ProductId))
323          .Select(r => r.Vendor)
324          .Distinct();
325
326      if (!string.IsNullOrEmpty(search))
327      {
328          var term = search.Trim().ToLower();
329          query = query.Where(v =>
330              (v.DisplayName != null && v.DisplayName.ToLower().Contains(term)) ||
331              (v.Email != null && v.Email.ToLower().Contains(term)));
332      }
333
334      return await query
335          .OrderBy(v => v.DisplayName)
336          .Select(v => new ForecastVendorOptionDto
337          {
338              Id = v.Id,
339              DisplayName = v.DisplayName ?? "",
340              Email = v.Email ?? ""
341          })
342          .ToListAsync();
343  }
344
345  // =====
346  // 7) FILTERED PRODUCT OPTIONS FOR FARMER
347  // =====
348  public async Task<List<ForecastProductOptionDto>> GetForecastProductsForFarmerAsync(string farmerId, string? search = null)
349  {
350      var query = _db.InventoryLot
351          .AsNoTracking()
352          .Where(i =>
353              i.FarmerId == farmerId &&
354              i.QuantityAvailable > 0 &&
355              i.Product.IsActive)
356          .Select(i => i.Product)
357          .Distinct();
358
359      if (!string.IsNullOrEmpty(search))
360      {
361          var term = search.Trim().ToLower();
362          query = query.Where(p => p.Name.ToLower().Contains(term));
363      }
364
365      return await query
366          .OrderBy(p => p.Name)
367          .Select(p => new ForecastProductOptionDto

```

```

env DemandForecastService.cs X MSSQL: Welcome & What's New
5_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > DemandForecastService.cs
{
    {
        return await query
    })
    .ToListAsync();
    }

    // =====
    // 8) PRIVATE HELPER FOR DAILY / WEEKLY SERIES
    // =====
    private async Task<List<AggregatedDemandPoint>> LoadAggregatedDemandSeriesAsync(
        int productId,
        string vendorId,
        DateTime beforeDate,
        string granularity)
    {
        var raw = await _db.DemandRequest
            .AsNoTracking()
            .Where(r => r.ProductId == productId &&
                r.VendorId == vendorId &&
                r.CreatedAt < beforeDate)
            .OrderBy(r => r.CreatedAt)
            .Select(r => new
            {
                r.CreatedAt,
                r.QuantityRequested
            })
            .ToListAsync();

        if (granularity.Equals("Weekly", StringComparison.OrdinalIgnoreCase))
        {
            return raw
                .GroupBy(x => StartOfWeek(x.CreatedAt.Date))
                .Select(g => new AggregatedDemandPoint
                {
                    PeriodDate = g.Key,
                    TotalQuantity = g.Sum(x => x.QuantityRequested)
                })
                .OrderBy(x => x.PeriodDate)
                .ToList();
        }

        return raw
            .GroupBy(x => x.CreatedAt.Date)
            .Select(g => new AggregatedDemandPoint
            {
                PeriodDate = g.Key,
                TotalQuantity = g.Sum(x => x.QuantityRequested)
            })
            .OrderBy(x => x.PeriodDate)
            .ToList();
    }

    private static DateTime StartOfWeek(DateTime date)
    {
        {
            int diff = (7 + (date.DayOfWeek - DayOfWeek.Monday)) % 7;
            return date.AddDays(-diff).Date;
        }
    }

    private class AggregatedDemandPoint
    {
        {
            public DateTime PeriodDate { get; set; }
            public decimal TotalQuantity { get; set; }
        }
    }
}

```

- *MLDemandForecastingEngine.cs*

```

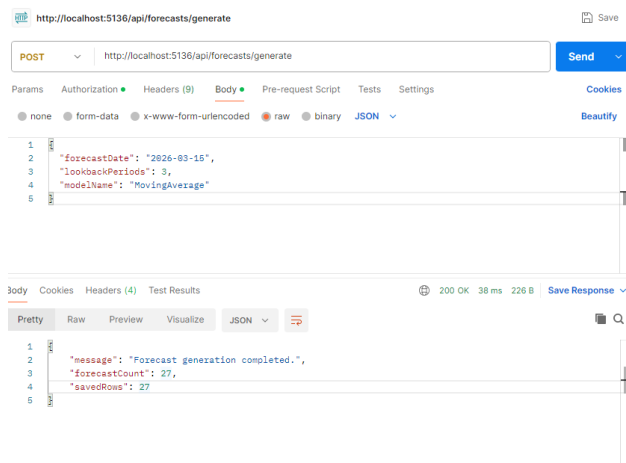
1  using Microsoft.ML;
2  using Microsoft.ML.Transforms.TimeSeries;
3
4  namespace FarmVendor.Api.Services.Forecasting;
5
6  public class MLDemandForecastingEngine
7  {
8      private readonly MLContext _mlContext;
9
10     public MLDemandForecastingEngine()
11     {
12         _mlContext = new MLContext(seed: 1);
13     }
14
15     public ForecastTrainingResult TrainAndForecast(
16         List<float> orderedSeries,
17         int horizon)
18     {
19         if (orderedSeries == null || orderedSeries.Count < 5)
20             throw new InvalidOperationException("Not enough historical data to train forecasting model.");
21
22         var data = orderedSeries
23             .Select(x => new DemandTimeSeriesPoint { Quantity = x })
24             .ToList();
25
26         IDataView dataView = _mlContext.Data.LoadFromEnumerable(data);
27
28         int trainSize = orderedSeries.Count;
29         int seriesLength = orderedSeries.Count;
30
31         // safer settings for small datasets
32         int windowSize = Math.Min(3, Math.Max(2, orderedSeries.Count / 3));
33
34         var pipeline = _mlContext.Forecasting.ForecastBySsa(
35             outputColumnName: nameof(DemandForecastPrediction.ForecastedQuantity),
36             inputColumnName: nameof(DemandTimeSeriesPoint.Quantity),
37             windowSize: windowSize,
38             seriesLength: seriesLength,
39             trainSize: trainSize,
40             horizon: horizon,
41             confidenceLevel: 0.95f,
42             confidenceLowerBoundColumn: nameof(DemandForecastPrediction.LowerBoundQuantity),
43             confidenceUpperBoundColumn: nameof(DemandForecastPrediction.UpperBoundQuantity));
44
45         SsaForecastingTransformer model = pipeline.Fit(dataView);
46
47         TimeSeriesPredictionEngine<DemandTimeSeriesPoint, DemandForecastPrediction> engine =
48             model.CreateTimeSeriesEngine<DemandTimeSeriesPoint, DemandForecastPrediction>(_mlContext);
49
50         DemandForecastPrediction prediction = engine.Predict();
51
52         return new ForecastTrainingResult
53         {
54             Forecast = prediction.ForecastedQuantity?.ToList() ?? new List<float>(),
55             LowerBounds = prediction.LowerBoundQuantity?.ToList() ?? new List<float>(),
56             UpperBounds = prediction.UpperBoundQuantity?.ToList() ?? new List<float>()
57         };
58     }
59 }

```

- *Postman Testing*

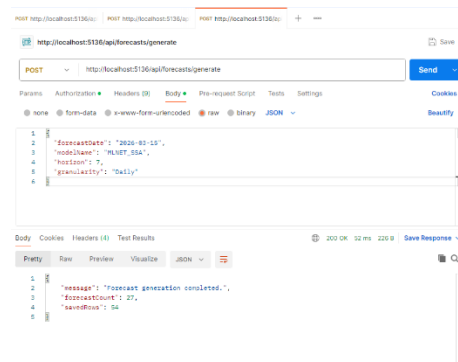
1. Moving Average request : POST : <http://localhost:5136/api/forecasts/generate>

```
{  
  "forecastDate": "2026-03-15",  
  "lookbackPeriods": 3,  
  "modelName": "MovingAverage"  
}
```



2. ML.NET request : POST : <http://localhost:5136/api/forecasts/generate>

```
{  
  "forecastDate": "2026-03-15",  
  "modelName": "MLNET_SSA",  
  "horizon": 7,  
  "granularity": "Daily"  
}
```



3. GET /api/forecasts?modelName=MovingAverage :

<http://localhost:5136/api/forecasts?modelName=MovingAverage>

4. GET /api/forecasts?modelName=MLNET_SSA : http://localhost:5136/api/forecasts?modelName=MLNET_SSA

REST client interface showing a GET request to `http://localhost:5136/api/forecasts?modelName=MovingAverage`. The response is a JSON array of forecast objects.

```
1 {
2   "demandForecastId": 100,
3   "vendorId": "ec324674-24e4-4fdc-948d-f29d9c8a2ccd",
4   "productId": 5,
5   "productName": "Eggs",
6   "forecastDate": "2026-03-15T00:00:00",
7   "forecastQty": 9.00,
8   "modelName": "MovingAverage",
9   "lookbackPeriods": 3,
10  "createdAt": "2026-03-11T01:13:36.4811718"
11 },
12 {
13   "demandForecastId": 107,
14   "vendorId": "ec324674-24e4-4fdc-948d-f29d9c8a2ccd",
15   "productId": 2,
16   "productName": "Onions",
17   "forecastDate": "2026-03-15T00:00:00",
18   "forecastQty": 27.00,
19   "modelName": "MovingAverage",
20   "lookbackPeriods": 3,
21   "createdAt": "2026-03-11T01:13:36.4811718"
22 },
23 ]
```

REST client interface showing a GET request to `http://localhost:5136/api/forecasts?modelName=MLNET_SSA`. The response is a JSON array of forecast objects.

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...

```
1 {
2   "demandForecastId": 100,
3   "vendorId": "ec324674-24e4-4fdc-948d-f29d9c8a2ccd",
4   "productId": 5,
5   "productName": "Eggs",
6   "forecastDate": "2026-03-15T00:00:00",
7   "forecastQty": 9.00,
8   "modelName": "MovingAverage",
9   "lookbackPeriods": 3,
10  "createdAt": "2026-03-11T01:13:36.4811718"
11 },
12 {
13   "demandForecastId": 107,
14   "vendorId": "ec324674-24e4-4fdc-948d-f29d9c8a2ccd",
15   "productId": 2,
16   "productName": "Onions",
17   "forecastDate": "2026-03-15T00:00:00",
18   "forecastQty": 27.00,
19   "modelName": "MovingAverage",
20   "lookbackPeriods": 3,
21   "createdAt": "2026-03-11T01:13:36.4811718"
22 },
23 ],
24 {
25   "demandForecastId": 106,
26   "vendorId": "e52185b1-fdc4-48d6-b0f9-31327f523a22",
27   "productId": 3,
28   "productName": "Potatoes",
29   "forecastDate": "2026-03-15T00:00:00",
30   "forecastQty": 21.00,
31   "modelName": "MovingAverage",
32   "lookbackPeriods": 3,
33   "createdAt": "2026-03-11T01:13:36.4811713"
34 },
35 {
36   "demandForecastId": 105,
37   "vendorId": "c68d44bf-b03f-4ce1-9975-4ec62237fc08",
38   "productId": 4,
39   "productName": "Milk",
40   "forecastDate": "2026-03-15T00:00:00",
41   "forecastQty": 53.00,
42 }
```

- *Database queries*

- Forecast dates exists

The screenshot shows a SQL Server Enterprise Manager interface. The top pane displays a query in the 'Query Editor' window, titled 'MLNET_SSA.sql'. The query is as follows:

```

1
2 --MLNET_SSA dates actually exist
3 USE FarmVendorDb;
4 GO
5
6 SELECT ForecastDate, ModelName, COUNT(*) AS ForecastCount
7 FROM DemandForecast
8 WHERE ModelName = 'MLNET_SSA'
9 GROUP BY ForecastDate, ModelName
10 ORDER BY ForecastDate;
11
12 SELECT ForecastDate, ModelName, COUNT(*)
13 FROM DemandForecast
14 WHERE ForecastDate = '2026-04-07'
15 GROUP BY ForecastDate, ModelName;

```

The bottom pane shows the 'QUERY RESULTS' tab. It displays two result sets. The first result set, titled 'Results (2)', contains 7 rows of data. The second result set, titled 'Messages', contains 2 rows of data.

	ForecastDate	ModelName	ForecastCount
1	2026-04-07 00:00:00.0000000	MLNET_SSA	14
2	2026-04-08 00:00:00.0000000	MLNET_SSA	14
3	2026-04-09 00:00:00.0000000	MLNET_SSA	14
4	2026-04-10 00:00:00.0000000	MLNET_SSA	14
5	2026-04-11 00:00:00.0000000	MLNET_SSA	14
6	2026-04-12 00:00:00.0000000	MLNET_SSA	14
7	2026-04-13 00:00:00.0000000	MLNET_SSA	14

	ForecastDate	ModelName	(No column name)
1	2026-04-07 00:00:00.0000000	MLNET_SSA	14
2	2026-04-07 00:00:00.0000000	MovingAverage	14

- MLNET_SSA forecast data existence check

env MLNET_SSA.sql M productsforforecast.sql MSSQL: Welcome & What's New

026_4495_S3_ManeeshaE > ReportsAndDocuments > sql_queries > productsforforecast.sql

```

3
4 SELECT
5     i.FarmerId,
6     p.ProductId,
7     p.Name AS ProductName,
8     i.QuantityAvailable,
9     i.ExpiryDate
10  FROM InventoryLot i
11  JOIN Product p ON i.ProductId = p.ProductId
12  WHERE i.FarmerId = (
13      SELECT TOP 1 Id
14      FROM AspNetUsers
15      WHERE DisplayName = 'Farmer One'
16  )
17  AND i.QuantityAvailable > 0
18  ORDER BY p.Name;
19
20 USE FarmVendorDb;
21 GO
22 --Check whether ML forecasts exist for 2026-04-01
23 SELECT
24     ForecastDate,
25     ModelName,
26     COUNT(*) AS ForecastCount
27  FROM DemandForecast
28  WHERE ForecastDate = '2026-04-07'
29  GROUP BY ForecastDate, ModelName
30  ORDER BY ModelName;
31 -- Check whether 2026-04-01 ML forecasts overlap with Farmer One's products
32 USE FarmVendorDb;
33 GO
34
35 SELECT
36     p.Name AS ProductName,
37     f.VendorId,
38     f.ForecastQty,
39     f.ForecastDate,
40     f.ModelName
41  FROM DemandForecast f
42  JOIN Product p ON f.ProductId = p.ProductId

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

Results (3) Messages

ForecastDate	ModelName	ForecastCount
2026-04-07 00:00:00.0000000	MLNET_SSA	14
2026-04-07 00:00:00.0000000	MovingAverage	14

ProductName	VendorId	ForecastQty	ForecastDate	ModelName
Milk	c7467bc1-05b3-4a5d-aa7d-24516b1442ae	56.70	2026-04-07 00:00:00.0000000	MLNET_SSA
Milk	2b0d536e-b91e-436a-bfa6-9fcc19946f39	37.81	2026-04-07 00:00:00.0000000	MLNET_SSA
Milk	6092c5ac-4914-4714-9391-b540854ca3d0	12.63	2026-04-07 00:00:00.0000000	MLNET_SSA
Milk	d0dcc510-cf8-4d86-9e28-2612deb9c007	11.30	2026-04-07 00:00:00.0000000	MLNET_SSA
Onions	f13bc95a-2cc4-43c6-b859-d74eacc84a22	59.70	2026-04-07 00:00:00.0000000	MLNET_SSA
Onions	0913075e-d644-465f-b4db-73992cbcd72	58.70	2026-04-07 00:00:00.0000000	MLNET_SSA
Onions	6092c5ac-4914-4714-9391-b540854ca3d0	50.70	2026-04-07 00:00:00.0000000	MLNET_SSA
Onions	4ee2bdbc-7529-4a7c-89e8-6e604a098aae	38.69	2026-04-07 00:00:00.0000000	MLNET_SSA
Onions	7eca8195-448a-4a5a-a064-b289f4335c56	28.79	2026-04-07 00:00:00.0000000	MLNET_SSA

- Forecast data check

W26.4495_S3_Maneeshah > ReportsAndDocuments > sql_queries > 2forecast_data_check.sql

```

1  (localdb)\MSSQLLocalDB | Dmaster
2  USE FarmVendorDb;
3  SELECT VendorId, ProductId, COUNT(*) AS RequestCount
4  FROM DemandRequest
5  GROUP BY VendorId, ProductId
6  ORDER BY RequestCount DESC
7
8  SELECT COUNT(*) FROM DemandForecast;
9
10 --data check for today
11 USE FarmVendorDb;
12 GO
13
14 SELECT ForecastDate, COUNT(*) AS ForecastCount
15 FROM DemandForecast
16 GROUP BY ForecastDate
17 ORDER BY ForecastDate DESC;
18
19
20 --Confirm whether forecast rows exist
21 USE FarmVendorDb;
22
23 SELECT TOP 50

```

Results (4) Messages

VendorId	ProductId	RequestCount
7eca8195-448a-4a5a-a064-b289f4335c56	2	26
84c6e2c2-3e74-4fc0-9afc-9fde91c81fe	3	26
c21f6e2-8142-4368-9662-cb86d9596c2d	3	26
d72c71c5-2e8d-4e24-bcb9-49b952f23a...	3	26
2b0d536e-b91e-436a-bfa6-9fcc19946f99	4	26
d0dc510-ctc8-4d86-9e28-2612deb9c007	4	26
d0094948-5524-471c-86a5-2c28f1c9ec5e	5	26
0913075e-d644-465f-b4db-73992cbdb...	5	25
09dd3dc1-b786-4258-b52f-03afa6e0f6c0	5	25

(No column name)

ForecastDate	ForecastCount
2026-04-13 00:00:00.0000000	14
2026-04-12 00:00:00.0000000	14
2026-04-11 00:00:00.0000000	14
2026-04-10 00:00:00.0000000	14
2026-04-09 00:00:00.0000000	14
2026-04-08 00:00:00.0000000	14
2026-04-07 00:00:00.0000000	28

DemandForecastId	VendorId	ProductId	ForecastDate	ForecastQty	ModelName	LookbackPeriods	CreatedAt
112	f13bc95a-2cc4-43c6-b859-d74eacc84a22	2	2026-04-07 00:00:00.0000000	60.00	MovingAverage	3	2026-04-07 22:37:15.7763969
111	d0dc510-ctc8-4d86-9e28-2612deb9c007	4	2026-04-07 00:00:00.0000000	11.67	MovingAverage	3	2026-04-07 22:37:15.7763949
110	d0094948-5524-471c-86a5-2c28f1c9ec5e	5	2026-04-07 00:00:00.0000000	13.00	MovingAverage	3	2026-04-07 22:37:15.7763936
109	c7467bc1-05b3-4a5d-aa7d-24516b1442ae	4	2026-04-07 00:00:00.0000000	57.00	MovingAverage	3	2026-04-07 22:37:15.7763923
108	7eca8195-448a-4a5a-a064-b289f4335c56	2	2026-04-07 00:00:00.0000000	19.67	MovingAverage	3	2026-04-07 22:37:15.7763897

Feature 2: Vendor Recommendation, Relationship Score, and Distance-Based Matching

This feature supports dispatch decision-making by identifying suitable vendors based on demand relevance, past interactions, and physical distance.

This feature includes:

- Recommended vendors section
- Distance calculation between farmer and vendor
- Relationship score logic
- Hover popup for vendor details
- Request-specific vendor information

The design consists of:

- Score-based ranking
- Location-aware prioritization
- Historical relationship analysis

Implementation

Frontend

- FarmerDashboard.jsx
- vendor hover popup UI
- recommendation table rendering

Backend

- FarmerRecommendationService.cs
- FarmerController.cs
- RelationshipScoreService.cs
- Haversine distance calculation logic

Database

RelationshipStat table storing:

- FarmerId
- VendorId
- TotalRequestedQty

- TotalDeliveredQty
- RelationshipScore

Technical Methods

- weighted score formula using demand, distance, and relationship
- geospatial calculation using latitude/longitude
- ranking through EF Core + LINQ queries

App

FarmerDashboard

FarmerVendor

Dashboard

Products

Requests

Dispatch

Forecast

Chat

Profile

Logout

Welcome, Farmer 1

Overview of your stock, requests, and vendor recommendations

Available Products3

Upcoming Stock1

Upcoming Requests30

Current Stock

Product	Qty	Expiry	Status
Onions	80 kg	2026-04-11	Fresh
Milk	90 L	2026-04-16	Fresh
Eggs	70 dozen	2026-04-21	Fresh

Upcoming Vendor Requests

Product	Qty	Needed By
Tomatoes	40 kg	2026-04-10
Potatoes	22 kg	2026-04-10
Milk	10 L	2026-04-10
Onions	10 kg	2026-04-10
Potatoes	10 kg	2026-04-11
Tomatoes	24 kg	2026-04-12
Milk	50 L	2026-04-12
Onions	20 kg	2026-04-12
Onions	75 kg	2026-04-13
Onions	10 kg	2026-04-13

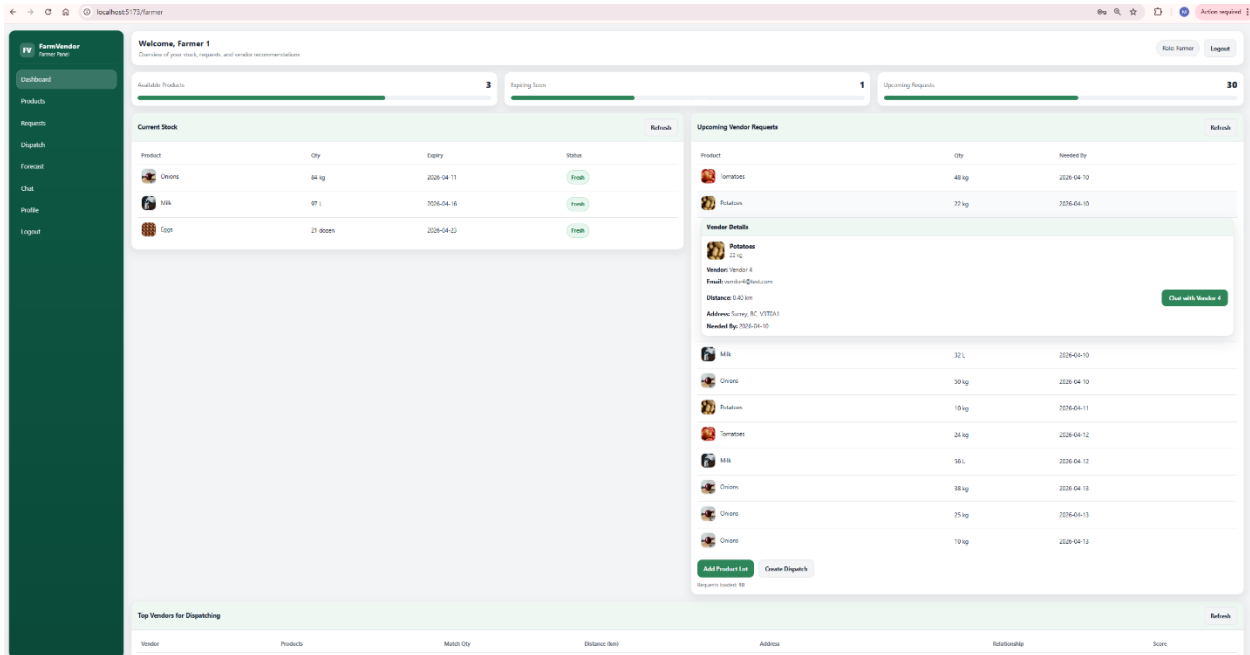
Top Vendors for Dispatching

Vendor	Products	Match Qty	Distance (km)	Address	Relationship	Score
Vendor 15	Onions, Milk	63.33	1.86	Surmy, BC V3T6A1	50	76.13
Vendor 1	Milk	11.3	0	Surmy, BC V3T6A1	50	76.5
Vendor 2	Onions	38.69	0.13	Surmy, BC V3T6A1	50	76.47
Vendor 3	Onions	59.7	0.27	Surmy, BC V3T6A1	50	76.45
Vendor 4	Onions	25.68	0.4	Surmy, BC V3T6A1	50	76.42

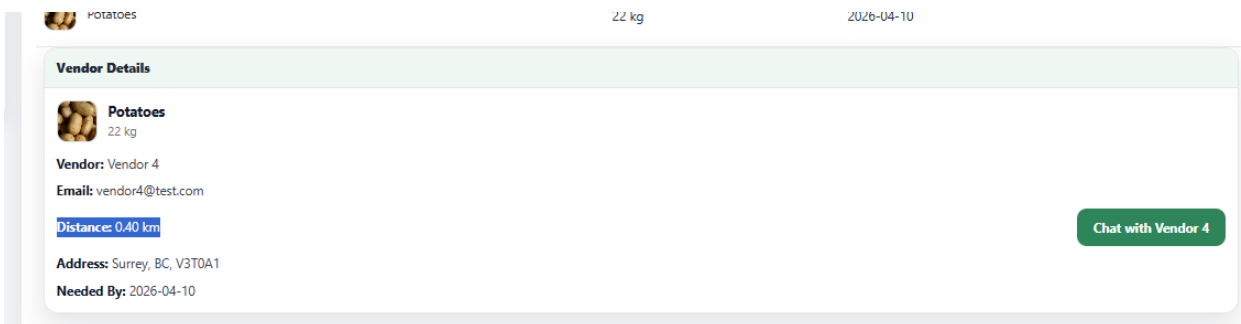
- Top Vendors for Dispatching section

Top Vendors for Dispatching							Refresh
Vendor	Products	Match Qty	Distance (km)	Address	Relationship	Score	
Vendor 15	Onions, Milk	63.33	1.86	Surmy, BC V3T6A1	50	76.13	
Vendor 1	Milk	11.3	0	Surmy, BC V3T6A1	50	76.5	
Vendor 2	Onions	38.69	0.13	Surmy, BC V3T6A1	50	76.47	
Vendor 3	Onions	59.7	0.27	Surmy, BC V3T6A1	50	76.45	
Vendor 4	Onions	25.68	0.4	Surmy, BC V3T6A1	50	76.42	

- Vendor specific hover popup

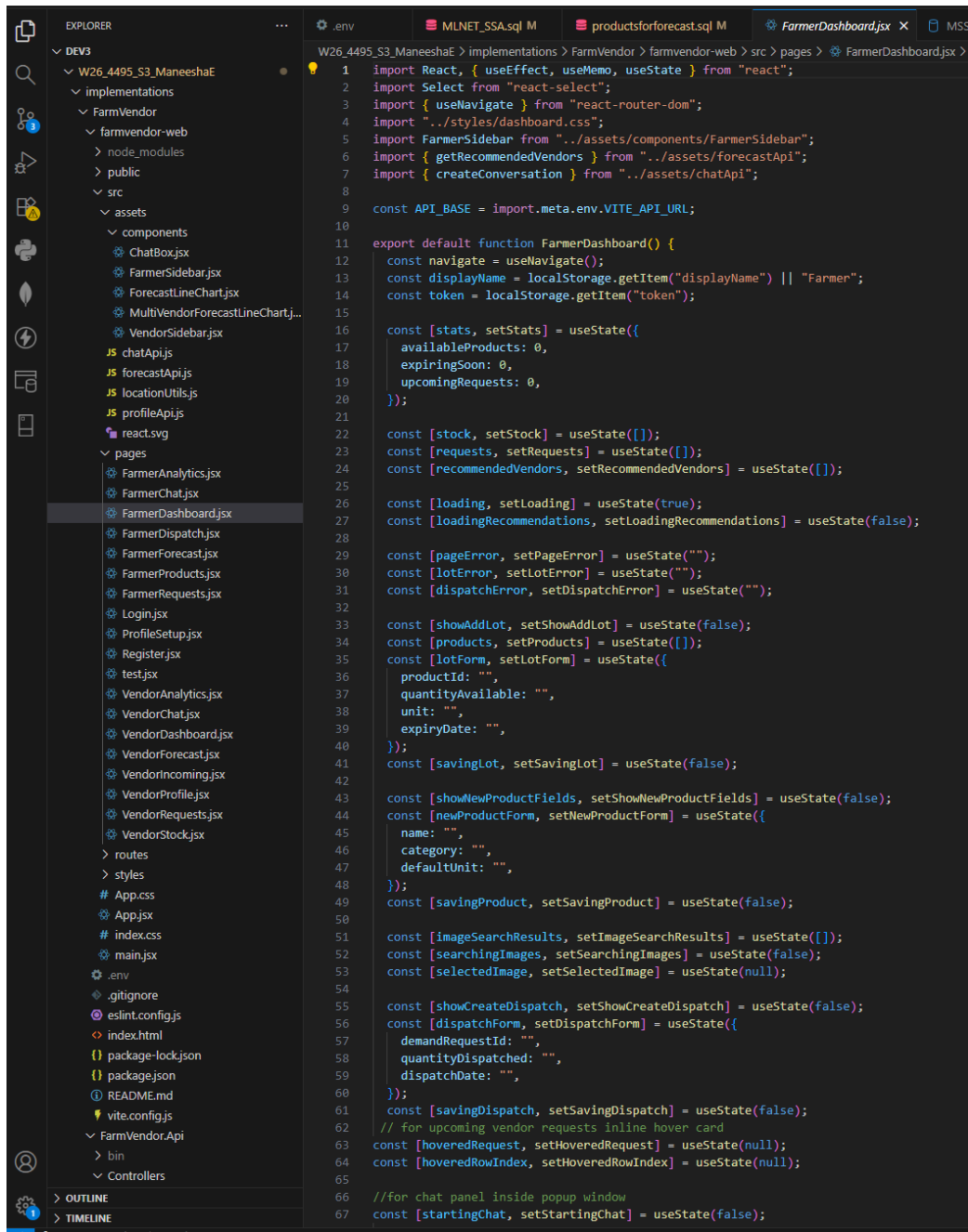


- Distance shown in request details



Frontend

- *FarmerDashboard.jsx*



```
1 import React, { useEffect, useMemo, useState } from "react";
2 import Select from "react-select";
3 import { useNavigate } from "react-router-dom";
4 import "../styles/dashboard.css";
5 import FarmerSidebar from "../assets/components/FarmerSidebar";
6 import { getRecommendedVendors } from "../assets/forecastApi";
7 import { createConversation } from "../assets/chatApi";
8
9 const API_BASE = import.meta.env.VITE_API_URL;
10
11 export default function FarmerDashboard() {
12   const navigate = useNavigate();
13   const displayName = localStorage.getItem("displayName") || "Farmer";
14   const token = localStorage.getItem("token");
15
16   const [stats, setStats] = useState({
17     availableProducts: 0,
18     expiringSoon: 0,
19     upcomingRequests: 0,
20   });
21
22   const [stock, setStock] = useState([]);
23   const [requests, setRequests] = useState([]);
24   const [recommendedVendors, setRecommendedVendors] = useState([]);
25
26   const [loading, setLoading] = useState(true);
27   const [loadingRecommendations, setLoadingRecommendations] = useState(false);
28
29   const [pageError, setPageError] = useState("");
30   const [lotError, setLotError] = useState("");
31   const [dispatchError, setDispatchError] = useState("");
32
33   const [showAddLot, setShowAddLot] = useState(false);
34   const [products, setProducts] = useState([]);
35   const [lotForm, setLotForm] = useState({
36     productId: "",
37     quantityAvailable: "",
38     unit: "",
39     expiryDate: "",
40   });
41   const [savingLot, setSavingLot] = useState(false);
42
43   const [showNewProductFields, setShowNewProductFields] = useState(false);
44   const [newProductForm, setNewProductForm] = useState({
45     name: "",
46     category: "",
47     defaultUnit: "",
48   });
49   const [savingProduct, setSavingProduct] = useState(false);
50
51   const [imageSearchResults, setImageSearchResults] = useState([]);
52   const [searchingImages, setSearchingImages] = useState(false);
53   const [selectedImage, setSelectedImage] = useState(null);
54
55   const [showCreateDispatch, setShowCreateDispatch] = useState(false);
56   const [dispatchForm, setDispatchForm] = useState({
57     demandRequestId: "",
58     quantityDispatched: "",
59     dispatchDate: "",
60   });
61   const [savingDispatch, setSavingDispatch] = useState(false);
62   // for upcoming vendor requests inline hover card
63   const [hoveredRequest, setHoveredRequest] = useState(null);
64   const [hoveredRowIndex, setHoveredRowIndex] = useState(null);
65
66   //for chat panel inside popup window
67   const [startingChat, setStartingChat] = useState(false);
```



```

v MLNET_SSA.sql M productsforforecast.sql M FarmerDashboard.jsx MSSQL: Welc
4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > Farm
export default function FarmerDashboard() {
  const loadDashboard = async () => {
    setStats(JSON.parse(statsText));
    setStock(JSON.parse(stockText));
    setRequests(JSON.parse(reqText));
  };

  const loadRecommendedVendors = async () => {
    setLoadingRecommendations(true);
    try {
      const data = await getRecommendedVendors(
        new Date().toISOString().slice(0, 10),
        token
      );
      setRecommendedVendors(data);
    } catch (e) {
      setPageError(e?.message || "Failed to load recommended vendors");
    } finally {
      setLoadingRecommendations(false);
    }
  };

  const searchProductImages = async () => {
    setLotError("");

    if (!newProductForm.name.trim()) {
      return setLotError("Enter product name before searching images.");
    }

    setSearchingImages(true);
    try {
      const res = await fetch(
        `${API_BASE}/api/products/search-images?query=${encodeURIComponent(
          newProductForm.name.trim()
        )}`,
        { headers: authHeaders }
      );

      const text = await res.text();
      if (!res.ok) throw new Error(text || "Failed to search images.");

      const data = JSON.parse(text);
      setImageSearchResults(Array.isArray(data) ? data : []);
    } catch (err) {
      setLotError(err?.message || "Failed to search images.");
    } finally {
      setSearchingImages(false);
    }
  };
};

```

```

6_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > FarmerDashboard.jsx
11 export default function FarmerDashboard() {
62
63     <div style={{ marginTop: 8, fontSize: 12, color: "#6b7280" }}>
64         Requests loaded: <b>{requests.length}</b>
65     </div>
66 </div>
67 </div>
68 </section>
69
70 <section style={{ marginTop: 14 }}>
71     <div className="card">
72         <div className="card-header">
73             <h2>Top Vendors for Dispatching</h2>
74             <button
75                 className="btn btn-secondary"
76                 onClick={loadRecommendedVendors}
77                 disabled={loadingRecommendations}
78             >
79                 {loadingRecommendations ? "Loading..." : "Refresh"}
80             </button>
81         </div>
82
83         <div className="card-body">
84             <table className="table">
85                 <thead>
86                     <tr>
87                         <th>Vendor</th>
88                         <th>Products</th>
89                         <th>Match Qty</th>
90                         <th>Distance (km)</th>
91                         <th>Address</th>
92                         <th>Relationship</th>
93                         <th>Score</th>
94                     </tr>
95                 </thead>
96                 <tbody>
97                     {recommendedVendors.length === 0 ? (
98                         <tr>
99                             <td colspan="6">No recommended vendors found.</td>
100                         </tr>
101                     ) : (
102                         recommendedVendors.map((v) => (
103                             <tr key={v.vendorId}>
104                                 <td>{v.vendorName}</td>
105                                 <td>{v.matchedProducts?.join(", ")}</td>
106                                 <td>{v.matchableQuantity}</td>
107                                 <td>{v.distanceKm}</td>
108
109                                 <td>
110                                     {[v.vendorCity, v.vendorProvince, v.vendorPostalCode]
111                                         .filter(Boolean)
112                                         .join(", ") || "N/A"}
113                                 </td>
114
115                                 <td>{v.relationshipScore}</td>
116                                 <td>{v.finalScore}</td>
117                             </tr>
118                         ))
119                     )}
120                 </tbody>
121             </table>
122         </div>
123     </div>
124 </section>
125 </div>
126 </main>
127 </div>

```

- upcoming vendor requests inline hover card

```

env MLNET_SSA.sql M productsforforecast.sql M FarmerDashboard.jsx MSSQL: Welcome
5_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > FarmerDas
11 export default function FarmerDashboard() {
12
13     const [dispatchForm, setDispatchForm] = useState({
14         demandRequestId: "",
15         quantityDispatched: "",
16         dispatchDate: "",
17     });
18
19     const [savingDispatch, setSavingDispatch] = useState(false);
20     // for upcoming vendor requests inline hover card
21     const [hoveredRequest, setHoveredRequest] = useState(null);
22     const [hoveredRowIndex, setHoveredRowIndex] = useState(null);
23
24     //for chat panel inside popup window
25     const [startingChat, setStartingChat] = useState(false);
26
27     const authHeaders = useMemo(() => {
28         return {
29             "Content-Type": "application/json",
30             Authorization: `Bearer ${token}`,
31         };
32     }, [token]);
33
34     const productOptions = useMemo(() => {
35         return products.map((p) => ({
36             value: p.productId,
37             label: p.name,
38             defaultUnit: p.defaultUnit,
39             imageThumbUrl: p.imageThumbUrl || "",
40             imageUrl: p.imageUrl || "",
41             category: p.category || "",
42         }));
43     }, [products]);
44
45     const selectedProductOption = useMemo(() => {
46         return (
47             productOptions.find((o) => String(o.value) === String(lotForm.productId)) || null
48         );
49     }, [productOptions, lotForm.productId]);
50
51     const dispatchRequestOptions = useMemo(() => {
52         return requests.map((r) => ({
53             value: r.demandRequestId,
54             demandRequestId: r.demandRequestId,
55             product: r.product,
56             qty: r.qty,
57             unit: r.unit,
58             neededBy: r.neededBy,
59             imageThumbUrl: r.imageThumbUrl || "",
60             imageUrl: r.imageUrl || "",
61             vendorId: r.vendorId || "",
62             vendorName: r.vendorName || "Vendor",
63             vendorEmail: r.vendorEmail || "",
64             distanceToVendor: r.distanceToVendor ?? null,
65             label: `#${r.demandRequestId} - ${r.product}`,
66         }));
67     }, [requests]);
68
69     const selectedDispatchRequestOption = useMemo(() => {
70         return (
71             dispatchRequestOptions.find(
72                 (o) => String(o.value) === String(dispatchForm.demandRequestId)
73             ) || null
74         );
75     }, [dispatchRequestOptions, dispatchForm.demandRequestId]);
76
77     const selectStyles = useMemo(
78         () => ({
79             control: (base, state) => ({

```

```

<div className="card">
  <div className="card-header">
    <h2>Upcoming Vendor Requests</h2>
    <button className="btn btn-secondary" onClick={loadDashboard}>
      Refresh
    </button>
  </div>

  <div className="card-body">
    <table className="table">
      <thead>
        <tr>
          <th>Product</th>
          <th>Qty</th>
          <th>Needed By</th>
        </tr>
      </thead>
      <tbody>
        {requests.length === 0 ? (
          <tr>
            <td colspan="3">No open demand requests found.</td>
          </tr>
        ) : (
          requests.map((row, idx) => (
            <React.Fragment key={row.demandRequestId ?? idx}>
              <tr>
                className="request-hover-row"
                onMouseEnter={() => handleRequestMouseEnter(row, idx)}
                onMouseLeave={handleRequestMouseLeave}
              >
                <td>{renderProductCell(row)}</td>
                <td>
                  {row.qty} {row.unit}
                </td>
                <td>{new Date(row.neededBy).toISOString().slice(0, 10)}</td>
              </tr>

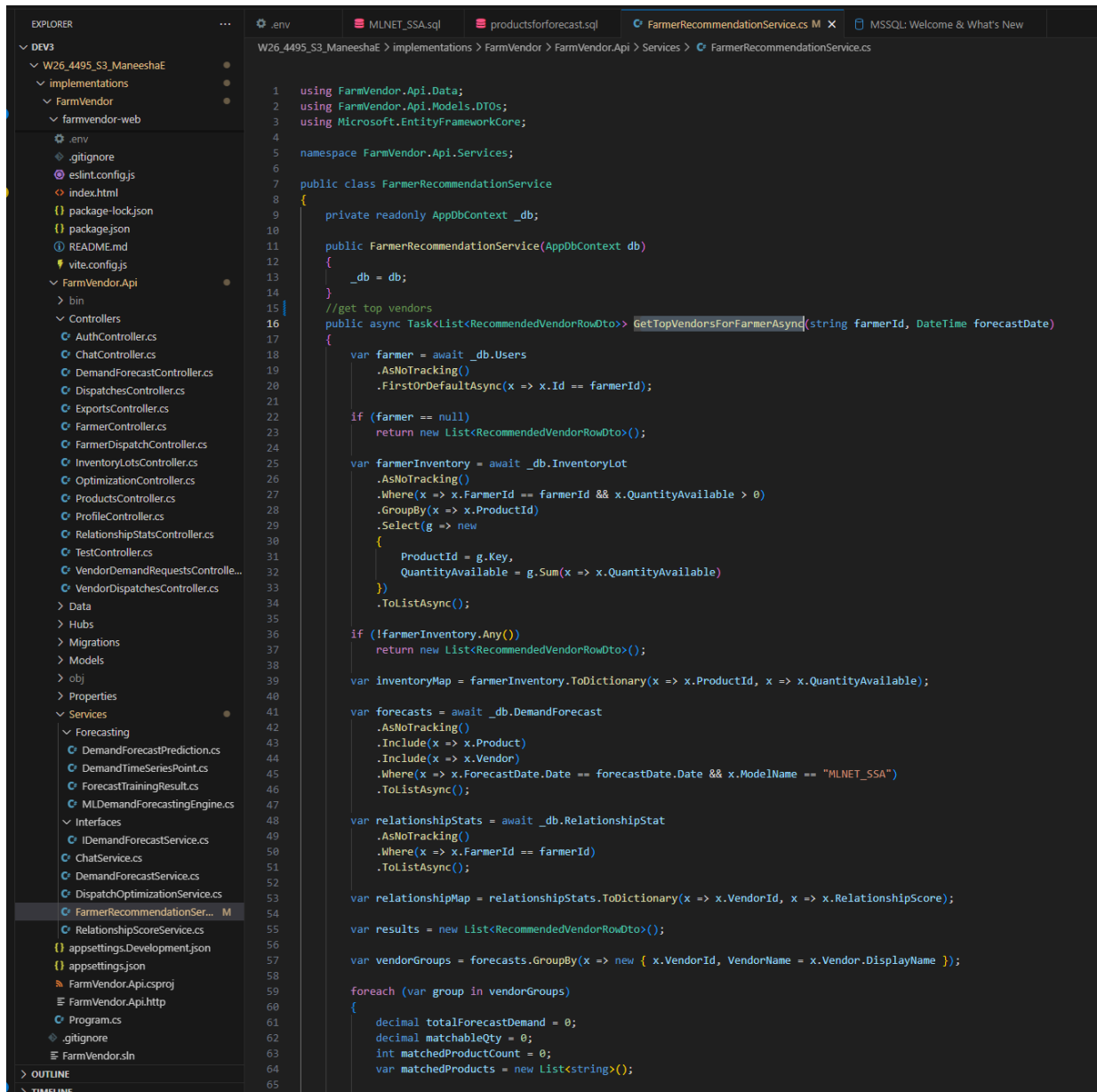
              {hoveredRowIndex === idx && hoveredRequest && (
                <tr>
                  className="request-hover-inline-row"
                  onMouseEnter={() => handleRequestMouseEnter(row, idx)}
                  onMouseLeave={handleRequestMouseLeave}
                >
                  <td colspan="3">
                    <div className="request-hover-inline-card">
                      <div className="request-hover-card-header">
                        <div className="request-hover-card-title">Vendor Details</div>
                      </div>

                      <div className="request-hover-card-body">
                        <div className="request-hover-product">
                          {hoveredRequest.imageThumbUrl || hoveredRequest.imageUrl ? (
                            <img
                              src={hoveredRequest.imageThumbUrl || hoveredRequest.imageUrl}
                              alt={hoveredRequest.product}
                              className="request-hover-thumb"
                            />
                          ) : (
                            <div className="request-hover-placeholder">
                              {hoveredRequest.product?.charAt(0)?.toUpperCase() || "P"}
                            </div>
                          )}
                        <div>
                          <div className="request-hover-product-name">

```

Backend

- *FarmerRecommendationService.cs*



```
1 using FarmVendor.Api.Data;
2 using FarmVendor.Api.Models.DTOS;
3 using Microsoft.EntityFrameworkCore;
4
5 namespace FarmVendor.Api.Services;
6
7 public class FarmerRecommendationService
8 {
9     private readonly AppDbContext _db;
10
11     public FarmerRecommendationService(AppDbContext db)
12     {
13         _db = db;
14     }
15
16     //get top vendors
17     public async Task<List<RecommendedVendorRowDto>> GetTopVendorsForFarmerAsync(string farmerId, DateTime forecastDate)
18     {
19         var farmer = await _db.Users
20             .AsNoTracking()
21             .FirstOrDefaultAsync(x => x.Id == farmerId);
22
23         if (farmer == null)
24             return new List<RecommendedVendorRowDto>();
25
26         var farmerInventory = await _db.InventoryLot
27             .AsNoTracking()
28             .Where(x => x.FarmerId == farmerId && x.QuantityAvailable > 0)
29             .GroupBy(x => x.ProductId)
30             .Select(g => new
31             {
32                 ProductId = g.Key,
33                 QuantityAvailable = g.Sum(x => x.QuantityAvailable)
34             })
35             .ToListAsync();
36
37         if (!farmerInventory.Any())
38             return new List<RecommendedVendorRowDto>();
39
40         var inventoryMap = farmerInventory.ToDictionary(x => x.ProductId, x => x.QuantityAvailable);
41
42         var forecasts = await _db.DemandForecast
43             .AsNoTracking()
44             .Include(x => x.Product)
45             .Include(x => x.Vendor)
46             .Where(x => x.ForecastDate.Date == forecastDate.Date && x.ModelName == "MLNET_SSA")
47             .ToListAsync();
48
49         var relationshipStats = await _db.RelationshipStat
50             .AsNoTracking()
51             .Where(x => x.FarmerId == farmerId)
52             .ToListAsync();
53
54         var relationshipMap = relationshipStats.ToDictionary(x => x.VendorId, x => x.RelationshipScore);
55
56         var results = new List<RecommendedVendorRowDto>();
57
58         var vendorGroups = forecasts.GroupBy(x => new { x.VendorId, VendorName = x.Vendor.DisplayName });
59
60         foreach (var group in vendorGroups)
61         {
62             decimal totalForecastDemand = 0;
63             decimal matchableQty = 0;
64             int matchedProductCount = 0;
65             var matchedProducts = new List<string>();
```

```

.env MLNET_SSA.sql productsforforecast.sql FarmerRecommendationService.cs M x MSSQL: Welcome & What's New
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > FarmerRecommendationService.cs
8 {
17 {
60 {
92 if (vendor != null &&
95 {
96     distanceKm = CalculateDistanceKm(
97         (double)farmer.Latitude.Value,
98         (double)farmer.Longitude.Value,
99         (double)vendor.Latitude.Value,
100         (double)vendor.Longitude.Value);
101     }
102
103     var demandMatchScore = totalForecastDemand == 0 ? 0 : (matchableQty / totalForecastDemand) * 100m;
104     var distanceScore = distanceKm >= 999 ? 0m : (decimal)Math.Max(0, 100 - distanceKm);
105     var productCoverageScore = matchedProductCount * 10m;
106
107     var finalScore =
108         (0.45m * demandMatchScore) +
109         (0.25m * relationshipScore) +
110         (0.20m * distanceScore) +
111         (0.10m * productCoverageScore);
112
113     results.Add(new RecommendedVendorRowDto
114     {
115         VendorId = group.Key.VendorId,
116         VendorName = group.Key.VendorName ?? "Vendor",
117         DistanceKm = Math.Round(distanceKm, 2),
118         RelationshipScore = Math.Round(relationshipScore, 2),
119         TotalForecastDemand = Math.Round(totalForecastDemand, 2),
120         MatchableQuantity = Math.Round(matchableQty, 2),
121         MatchedProductCount = matchedProductCount,
122         FinalScore = Math.Round(finalScore, 2),
123
124         VendorCity = vendor?.City,
125         VendorProvince = vendor?.Province,
126         VendorPostalCode = vendor?.PostalCode,
127
128         MatchedProducts = matchedProducts.Distinct().ToList()
129     });
130 }
131
132 return results
133     .OrderByDescending(x => x.FinalScore)
134     .Take(5)
135     .ToList();
136
137
138
139
140 private static double CalculateDistanceKm(double lat1, double lon1, double lat2, double lon2)
141 {
142     double R = 6371;
143     double dLat = DegreesToRadians(lat2 - lat1);
144     double dLon = DegreesToRadians(lon2 - lon1);
145
146     double a =
147         Math.Sin(dLat / 2) * Math.Sin(dLat / 2) +
148         Math.Cos(DegreesToRadians(lat1)) * Math.Cos(DegreesToRadians(lat2)) *
149         Math.Sin(dLon / 2) * Math.Sin(dLon / 2);
150
151     double c = 2 * Math.Atan2(Math.Sqrt(a), Math.Sqrt(1 - a));
152     return R * c;
153 }
154
155 private static double DegreesToRadians(double deg) => deg * Math.PI / 180.0;
156 }

```

- *FarmerController.cs*

```
.env MLNET_SSA.sql productsforforecast.sql FarmerRecommendationService.cs M FarmerController.cs
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > FarmerController.cs

1  using FarmVendor.Api.Data;
2  using FarmVendor.Api.Models.DTOS;
3  using FarmVendor.Api.Services;
4  using Microsoft.AspNetCore.Authorization;
5  using Microsoft.AspNetCore.Mvc;
6  using Microsoft.EntityFrameworkCore;
7  using System.Security.Claims;
8
9  namespace FarmVendor.Api.Controllers;
10
11 [ApiController]
12 [Route("api/farmer")]
13 [Authorize] // requires JWT
14 public class FarmerController : ControllerBase
15 {
16     private readonly AppDbContext _db;
17     private readonly FarmerRecommendationService _recommendationService;
18
19     public FarmerController(AppDbContext db, FarmerRecommendationService recommendationService)
20     {
21         _db = db;
22         _recommendationService = recommendationService;
23     }
24
25     private string GetUserId()
26     => User.FindFirstValue(ClaimTypes.NameIdentifier)
27         ?? User.FindFirstValue("sub")
28         ?? "";
29
30     // 1) Stats for dashboard
31     [HttpGet("dashboard/stats")]
32     public async Task<ActionResult> GetStats()
33     {
34         var farmerId = GetUserId();
35         if (string.IsNullOrEmpty(farmerId)) return Unauthorized();
36
37         var now = DateTime.UtcNow;
38         var soon = now.AddDays(7);
39
40         var availableProducts = await _db.InventoryLot
41             .Where(l => l.FarmerId == farmerId && l.QuantityAvailable > 0)
42             .Select(l => l.ProductId)
43             .Distinct()
44             .CountAsync();
45
46         var expiringSoon = await _db.InventoryLot
47             .Where(l => l.FarmerId == farmerId && l.ExpiryDate != null && l.ExpiryDate <= soon)
48             .CountAsync();
49
50         var upcomingRequests = await _db.DemandRequest
51             .Where(r => r.Status == "Open" && r.NeededBy >= now)
52             .CountAsync();
53
54         return Ok(new
55         {
56             availableProducts,
57             expiringSoon,
58             upcomingRequests
59         });
60     }
61
62     // 2) Current stock table
63     [HttpGet("dashboard/stock")]
64     public async Task<ActionResult> GetStock()
65     {
66         var farmerId = GetUserId();
```

```

.env MLNET_SSA.sql productsforforecast.sql FarmerRecommendationService.cs M FarmerController.cs
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > FarmerController.cs
151 }
229 {
232     var rows = await _db.DemandRequest
254     return Ok(rows);
255 }
256
257 // 8) Requests history
258 [HttpGet("requests/history")]
259 public async Task<IActionResult> GetRequestHistory()
260 {
261     var rows = await _db.DemandRequest
262         .Where(r => r.Status != "Open")
263         .Include(r => r.Product)
264         .Include(r => r.Vendor)
265         .OrderByDescending(r => r.NeededBy)
266         .Take(50)
267         .Select(r => new FarmerDemandRequestRowDto
268         {
269             DemandRequestId = r.DemandRequestId,
270             ProductId = r.ProductId,
271             Product = r.Product.Name,
272             Qty = r.QuantityRequested,
273             Unit = r.Unit,
274             NeededBy = r.NeededBy,
275             Status = r.Status,
276             VendorId = r.VendorId,
277             VendorName = r.Vendor != null ? r.Vendor.DisplayName : null,
278             VendorEmail = r.Vendor != null ? r.Vendor.Email : null
279         })
280         .ToListAsync();
281     return Ok(rows);
282 }
283
284
285 // 9) pp recommended vendors for logged-in farmer
286 [HttpGet("recommended-vendors")]
287 public async Task<IActionResult> GetRecommendedVendors([FromQuery] DateTime? forecastDate)
288 {
289     var farmerId = GetUserId();
290
291     if (string.IsNullOrEmpty(farmerId))
292         return Unauthorized();
293
294     var date = forecastDate?.Date ?? DateTime.UtcNow.Date;
295
296     var rows = await _recommendationService.GetTopVendorsForFarmerAsync(farmerId, date);
297     return Ok(rows);
298 }
299
300
301 private static double CalculateDistanceKm(double lat1, double lon1, double lat2, double lon2)
302 {
303     double R = 6371;
304     double dLat = DegreesToRadians(lat2 - lat1);
305     double dLon = DegreesToRadians(lon2 - lon1);
306
307     double a =
308         Math.Sin(dLat / 2) * Math.Sin(dLat / 2) +
309         Math.Cos(DegreesToRadians(lat1)) * Math.Cos(DegreesToRadians(lat2)) *
310         Math.Sin(dLon / 2) * Math.Sin(dLon / 2);
311
312     double c = 2 * Math.Atan2(Math.Sqrt(a), Math.Sqrt(1 - a));
313     return R * c;
314 }
315
316 private static double DegreesToRadians(double deg) => deg * Math.PI / 180.0;

```


- Haversine formula applied in FarmerController.cs

```

151 }
260 {
261     var rows = await _db.DemandRequest
268     {
275         Status = r.Status,
276         VendorId = r.VendorId,
277         VendorName = r.Vendor != null ? r.Vendor.DisplayName : null,
278         VendorEmail = r.Vendor != null ? r.Vendor.Email : null
279     }
280     .ToListAsync();
281
282     return Ok(rows);
283 }
284
285 // 9) Top recommended vendors for logged-in farmer
286 [HttpGet("recommended-vendors")]
287 public async Task<IActionResult> GetRecommendedVendors([FromQuery] DateTime? forecastDate)
288 {
289     var farmerId = GetUserId();
290
291     if (string.IsNullOrEmpty(farmerId))
292         return Unauthorized();
293
294     var date = forecastDate?.Date ?? DateTime.UtcNow.Date;
295
296     var rows = await _recommendationService.GetTopVendorsForFarmerAsync(farmerId, date);
297     return Ok(rows);
298 }
299
300 //Haversine distance calculation logic
301 private static double CalculateDistanceKm(double lat1, double lon1, double lat2, double lon2)
302 {
303     double R = 6371;
304     double dLat = DegreesToRadians(lat2 - lat1);
305     double dLon = DegreesToRadians(lon2 - lon1);
306
307     double a =
308         Math.Sin(dLat / 2) * Math.Sin(dLat / 2) +
309         Math.Cos(DegreesToRadians(lat1)) * Math.Cos(DegreesToRadians(lat2)) *
310         Math.Sin(dLon / 2) * Math.Sin(dLon / 2);
311
312     double c = 2 * Math.Atan2(Math.Sqrt(a), Math.Sqrt(1 - a));
313     return R * c;
314 }
315
316 private static double DegreesToRadians(double deg) => deg * Math.PI / 180.0;

```

- *RelationshipScoreService.cs*

```

RelationshipScoreService.cs X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > RelationshipScoreService.cs

1  using FarmVendor.Api.Data;
2  using FarmVendor.Api.Models;
3  using Microsoft.EntityFrameworkCore;
4
5  namespace FarmVendor.Api.Services;
6
7  public class RelationshipScoreService
8  {
9      private readonly AppDbContext _db;
10
11     public RelationshipScoreService(AppDbContext db)
12     {
13         _db = db;
14     }
15
16     public async Task UpdateRelationshipScoreAsync(string farmerId, string vendorId)
17     {
18         if (string.IsNullOrEmpty(farmerId) || string.IsNullOrEmpty(vendorId))
19             return;
20
21         // All dispatches between this farmer and vendor
22         var dispatches = await _db.Dispatch
23             .AsNoTracking()
24             .Include(d => d.DemandRequest)
25             .Where(d => d.FarmerId == farmerId && d.VendorId == vendorId)
26             .ToListAsync();
27
28         if (dispatches.Count == 0)
29             return;
30
31         int totalDispatches = dispatches.Count;
32         int deliveredCount = dispatches.Count(d => d.DeliveryStatus == "Delivered");
33         int cancelledCount = dispatches.Count(d => d.DeliveryStatus == "Cancelled");
34
35         int onTimeDeliveredCount = dispatches.Count(d =>
36             d.DeliveryStatus == "Delivered" &&
37             d.DemandRequest != null &&
38             d.DispatchDate <= d.DemandRequest.NeededBy);
39
40         decimal totalRequestedQty = dispatches
41             .Where(d => d.DemandRequest != null)
42             .Sum(d => d.DemandRequest!.QuantityRequested);
43
44         decimal totalDeliveredQty = dispatches
45             .Where(d => d.DeliveryStatus == "Delivered")
46             .Sum(d => d.QuantityDispatched);
47
48         decimal onTimeRate = deliveredCount == 0
49             ? 0
50             : (decimal)onTimeDeliveredCount / deliveredCount;
51
52         decimal fulfillmentRate = totalRequestedQty == 0
53             ? 0
54             : totalDeliveredQty / totalRequestedQty;
55
56         decimal cancellationRate = totalDispatches == 0
57             ? 0
58             : (decimal)cancelledCount / totalDispatches;
59
60         // Weighted score out of 100
61         // need to adjust weights later for experiments
62         decimal score =
63             (onTimeRate * 50m) + // 50%
64             (fulfillmentRate * 40m) + // 40%
65             ((1 - cancellationRate) * 10m); // 10%
66

```

```

RelationshipScoreService.cs X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > RelationshipScoreService.cs
8      {
17     {
35         int onTimeDeliveredCount = dispatches.Count(d =>
36             d.DeliveryStatus == "Delivered" &&
37             d.DemandRequest != null &&
38             d.DispatchDate <= d.DemandRequest.NeededBy);
39
40         decimal totalRequestedQty = dispatches
41             .Where(d => d.DemandRequest != null)
42             .Sum(d => d.DemandRequest!.QuantityRequested);
43
44         decimal totalDeliveredQty = dispatches
45             .Where(d => d.DeliveryStatus == "Delivered")
46             .Sum(d => d.QuantityDispatched);
47
48         decimal onTimeRate = deliveredCount == 0
49             ? 0
50             : (decimal)onTimeDeliveredCount / deliveredCount;
51
52         decimal fulfillmentRate = totalRequestedQty == 0
53             ? 0
54             : totalDeliveredQty / totalRequestedQty;
55
56         decimal cancellationRate = totalDispatches == 0
57             ? 0
58             : (decimal)cancelledCount / totalDispatches;
59
60         // Weighted score out of 100
61         // need to adjust weights later for experiments
62         decimal score =
63             (onTimeRate * 50m) +           // 50%
64             (fulfillmentRate * 40m) +      // 40%
65             ((1 - cancellationRate) * 10m); // 10%
66
67         score = Math.Round(score, 2);
68
69         var existing = await _db.RelationshipStat
70             .FirstOrDefaultAsync(r => r.FarmerId == farmerId && r.VendorId == vendorId);
71
72         if (existing == null)
73         {
74             existing = new RelationshipStat
75             {
76                 FarmerId = farmerId,
77                 VendorId = vendorId
78             };
79             _db.RelationshipStat.Add(existing);
80         }
81
82         existing.TotalDispatches = totalDispatches;
83         existing.DeliveredCount = deliveredCount;
84         existing.CancelledCount = cancelledCount;
85         existing.OnTimeDeliveredCount = onTimeDeliveredCount;
86         existing.TotalRequestedQty = Math.Round(totalRequestedQty, 2);
87         existing.TotalDeliveredQty = Math.Round(totalDeliveredQty, 2);
88         existing.RelationshipScore = score;
89         existing.LastUpdatedAt = DateTime.UtcNow;
90
91         await _db.SaveChangesAsync();
92     }
93 }

```

Database queries

- RelationshipStat table

```
5
6 SELECT COLUMN_NAME
7 FROM INFORMATION_SCHEMA.COLUMNS
8 WHERE TABLE_NAME = 'RelationshipStat';
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Results (1) Messages

COLUMN_NAME
RelationshipStatId
FarmerId
VendorId
SuccessfulDispatchCount
CancelledDispatchCount
LastDispatchDate
UpdatedAt
CancelledCount
DeliveredCount
LastUpdatedAt
OnTimeDeliveredCount
RelationshipScore
TotalDeliveredQty
TotalDispatches
TotalRequestedQty

Feature 3: Realtime chatmodule

This feature enables real-time communication between farmers and vendors, supporting quick coordination, clarification, and negotiation during dispatch and fulfillment.

This feature includes:

- Create conversation
- Conversation list
- Real-time message sending
- Persistent message history
- Farmer and Vendor chat pages

The design consists of:

- SignalR-based live communication
- conversation-based organization
- stored message history
- authenticated hub connection

Implementation

Frontend

- FarmerDashboard.jsx
- VendorChat.jsx
- FarmerChat.jsx
- ChatBox.jsx
- chatApi.js

Backend

- ChatController.cs
- ChatHub.cs
- ChatService.cs

Database

- Conversation table storing:
 - FarmerId
 - VendorId
 - CreatedAt
- ChatMessage table storing:
 - ConversationId
 - SenderId
 - SenderRole
 - MessageText
 - SentAt
 - IsRead

Technical Methods

- SignalR WebSocket / fallback transport
- JWT-secured hub authentication
- real-time broadcast using conversation groups

App

- farmerDashboard

Upcoming Vendor Requests

Refresh

Product	Qty	Needed By
 Tomatoes	48 kg	2026-04-10

Vendor Details

 **Tomatoes**
48 kg

Vendor: Vendor 3
Email: vendor3@test.com
Distance: 0.27 km
Address: Surrey, BC, V3T0A1
Needed By: 2026-04-10

Chat with Vendor 3

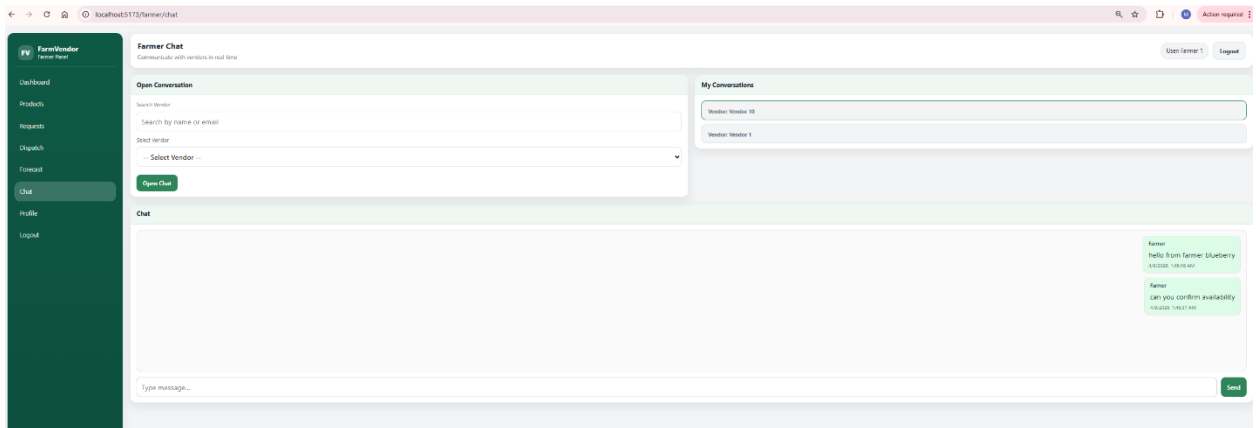
 Potatoes	22 kg	2026-04-10
 Milk	32 L	2026-04-10
 Onions	50 kg	2026-04-10
 Potatoes	10 kg	2026-04-11
 Tomatoes	24 kg	2026-04-12
 Milk	56 L	2026-04-12
 Onions	38 kg	2026-04-13
 Onions	25 kg	2026-04-13
 Onions	10 kg	2026-04-13

Add Product Lot

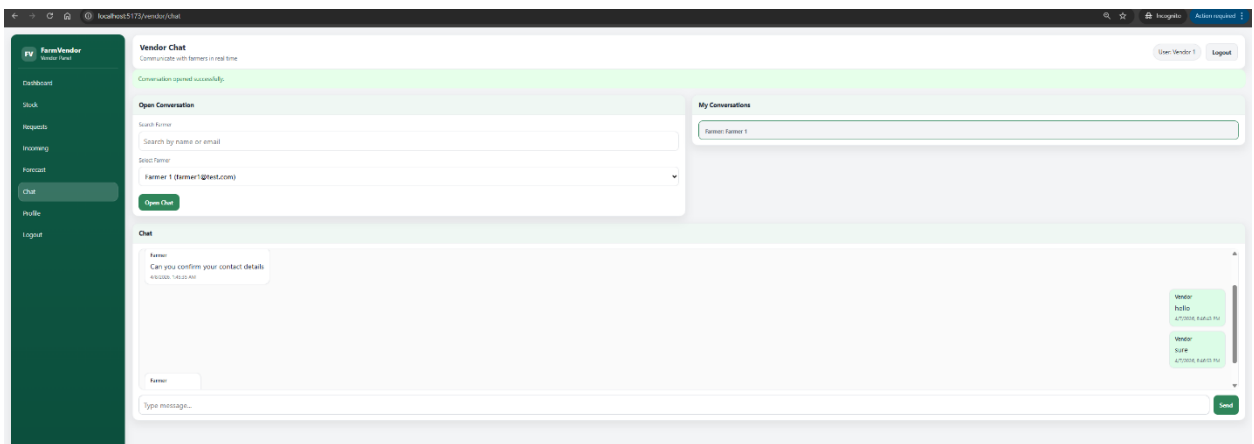
Create Dispatch

Requests loaded: 10

- Farmer chat panel



- Vendor chat panel



Frontend

- FarmerDashboard.jsx

```
relationshipStatsql U • chartDataSql • FarmerDashboard.jsx X
implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > FarmerDashboard
Function FarmerDashboard() {
  </button>
</div>

<div className="card-body">
  <table className="table">
    <thead>
      <tr>
        <th>Product</th>
        <th>Qty</th>
        <th>Needed By</th>
      </tr>
    </thead>
    <tbody>
      {requests.length === 0 ? (
        <tr>
          <td colspan="3">No open demand requests found.</td>
        </tr>
      ) : (
        requests.map((row, idx) => (
          <React.Fragment key={row.demandRequestId ?? idx}>
            <tr>
              className="request-hover-row"
              onMouseEnter={() => handleRequestMouseEnter(row, idx)}
              onMouseLeave={handleRequestMouseLeave}
            >
              <td>{renderProductCell(row)}</td>
              <td>
                {row.qty} {row.unit}
              </td>
              <td>{new Date(row.neededBy).toISOString().slice(0, 10)}</td>
            </tr>

            {hoveredRowIndex === idx && hoveredRequest && (
              <tr>
                className="request-hover-inline-row"
                onMouseEnter={() => handleRequestMouseEnter(row, idx)}
                onMouseLeave={handleRequestMouseLeave}
              >
                <td colspan="3">
                  <div className="request-hover-inline-card">
                    <div className="request-hover-card-header">
                      <div className="request-hover-card-title">Vendor Details</div>
                    </div>

                    <div className="request-hover-card-body">
                      <div className="request-hover-product">
                        {hoveredRequest.imageThumbUrl || hoveredRequest.imageUrl ? (
                          <img
                            src={hoveredRequest.imageThumbUrl || hoveredRequest.imageUrl}
                            alt={hoveredRequest.product}
                            className="request-hover-thumb"
                          />
                        ) : (
                          <div className="request-hover-placeholder">
                            {hoveredRequest.product?.charAt(0)?.toUpperCase() || "P"}
                          </div>
                        )}
                      </div>
                    </div>
                  </div>
                </td>
              </tr>
            )}
          </React.Fragment>
        )
      )}
    </tbody>
  </table>
</div>
<div className="request-hover-product-card">
```

```
<button
  type="button"
  className="btn btn-primary request-hover-chat-btn"
  onClick={() => startVendorChat(hoveredRequest)}
  disabled={startingChat}
>
  {startingChat
    ? "Opening..."
    : `Chat with ${hoveredRequest.vendorName || "Vendor"}`}
</button>
```

```

FarmerController.cs M  relationshipStat.sql U  chartData.sql  FarmerDashboard.jsx X
4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx >
1  export default function FarmerDashboard() {
2    const renderDispatchRequestOptionLabel = (option) => {
3      <div className="dispatch-select-name">
4        {option.product}
5      </div>
6      <div className="dispatch-select-meta">
7        {option.qty} {option.unit} • Needed:{" "}
8        {new Date(option.neededBy).toISOString().slice(0, 10)}
9      </div>
10     </div>
11   };
12   const handleRequestMouseEnter = (row, idx) => {
13     setHoveredRequest(row);
14     setHoveredRowIndex(idx);
15   };
16   const handleRequestMouseLeave = () => {
17     setHoveredRequest(null);
18     setHoveredRowIndex(null);
19   };
20   //function to start chat
21   const startVendorChat = async (requestRow) => {
22     if (!requestRow?.vendorId) {
23       setPageError("Vendor is not available for chat.");
24       return;
25     }
26     setStartingChat(true);
27     setPageError("");
28     try {
29       const result = await createConversation(
30         { vendorId: requestRow.vendorId },
31         token
32       );
33       // Option 1: navigate to farmer chat page with conversation id
34       navigate("/farmer/chat", {
35         state: {
36           conversationId: result.conversationId,
37           vendorId: requestRow.vendorId,
38           vendorName: requestRow.vendorName,
39         },
40       });
41       // Option 2:
42       // if your FarmerChat page reads query params instead of state, use:
43       // navigate(`/farmer/chat?conversationId=${result.conversationId}`);
44     } catch (err) {
45       setPageError(err?.message || "Failed to start chat.");
46     } finally {
47       setStartingChat(false);
48     }
49   };
50 }

```

- FarmerChat.jsx

```

FarmerController.cs M  relationshipStat.sql U  chartData.sql M  FarmerChat.jsx X
26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerChat.jsx > ...
1  import React, { useEffect, useState } from "react";
2  import { useNavigate } from "react-router-dom";
3  import "../styles/dashboard.css";
4  import ChatBox from "../assets/components/ChatBox";
5  import FarmerSidebar from "../assets/components/FarmerSidebar";
6  import {
7    createConversation,
8    getMyConversations,
9    getChatUsers,
10 } from "../assets/chatApi";
11
12 export default function FarmerChat() {
13   const navigate = useNavigate();
14   const token = localStorage.getItem("token");
15   const displayName = localStorage.getItem("displayName") || "Farmer";
16   const currentUserId = localStorage.getItem("userId") || "";
17
18   const [error, setError] = useState("");
19   const [success, setSuccess] = useState("");
20   const [loading, setLoading] = useState(true);
21
22   const [conversations, setConversations] = useState([]);
23   const [selectedConversationId, setSelectedConversationId] = useState(null);
24
25   const [vendors, setVendors] = useState([]);
26   const [vendorSearch, setVendorSearch] = useState("");
27   const [selectedVendorId, setSelectedVendorId] = useState("");
28
29   const logout = () => {
30     localStorage.clear();
31     navigate("/login");
32   };
33
34   const loadConversations = async () => {
35     const data = await getMyConversations(token);
36     setConversations(data);
37     if (data.length > 0 && !selectedConversationId) {
38       setSelectedConversationId(data[0].conversationId);
39     }
40   };
41
42   const loadVendors = async (search = "") => {
43     const data = await getChatUsers("Vendor", token, search);
44     setVendors(data);
45   };
46
47   const handleCreateConversation = async (e) => {
48     e.preventDefault();
49     setError("");
50     setSuccess("");
51
52     if (!selectedVendorId) {
53       setError("Please select a vendor.");
54       return;
55     }
56
57     try {
58       const result = await createConversation(
59         { vendorId: selectedVendorId },
60         token
61       );

```

- VendorChat.jsx

```

FarmerController.cs M  relationshipStat.sql U  chartData.sql M  VendorChat.jsx X
26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > VendorCh

1  import React, { useEffect, useState } from "react";
2  import { useNavigate } from "react-router-dom";
3  import "../styles/dashboard.css";
4  import ChatBox from "../assets/components/ChatBox";
5  import VendorSidebar from "../assets/components/VendorSidebar";
6  import {
7      createConversation,
8      getMyConversations,
9      getChatUsers,
10 } from "../assets/chatApi";
11
12 export default function VendorChat() {
13     const navigate = useNavigate();
14     const token = localStorage.getItem("token");
15     const displayName = localStorage.getItem("displayName") || "Vendor";
16     const currentUserId = localStorage.getItem("userId") || "";
17
18     const [error, setError] = useState("");
19     const [success, setSuccess] = useState("");
20     const [loading, setLoading] = useState(true);
21
22     const [conversations, setConversations] = useState([]);
23     const [selectedConversationId, setSelectedConversationId] = useState(null);
24
25     const [farmers, setFarmers] = useState([]);
26     const [farmerSearch, setFarmerSearch] = useState("");
27     const [selectedFarmerId, setSelectedFarmerId] = useState("");
28
29     const logout = () => {
30         localStorage.clear();
31         navigate("/login");
32     };
33
34     const loadConversations = async () => {
35         const data = await getMyConversations(token);
36         setConversations(data);
37
38         if (data.length > 0 && !selectedConversationId) {
39             setSelectedConversationId(data[0].conversationId);
40         }
41     };
42
43     const loadFarmers = async (search = "") => {
44         const data = await getChatUsers("Farmer", token, search);
45         setFarmers(data);
46     };
47
48     const handleCreateConversation = async (e) => {
49         e.preventDefault();
50         setError("");
51         setSuccess("");
52
53         if (!selectedFarmerId) {
54             setError("Please select a farmer.");
55             return;
56         }
57
58         try {
59             const result = await createConversation(

```

```

FarmerController.cs M  relationshipStats.sql U  chartData.sql M  JS chatApi.js X
26.4495_S3_ManeeshaE > implementations > FarmVendor > farmvender-web > src > assets > JS chatApi.js > ...
1  const API_BASE = import.meta.env.VITE_API_URL;
2
3  export async function createConversation(payload, token) {
4      const res = await fetch(`${API_BASE}/api/chat/conversation`, {
5          method: "POST",
6          headers: {
7              "Content-Type": "application/json",
8              Authorization: `Bearer ${token}`,
9          },
10         body: JSON.stringify(payload),
11     });
12
13     const text = await res.text();
14     if (!res.ok) throw new Error(text || "Failed to create conversation");
15     return JSON.parse(text);
16 }
17
18 export async function getMyConversations(token) {
19     const res = await fetch(`${API_BASE}/api/chat/conversations`, {
20         headers: {
21             Authorization: `Bearer ${token}`,
22         },
23     });
24
25     const text = await res.text();
26     if (!res.ok) throw new Error(text || "Failed to load conversations");
27     return JSON.parse(text);
28 }
29
30 export async function getConversationMessages(conversationId, token) {
31     const res = await fetch(`${API_BASE}/api/chat/messages/${conversationId}`, {
32         headers: {
33             Authorization: `Bearer ${token}`,
34         },
35     });
36
37     const text = await res.text();
38     if (!res.ok) throw new Error(text || "Failed to load messages");
39     return JSON.parse(text);
40 }
41
42 export async function getChatUsers(role, token, search = "") {
43     const url = new URL(`${API_BASE}/api/chat/users`);
44     url.searchParams.append("role", role);
45     if (search.trim()) {
46         url.searchParams.append("search", search.trim());
47     }
48
49     const res = await fetch(url, {
50         headers: {
51             Authorization: `Bearer ${token}`,
52         },
53     });
54
55     const text = await res.text();
56     if (!res.ok) throw new Error(text || "Failed to load users");
57     return JSON.parse(text);
58 }

```

- ChatBox.jsx

```

1  import React, { useEffect, useRef, useState } from "react";
2  import * as signalR from "@microsoft/signalr";
3  import { getConversationMessages } from "../chatApi";
4
5  const API_BASE = import.meta.env.VITE_API_URL;
6
7  export default function ChatBox({ conversationId, token, currentUserId }) {
8    const [messages, setMessages] = useState([]);
9    const [text, setText] = useState("");
10   const [connection, setConnection] = useState(null);
11   const [error, setError] = useState("");
12   const bottomRef = useRef(null);
13
14   useEffect(() => {
15     if (!conversationId || !token) return;
16
17     const load = async () => {
18       try {
19         const existing = await getConversationMessages(conversationId, token);
20         setMessages(existing);
21       } catch (e) {
22         setError(e?.message || "Failed to load messages");
23       }
24     };
25
26     load();
27   }, [conversationId, token]);
28
29   useEffect(() => {
30     if (!conversationId || !token) return;
31
32     const newConnection = new signalR.HubConnectionBuilder()
33       .withUrl(`${API_BASE}/hubs/chat`, {
34         accessTokenFactory: () => token,
35       })
36       .withAutomaticReconnect()
37       .build();
38
39     setConnection(newConnection);
40
41     return () => {
42       newConnection.stop();
43     };
44   }, [conversationId, token]);
45
46   useEffect(() => {
47     if (!connection) return;
48
49     connection
50       .start()
51       .then(async () => {
52         await connection.invoke("JoinConversation", `conversation-${conversationId}`);
53
54         connection.on("ReceiveMessage", (message) => {
55           setMessages((prev) => [...prev, message]);
56         });
57       })
58       .catch((err) => setError(err.message));
59
60     return () => {
61       if (connection) {

```

Backend

- ChatController.cs

```
ChatController.cs X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > ChatController.cs

1  using FarmVendor.Api.Data;
2  using FarmVendor.Api.Models;
3  using FarmVendor.Api.Models.DTOS;
4  using FarmVendor.Api.Services;
5  using Microsoft.AspNetCore.Authorization;
6  using Microsoft.AspNetCore.Identity;
7  using Microsoft.AspNetCore.Mvc;
8  using Microsoft.EntityFrameworkCore;
9  using System.Security.Claims;
10
11 namespace FarmVendor.Api.Controllers;
12
13 [ApiController]
14 [Route("api/chat")]
15 [Authorize]
16 public class ChatController : ControllerBase
17 {
18     private readonly ChatService _chatService;
19     private readonly AppDbContext _db;
20     private readonly UserManager<ApplicationUser> _userManager;
21
22     public ChatController(
23         ChatService chatService,
24         AppDbContext db,
25         UserManager<ApplicationUser> userManager)
26     {
27         _chatService = chatService;
28         _db = db;
29         _userManager = userManager;
30     }
31
32     [HttpGet("users")]
33     public async Task<ActionResult> GetUsersByRole([FromQuery] string role, [FromQuery] string? search = null)
34     {
35         if (string.IsNullOrEmpty(role))
36             return BadRequest("Role is required.");
37
38         var usersInRole = await _userManager.GetUsersInRoleAsync(role);
39
40         var query = usersInRole.AsQueryable();
41
42         if (!string.IsNullOrEmpty(search))
43         {
44             var term = search.Trim().ToLower();
45             query = query.Where(u =>
46                 (u.DisplayName != null && u.DisplayName.ToLower().Contains(term)) ||
47                 (u.Email != null && u.Email.ToLower().Contains(term)));
48         }
49
50         var result = query
51             .OrderBy(u => u.DisplayName)
52             .Select(u => new ChatUserLookupDto
53             {
54                 Id = u.Id,
55                 DisplayName = u.DisplayName ?? "",
56                 Email = u.Email ?? "",
57                 Role = role
58             })
59             .ToList();
60     }
61 }
```

```
17  {
34  {
62  }
63
64  [HttpPost("conversation")]
65  public async Task<IActionResult> CreateConversation([FromBody] CreateConversationDto dto)
66  {
67      var currentUserId =
68          User.FindFirstValue(ClaimTypes.NameIdentifier) ??
69          User.FindFirstValue("sub");
70
71      var currentRole =
72          User.FindFirstValue(ClaimTypes.Role) ??
73          User.FindFirst("http://schemas.microsoft.com/ws/2008/06/identity/claims/role")?.Value;
74
75      if (string.IsNullOrEmpty(currentUserId))
76          return Unauthorized("User ID not found in token.");
77
78      string farmerId;
79      string vendorId;
80
81      if (string.Equals(currentRole, "Farmer", StringComparison.OrdinalIgnoreCase))
82      {
83          if (string.IsNullOrEmpty(dto.VendorId))
84              return BadRequest("VendorId is required.");
85
86          farmerId = currentUserId;
87          vendorId = dto.VendorId.Trim();
88      }
89      else if (string.Equals(currentRole, "Vendor", StringComparison.OrdinalIgnoreCase))
90      {
91          if (string.IsNullOrEmpty(dto.FarmerId))
92              return BadRequest("FarmerId is required.");
93
94          farmerId = dto.FarmerId.Trim();
95          vendorId = currentUserId;
96      }
97      else
98      {
99          return BadRequest("Unsupported role.");
100     }
101
102     var farmerExists = await _db.Users.AnyAsync(u => u.Id == farmerId);
103     var vendorExists = await _db.Users.AnyAsync(u => u.Id == vendorId);
104
105     if (!farmerExists)
106         return BadRequest($"FarmerId '{farmerId}' does not exist.");
107     if (!vendorExists)
108         return BadRequest($"VendorId '{vendorId}' does not exist.");
109
110     var conversation = await _chatService.CreateOrGetConversationAsync(farmerId, vendorId);
111
112     return Ok(new
113     {
114         conversationId = conversation.ConversationId,
115         farmerId = conversation.FarmerId,
116         vendorId = conversation.VendorId
117     });
118 }
119
120 // [HttpGet("conversations")]
```



```
17 {
129 // {
130 //     conversationId = c.ConversationId,
131 //     farmerId = c.FarmerId,
132 //     vendorId = c.VendorId,
133 //     createdAt = c.CreatedAt
134 // });
135 // }
136
137 [HttpGet("conversations")]
138 public async Task<IActionResult> GetMyConversations()
139 {
140     var userId = User.FindFirstValue(ClaimTypes.NameIdentifier) ?? User.FindFirstValue("sub");
141     if (string.IsNullOrEmpty(userId))
142         return Unauthorized();
143
144     var conversations = await _db.Conversation
145         .Include(c => c.Farmer)
146         .Include(c => c.Vendor)
147         .OrderByDescending(c => c.CreatedAt)
148         .ToListAsync();
149
150     var result = conversations
151         .Where(c => c.FarmerId == userId || c.VendorId == userId)
152         .Select(c => new
153         {
154             conversationId = c.ConversationId,
155             farmerId = c.FarmerId,
156             vendorId = c.VendorId,
157
158             farmerName = c.Farmer.DisplayName,
159             vendorName = c.Vendor.DisplayName,
160
161             createdAt = c.CreatedAt
162         });
163
164     return Ok(result);
165 }
166
167 [HttpGet("messages/{conversationId:int}")]
168 public async Task<IActionResult> GetMessages(int conversationId)
169 {
170     var messages = await _chatService.GetMessagesAsync(conversationId);
171     return Ok(messages);
172 }
173 }
```

- ChatHub.cs

```
ChatHub.cs X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Hubs > ChatHub.cs

1  using FarmVendor.Api.Services;
2  using Microsoft.AspNetCore.Authorization;
3  using Microsoft.AspNetCore.SignalR;
4  using System.Security.Claims;
5
6  namespace FarmVendor.Api.Hubs;
7
8  [Authorize]
9  public class ChatHub : Hub
10 {
11     private readonly ChatService _chatService;
12
13     public ChatHub(ChatService chatService)
14     {
15         _chatService = chatService;
16     }
17
18     public async Task JoinConversation(string conversationKey)
19     {
20         await Groups.AddToGroupAsync(Context.ConnectionId, conversationKey);
21     }
22
23     public async Task LeaveConversation(string conversationKey)
24     {
25         await Groups.RemoveFromGroupAsync(Context.ConnectionId, conversationKey);
26     }
27
28     public async Task SendMessage(int conversationId, string messageText)
29     {
30         var userId = Context.User?.FindFirstValue(ClaimTypes.NameIdentifier)
31             ?? Context.User?.FindFirstValue("sub");
32
33         var role = Context.User?.FindFirstValue(ClaimTypes.Role) ?? "";
34
35         if (string.IsNullOrEmpty(userId))
36             throw new HubException("Unauthorized");
37
38         if (string.IsNullOrEmpty(messageText))
39             throw new HubException("Message cannot be empty.");
40
41         var saved = await _chatService.SaveMessageAsync(conversationId, userId, role, messageText);
42
43         var conversationKey = $"conversation-{conversationId}";
44
45         await Clients.Group(conversationKey).SendAsync("ReceiveMessage", new
46         {
47             chatMessageId = saved.ChatMessageId,
48             conversationId = saved.ConversationId,
49             senderId = saved.SenderId,
50             senderRole = saved.SenderRole,
51             messageText = saved.MessageText,
52             sentAt = saved.SentAt,
53             isRead = saved.IsRead
54         });
55     }
56 }
```

- ChatService.cs

```

ChatService.cs X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > ChatService.cs

1  using FarmVendor.Api.Data;
2  using FarmVendor.Api.Models;
3  using FarmVendor.Api.Models.DTOs;
4  using Microsoft.EntityFrameworkCore;
5
6  namespace FarmVendor.Api.Services;
7
8  public class ChatService
9  {
10     private readonly AppDbContext _db;
11
12     public ChatService(AppDbContext db)
13     {
14         _db = db;
15     }
16
17     public async Task<Conversation> CreateOrGetConversationAsync(string farmerId, string vendorId)
18     {
19         var existing = await _db.Conversation
20             .FirstOrDefaultAsync(c => c.FarmerId == farmerId && c.VendorId == vendorId);
21
22         if (existing != null) return existing;
23
24         var conversation = new Conversation
25         {
26             FarmerId = farmerId,
27             VendorId = vendorId,
28             CreatedAt = DateTime.UtcNow
29         };
30
31         _db.Conversation.Add(conversation);
32         await _db.SaveChangesAsync();
33
34         return conversation;
35     }
36
37     public async Task<List<Conversation>> GetMyConversationsAsync(string userId)
38     {
39         return await _db.Conversation
40             .Include(c => c.Messages)
41             .Where(c => c.FarmerId == userId || c.VendorId == userId)
42             .OrderByDescending(c => c.CreatedAt)
43             .ToListAsync();
44     }
45
46     public async Task<List<ChatMessageRowDto>> GetMessagesAsync(int conversationId)
47     {
48         return await _db.ChatMessage
49             .AsNoTracking()
50             .Where(m => m.ConversationId == conversationId)
51             .OrderBy(m => m.SentAt)
52             .Select(m => new ChatMessageRowDto
53             {
54                 ChatMessageId = m.ChatMessageId,
55                 ConversationId = m.ConversationId,
56                 SenderId = m.SenderId,
57                 SenderRole = m.SenderRole,
58                 MessageText = m.MessageText,
59                 SentAt = m.SentAt,

```

ChatService.cs X

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Services > ChatService.cs

```
9  {
18 {
30
31     _db.Conversation.Add(conversation);
32     await _db.SaveChangesAsync();
33
34     return conversation;
35 }
36
37 public async Task<List<Conversation>> GetMyConversationsAsync(string userId)
38 {
39     return await _db.Conversation
40         .Include(c => c.Messages)
41         .Where(c => c.FarmerId == userId || c.VendorId == userId)
42         .OrderByDescending(c => c.CreatedAt)
43         .ToListAsync();
44 }
45
46 public async Task<List<ChatMessageRowDto>> GetMessagesAsync(int conversationId)
47 {
48     return await _db.ChatMessage
49         .AsNoTracking()
50         .Where(m => m.ConversationId == conversationId)
51         .OrderBy(m => m.SentAt)
52         .Select(m => new ChatMessageRowDto
53         {
54             ChatMessageId = m.ChatMessageId,
55             ConversationId = m.ConversationId,
56             SenderId = m.SenderId,
57             SenderRole = m.SenderRole,
58             MessageText = m.MessageText,
59             SentAt = m.SentAt,
60             IsRead = m.IsRead
61         })
62         .ToListAsync();
63 }
64
65 public async Task<ChatMessage> SaveMessageAsync(int conversationId, string senderId, string senderRole, string messageText)
66 {
67     var message = new ChatMessage
68     {
69         ConversationId = conversationId,
70         SenderId = senderId,
71         SenderRole = senderRole,
72         MessageText = messageText,
73         SentAt = DateTime.UtcNow,
74         IsRead = false
75     };
76
77     _db.ChatMessage.Add(message);
78     await _db.SaveChangesAsync();
79
80     return message;
81 }
82 }
```

Database

- ChatMessage table

ChatMessage.sql U • relationshipStat.sql U

W26.4495_S3_ManeeshaE > ReportsAndDocuments > sql_queries > ChatMessage.sql

```
(localdb)\MSSQLLocalDB | FarmVendorDb
1  USE FarmVendorDb;
2  GO
3
4
5  SELECT COLUMN_NAME
6  FROM INFORMATION_SCHEMA.COLUMNS
7  WHERE TABLE_NAME = 'ChatMessage';
8
9  select * from ChatMessage
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

Results (2) Messages

	COLUMN_NAME	
1	ChatMessageId	
2	ConversationId	
3	SenderId	
4	SenderRole	
5	MessageText	
6	SentAt	
7	IsRead	

	ChatMessageId		ConversationId		SenderId		SenderRole		MessageText		SentAt		IsRead	
1	1		1		8c3c778b-055d-4130-a7f2-c78237b7e9e2		Farmer		hello this is farmer from Langley		2026-04-08 01:45:02.1745650		0	
2	2		1		8c3c778b-055d-4130-a7f2-c78237b7e9e2		Farmer		Can you confirm your contact details		2026-04-08 01:45:35.7382047		0	
3	3		2		8c3c778b-055d-4130-a7f2-c78237b7e9e2		Farmer		hello from farmer blueberry		2026-04-08 01:46:18.5229035		0	
4	4		2		8c3c778b-055d-4130-a7f2-c78237b7e9e2		Farmer		can you confirm availability		2026-04-08 01:46:31.3802766		0	
5	5		1		d0dcc510-cfc8-4d86-9e28-2612deb9c007		Vendor		hello		2026-04-08 01:46:43.2506930		0	
6	6		1		d0dcc510-cfc8-4d86-9e28-2612deb9c007		Vendor		sure		2026-04-08 01:46:53.1551771		0	
7	7		1		8c3c778b-055d-4130-a7f2-c78237b7e9e2		Farmer		ok		2026-04-08 01:47:53.3088562		0	

- Conversation Table

W26_4495_S3_ManeeshaE > ReportsAndDocuments > sql_queries > ConversationId.sql

```

1  USE FarmVendorDb;
2  GO
3
4
5  SELECT COLUMN_NAME
6  FROM INFORMATION_SCHEMA.COLUMNS
7  WHERE TABLE_NAME = 'Conversation';
8
9  select * from Conversation

```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

Results (2) Messages

	COLUMN_NAME	↑↓	🔍
1	ConversationId		
2	FarmerId		
3	VendorId		
4	CreatedAt		

	ConversationId	↑↓	🔍	FarmerId	↑↓	🔍	VendorId	↑↓	🔍	CreatedAt	↑↓	🔍
1	1			8c3c778b-055d-4130-a7f2-c78237b7e9e2			d0dcc510-cfc8-4d86-9e28-2612deb9c007			2026-04-08 01:44:28.7062458		
2	2			8c3c778b-055d-4130-a7f2-c78237b7e9e2			7eca8195-448a-4a5a-a064-b289f4335c56			2026-04-08 01:46:02.3087472		

Feature 3: Forecast Export Functionality

This feature allows users to export forecast results in CSV format for external analysis, reporting, and presentation.

This feature includes:

- Export Results button
- CSV generation from forecast table
- Downloadable file with forecast details

Implementation

Frontend

- FarmerForecast.jsx
- CSV blob generation using JavaScript
- dynamic file download handling

Backend

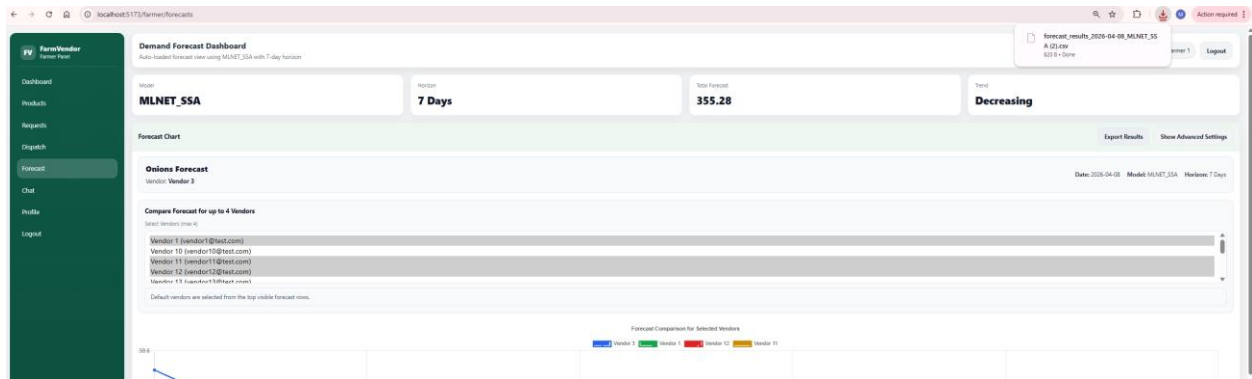
- No dedicated backend export endpoint required for current version
- data taken from forecast rows already loaded through API

Exported Fields

- Vendor Name
- Product
- Forecast Date
- Quantity
- Model
- Lookback
- Created At

App

- Farmer forecast dashboard



forecast_results_2026-04-08_MLNET_SSA (2) - Excel

File Home Insert Page Layout Formulas Data Review View Help Acrobat

Clipboard Font Alignment

POSSIBLE DATA LOSS Some features might be lost if you save this workbook in the comma-delimited (.csv) format. To preserve it

	A	B	C	D	E	F	G	H	I	J
	Vendor	Product	Forecast Date	Qty	Model	Lookback	Created At			
1	Vendor 3	Onions	2026-04-08	59.57	MLNET_SSA		2026-04-08			
2	Vendor 1	Milk	2026-04-08	11.09	MLNET_SSA		2026-04-08			
3	Vendor 12	Eggs	2026-04-08	12.77	MLNET_SSA		2026-04-08			
4	Vendor 11	Milk	2026-04-08	56.57	MLNET_SSA		2026-04-08			
5	Vendor 10	Onions	2026-04-08	29.75	MLNET_SSA		2026-04-08			
6	Vendor 15	Milk	2026-04-08	12.44	MLNET_SSA		2026-04-08			
7	Vendor 15	Onions	2026-04-08	50.56	MLNET_SSA		2026-04-08			
8	Vendor 4	Onions	2026-04-08	25.53	MLNET_SSA		2026-04-08			
9	Vendor 8	Eggs	2026-04-08	30.54	MLNET_SSA		2026-04-08			
10	Vendor 2	Onions	2026-04-08	38.55	MLNET_SSA		2026-04-08			
11	Vendor 14	Milk	2026-04-08	37.81	MLNET_SSA		2026-04-08			
12	Vendor 6	Eggs	2026-04-08	29.54	MLNET_SSA		2026-04-08			
13	Vendor 13	Eggs	2026-04-08	35.55	MLNET_SSA		2026-04-08			
14	Vendor 13	Onions	2026-04-08	58.57	MLNET_SSA		2026-04-08			
15										
16										
17										
18										
19										
20										
21										
22										
23										
24										
25										
26										
27										
28										
29										
30										
31										
32										
33										
34										
35										
36										
37										
38										
39										
40										
41										
42										
43										
44										
45										
46										
47										
48										
49										
50										
51										
52										
53										
54										

forecast_results_2026-04-08_MLN

Ready Accessibility: Unavailable

Frontend

- FarmerForecast.jsx

```
atMessage.sql ConversationId.sql FarmerForecast.jsx M X
4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerForecast.jsx > FarmerAnalytics > e
export default function FarmerAnalytics() {
  const escapeCsv = (value) => {
    return `${str.replace(/"/g, '""')}`;
  }
  return str;
};
//export forecast result
const exportForecastResultsCsv = () => {
  if (!rows || rows.length === 0) {
    setError("No forecast rows available to export.");
    return;
  }

  const header = [
    "Vendor",
    "Product",
    "Forecast Date",
    "Qty",
    "Model",
    "Lookback",
    "Created At",
  ];

  const csvRows = rows.map((r) => [
    r.vendorName || r.vendorId,
    r.productName,
    new Date(r.forecastDate).toISOString().slice(0, 10),
    r.forecastQty,
    r.modelName,
    r.lookbackPeriods ?? "",
    new Date(r.createdAt).toISOString().slice(0, 10),
  ]);

  const csvContent = [
    header.map(escapeCsv).join(","),
    ...csvRows.map((row) => row.map(escapeCsv).join(",")),
  ].join("\n");

  const blob = new Blob([csvContent], { type: "text/csv;charset=utf-8;" });
  const url = URL.createObjectURL(blob);

  const safeDate = (form.forecastDate || new Date().toISOString().slice(0, 10)).replaceAll("/", "-");
  const safeModel = (form.modelName || "forecast").replaceAll(" ", "_");
  const fileName = `forecast_results_${safeDate}_${safeModel}.csv`;

  const link = document.createElement("a");
  link.href = url;
  link.setAttribute("download", fileName);
  document.body.appendChild(link);
  link.click();
  document.body.removeChild(link);
  URL.revokeObjectURL(url);
};
```

```
ChatMessage.sql ConversationId.sql FarmerForecast.jsx M X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerForecast.jsx > FarmerAnalytics
16 export default function FarmerAnalytics() {
486
487     {error && <div className="auth-alert error">{error}</div>}
488     {success && <div className="auth-alert success">{success}</div>}
489
490     <section className="forecast-kpis" style={{ marginTop: 14 }}>
491         <div className="forecast-kpi-card">
492             <div className="forecast-kpi-label">Model</div>
493             <div className="forecast-kpi-value">MLNET_SSA</div>
494         </div>
495         <div className="forecast-kpi-card">
496             <div className="forecast-kpi-label">Horizon</div>
497             <div className="forecast-kpi-value">7 Days</div>
498         </div>
499         <div className="forecast-kpi-card">
500             <div className="forecast-kpi-label">Total Forecast</div>
501             <div className="forecast-kpi-value">{summary.total}</div>
502         </div>
503         <div className="forecast-kpi-card">
504             <div className="forecast-kpi-label">Trend</div>
505             <div className="forecast-kpi-value">{summary.trend}</div>
506         </div>
507     </section>
508
509     <section style={{ marginTop: 14 }}>
510         <div className="card">
511             <div className="card-header">
512                 <h2>Forecast Chart</h2>
513                 <div style={{ display: "flex", gap: 10, alignItems: "center" }}>
514                     <button
515                         className="btn btn-secondary"
516                         onClick={exportForecastResultsCsv}
517                         disabled={rows.length === 0}
518                     >
519                         Export Results
520                     </button>
521                     <button
522                         className="btn btn-secondary"
523                         onClick={() => setAdvancedOpen((prev) => !prev)}
524                     >
525                         {advancedOpen ? "Hide Advanced Settings" : "Show Advanced Settings"}
526                     </button>
527                 </div>
528             </div>
529
```

Feature 4: Role-Based Authentication (Registration & Login)

This feature allows users to access the system as either a Farmer or a Vendor. Authentication is implemented using ASP.NET Identity with JWT-based authorization, ensuring secure access, protected API calls, and role-based navigation.

This feature includes:

- User registration with role selection
- Secure login with JWT token generation
- Role-based dashboard access
- Route protection in the frontend

The design consists of:

- Registration page with role selection
- Login page with credential validation
- Backend JWT token generation
- Role-based route protection using React guards

Implementation

Frontend

- Register.jsx – handles registration form, role selection, and API call
- Login.jsx – handles login, stores JWT in localStorage, redirects user dashboard
- RequireAuth.jsx – protects authenticated routes
- RequireRole.jsx – restricts role access based on user type

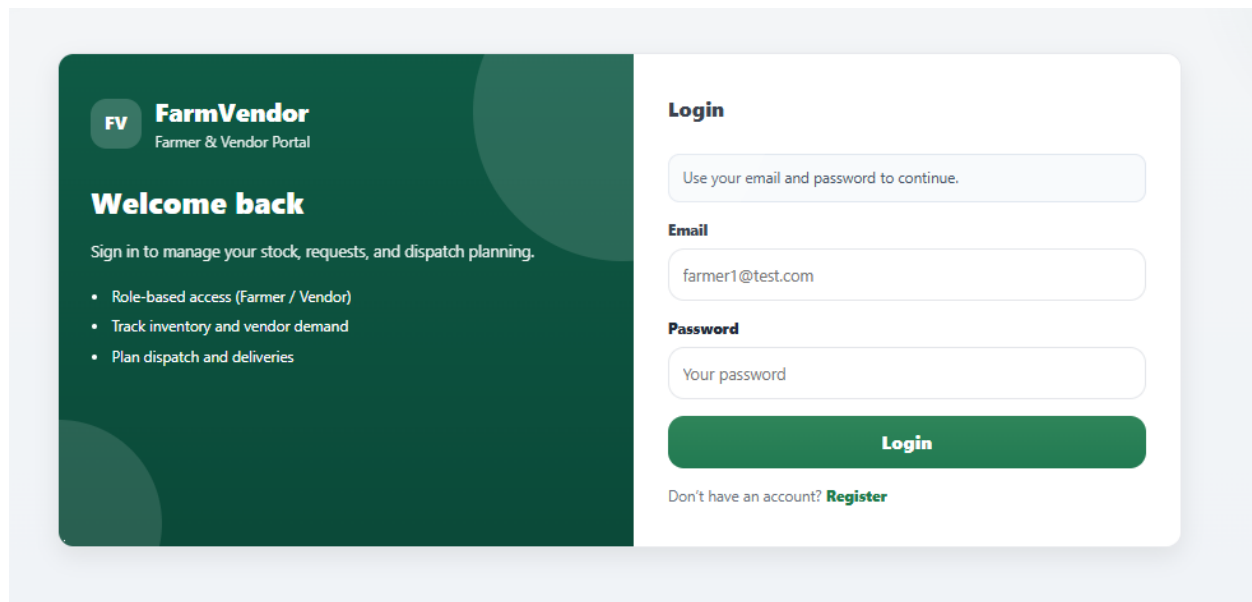
Backend

- AuthController.cs – login and register endpoints
- ApplicationUser.cs – extended Identity user with custom fields
- Program.cs – JWT authentication and authorization middleware
- JWT token generation logic using issuer, audience, and signing key

Database

- AspNetUsers table stores user accounts
- Identity role tables map users to Farmer / Vendor roles

App



The login page features a dark green sidebar on the left with the FarmVendor logo (FV) and the text "Farmer & Vendor Portal". The main content area is white. It includes a "Welcome back" message, a sign-in instruction, and a list of features: "Role-based access (Farmer / Vendor)", "Track inventory and vendor demand", and "Plan dispatch and deliveries". The login form on the right has a "Login" heading, a placeholder "Use your email and password to continue.", and input fields for "Email" (containing "farmer1@test.com") and "Password" (containing "Your password"). A green "Login" button is at the bottom, followed by a link "Don't have an account? Register".

FV FarmVendor
Farmer & Vendor Portal

Welcome back

Sign in to manage your stock, requests, and dispatch planning.

- Role-based access (Farmer / Vendor)
- Track inventory and vendor demand
- Plan dispatch and deliveries

Login

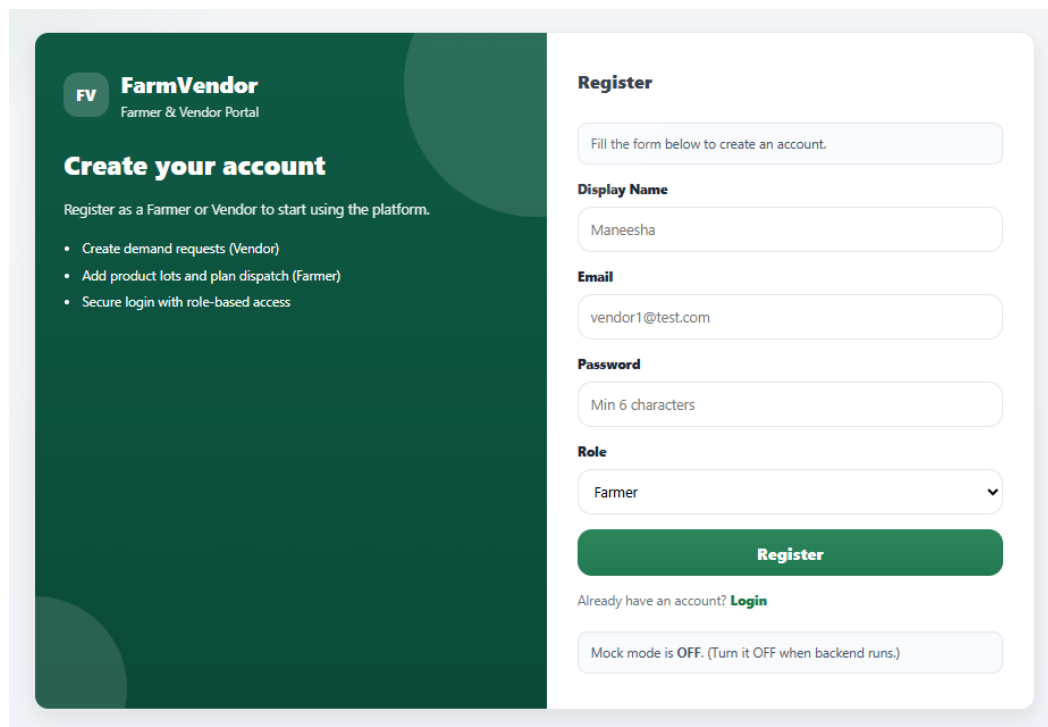
Use your email and password to continue.

Email
farmer1@test.com

Password
Your password

Login

Don't have an account? [Register](#)



The register page features a dark green sidebar on the left with the FarmVendor logo (FV) and the text "Farmer & Vendor Portal". The main content area is white. It includes a "Create your account" heading, a registration instruction, and a list of features: "Create demand requests (Vendor)", "Add product lots and plan dispatch (Farmer)", and "Secure login with role-based access". The register form on the right has a "Register" heading, a placeholder "Fill the form below to create an account.", and input fields for "Display Name" (containing "Maneesha"), "Email" (containing "vendor1@test.com"), and "Password" (containing "Min 6 characters"). There is a "Role" dropdown menu set to "Farmer". A green "Register" button is at the bottom, followed by a link "Already have an account? Login". At the very bottom, a status bar indicates "Mock mode is OFF. (Turn it OFF when backend runs.)".

FV FarmVendor
Farmer & Vendor Portal

Create your account

Register as a Farmer or Vendor to start using the platform.

- Create demand requests (Vendor)
- Add product lots and plan dispatch (Farmer)
- Secure login with role-based access

Register

Fill the form below to create an account.

Display Name
Maneesha

Email
vendor1@test.com

Password
Min 6 characters

Role
Farmer

Register

Already have an account? [Login](#)

Mock mode is OFF. (Turn it OFF when backend runs.)

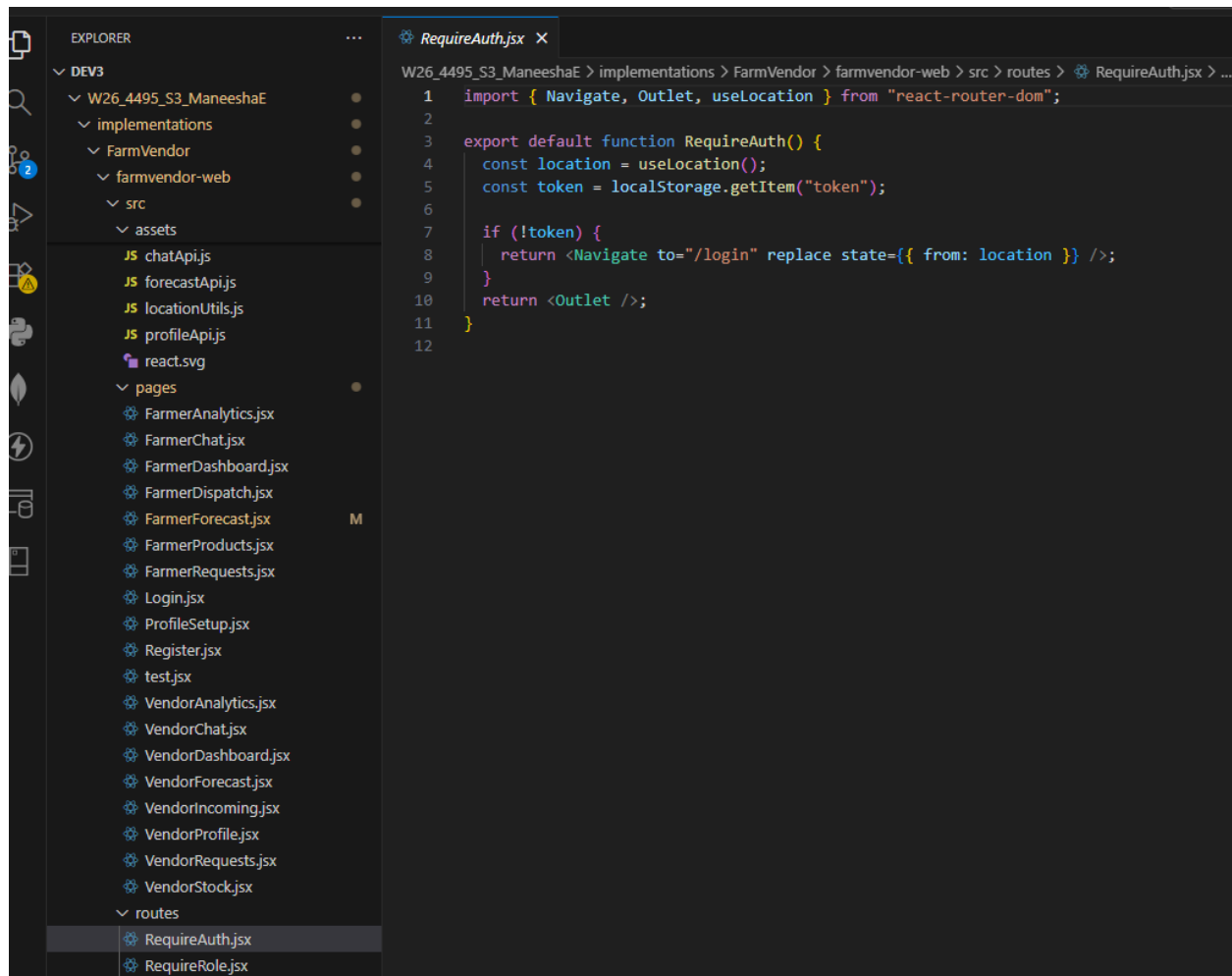
- Register.jsx

```
Register.jsx X
/26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > Register.jsx > ...
1 import React, { useState } from "react";
2 import { Link, useNavigate } from "react-router-dom";
3 import "../styles/auth.css";
4
5 const API_BASE = import.meta.env.VITE_API_URL;
6 const USE MOCK = false; //backend is running
7
8 export default function Register() {
9   const navigate = useNavigate();
10
11   const [form, setForm] = useState({
12     displayName: "",
13     email: "",
14     password: "",
15     role: "Farmer",
16   });
17
18   const [loading, setLoading] = useState(false);
19   const [error, setError] = useState("");
20   const [success, setSuccess] = useState("");
21
22   const onChange = (e) => {
23     setForm((p) => ({ ...p, [e.target.name]: e.target.value }));
24   };
25
26   const handleRegister = async (e) => {
27     e.preventDefault();
28     setError("");
29     setSuccess("");
30     setLoading(true);
31
32     try {
33       if (USE MOCK) {
34         setSuccess("Registered successfully (mock). You can login now.");
35         setTimeout(() => navigate("/login"), 800);
36         return;
37       }
38
39       const res = await fetch(`${API_BASE}/api/Auth/register`, {
40         method: "POST",
41         headers: { "Content-Type": "application/json" },
42         body: JSON.stringify(form),
43       });
44
45       if (!res.ok) { //checking for strong password
46         const text = await res.text();
47         throw new Error(`Registration failed (${res.status}): ${text}`);
48       }
49
50       setSuccess("Registered successfully. Please login.");
51       setTimeout(() => navigate("/login"), 900);
52     } catch (err) {
53       setError(err?.message || "Something went wrong");
54     } finally {
55       setLoading(false);
56     }
57   };
58
59   return (
60     <div className="auth-page">
61       <div className="auth-card">
62         {/* Left panel */}
63         <div className="auth-header">
64           <div className="auth-brand">
65             <div className="auth-logo">FV</div>
66             <div>
67               <div className="auth-title">FarmVendor</div>
```

- Login.jsx

```
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > Login.jsx > ...
1 import React, { useState } from "react";
2 import { useNavigate, Link } from "react-router-dom";
3 import "../styles/auth.css";
4
5 const API_BASE = import.meta.env.VITE_API_URL;
6 console.log("API_BASE =", API_BASE);
7 const USE MOCK = false;
8
9 export default function Login() {
10   const navigate = useNavigate();
11   const [form, setForm] = useState({ email: "", password: "" });
12   const [loading, setLoading] = useState(false);
13   const [error, setError] = useState("");
14
15   const onChange = (e) => {
16     setForm((p) => ({ ...p, [e.target.name]: e.target.value }));
17   };
18
19   const handleLogin = async (e) => {
20     e.preventDefault();
21     setError("");
22     setLoading(true);
23
24     try {
25       if (USE MOCK) {
26         const role = form.email.toLowerCase().includes("vendor") ? "Vendor" : "Farmer";
27
28         localStorage.setItem("token", "mock-jwt-token");
29         localStorage.setItem("role", role);
30         localStorage.setItem("displayName", role === "Vendor" ? "Vendor User" : "Farmer User");
31         localStorage.setItem("userId", role === "Vendor" ? "mock-vendor-id" : "mock-farmer-id");
32         localStorage.setItem("profileComplete", "false");
33
34         navigate("/profile/setup");
35         return;
36       }
37
38       const res = await fetch(`${API_BASE}/api/Auth/login`, {
39         method: "POST",
40         headers: { "Content-Type": "application/json" },
41         body: JSON.stringify({
42           email: form.email,
43           password: form.password,
44         }),
45       });
46
47       if (!res.ok) {
48         const text = await res.text();
49         throw new Error(text || "Login failed");
50       }
51
52       const data = await res.json();
53
54       localStorage.setItem("token", data.token);
55       localStorage.setItem("role", data.role);
56       localStorage.setItem("displayName", data.displayName);
57       localStorage.setItem("userId", data.userId);
58       localStorage.setItem("profileComplete", String(data.profileComplete));
59
60       if (!data.profileComplete) {
61         navigate("/profile/setup");
62       } else {
63         navigate(data.role === "Vendor" ? "/vendor" : "/farmer");
64       }
65     } catch (err) {
66       setError(err?.message || "Something went wrong");
67     } finally {
68     }
69   }
70 }
```

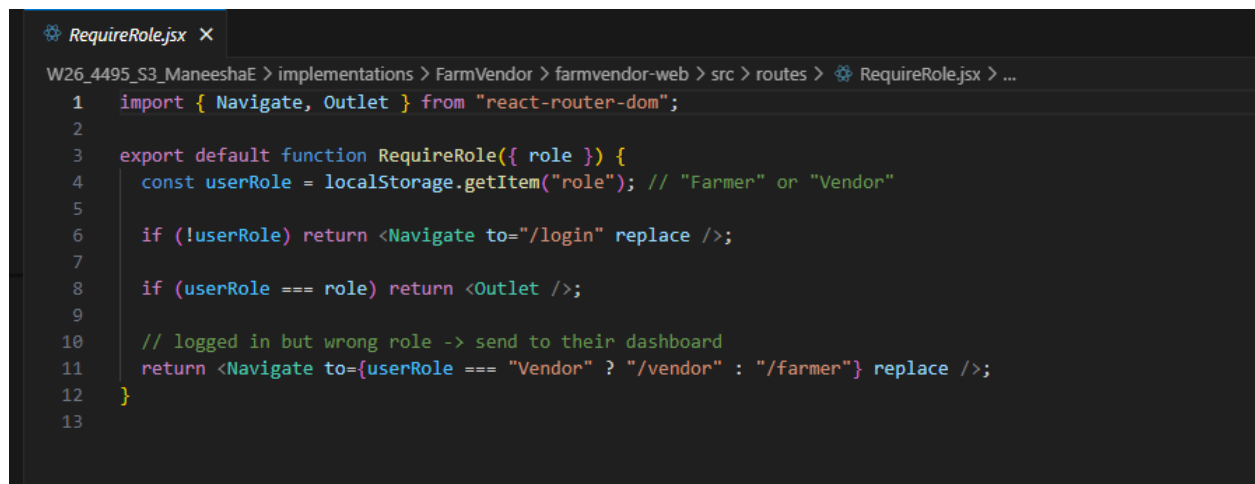
- RequireAuth.jsx



The screenshot shows the VS Code interface. On the left, the Explorer view displays the project structure under 'DEV3'. The 'routes' folder is expanded, showing 'RequireAuth.jsx' and 'RequireRole.jsx'. The main Editor view shows the code for 'RequireAuth.jsx'.

```
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > routes > RequireAuth.jsx > ...  
1  import { Navigate, Outlet, useLocation } from "react-router-dom";  
2  
3  export default function RequireAuth() {  
4      const location = useLocation();  
5      const token = localStorage.getItem("token");  
6  
7      if (!token) {  
8          return <Navigate to="/login" replace state={{ from: location }} />;  
9      }  
10     return <Outlet />;  
11 }  
12
```

- RequireRole.jsx



The screenshot shows the VS Code Editor view with the 'RequireRole.jsx' file open. The breadcrumb at the top indicates the file path: 'W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > routes > RequireRole.jsx > ...'.

```
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > routes > RequireRole.jsx > ...  
1  import { Navigate, Outlet } from "react-router-dom";  
2  
3  export default function RequireRole({ role }) {  
4      const userRole = localStorage.getItem("role"); // "Farmer" or "Vendor"  
5  
6      if (!userRole) return <Navigate to="/login" replace />;  
7  
8      if (userRole === role) return <Outlet />;  
9  
10     // logged in but wrong role -> send to their dashboard  
11     return <Navigate to={userRole === "Vendor" ? "/vendor" : "/farmer"} replace />;  
12 }  
13
```

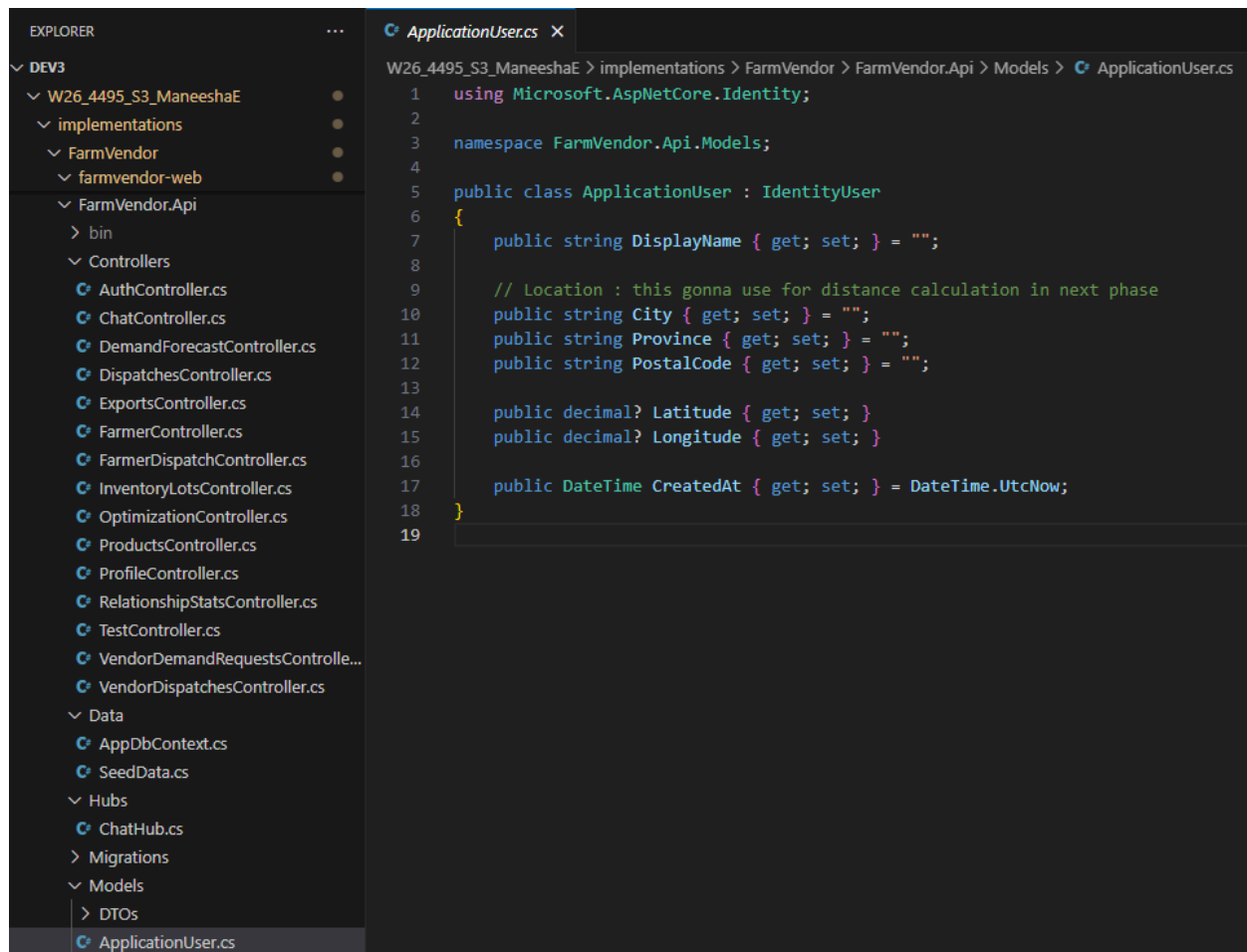
Backend

- AuthController.cs

```
AuthController.cs X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > AuthController.cs
1  using FarmVendor.Api.Models;
2  using Microsoft.AspNetCore.Identity;
3  using Microsoft.AspNetCore.Mvc;
4  using Microsoft.IdentityModel.Tokens;
5  using System.IdentityModel.Tokens.Jwt;
6  using System.Security.Claims;
7  using System.Text;
8
9  namespace FarmVendor.Api.Controllers;
10
11 [ApiController]
12 [Route("api/[controller]")]
13 public class AuthController : ControllerBase
14 {
15     private readonly UserManager<ApplicationUser> _userManager;
16     private readonly IConfiguration _config;
17
18     public AuthController(UserManager<ApplicationUser> userManager, IConfiguration config)
19     {
20         _userManager = userManager;
21         _config = config;
22     }
23
24     public record RegisterDto(string Email, string Password, string DisplayName, string Role);
25     public record LoginDto(string Email, string Password);
26
27     [HttpPost("register")]
28     public async Task<IActionResult> Register(RegisterDto dto)
29     {
30         if (dto.Role != "Farmer" && dto.Role != "Vendor")
31             return BadRequest("Role must be Farmer or Vendor");
32
33         var user = new ApplicationUser
34         {
35             UserName = dto.Email,
36             Email = dto.Email,
37             DisplayName = dto.DisplayName
38         };
39
40         var result = await _userManager.CreateAsync(user, dto.Password);
41         if (!result.Succeeded) return BadRequest(result.Errors);
42
43         await _userManager.AddToRoleAsync(user, dto.Role);
44
45         return Ok(new { message = "Registered successfully" });
46     }
47 }
```



```
AuthController.cs X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > AuthController.cs
14 {
29 {
40     var result = await _userManager.CreateAsync(user, dto.Password);
41     if (!result.Succeeded) return BadRequest(result.Errors);
42
43     await _userManager.AddToRoleAsync(user, dto.Role);
44
45     return Ok(new { message = "Registered successfully" });
46 }
47
48 [HttpPost("login")]
49 public async Task<IActionResult> Login(LoginDto dto)
50 {
51     var user = await _userManager.FindByEmailAsync(dto.Email);
52     if (user == null) return Unauthorized("Invalid credentials");
53
54     var valid = await _userManager.CheckPasswordAsync(user, dto.Password);
55     if (!valid) return Unauthorized("Invalid credentials");
56
57     var roles = await _userManager.GetRolesAsync(user);
58
59     var claims = new List<Claim>
60     {
61         new Claim(JwtRegisteredClaimNames.Sub, user.Id),
62         new Claim(JwtRegisteredClaimNames.Email, user.Email ?? ""),
63         new Claim("displayName", user.DisplayName)
64     };
65
66     foreach (var role in roles)
67         claims.Add(new Claim(ClaimTypes.Role, role));
68
69     var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));
70     var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
71
72     var token = new JwtSecurityToken(
73         issuer: _config["Jwt:Issuer"],
74         audience: _config["Jwt:Audience"],
75         claims: claims,
76         expires: DateTime.UtcNow.AddHours(6),
77         signingCredentials: creds
78     );
79
80     return Ok(new
81     {
82         token = new JwtSecurityTokenHandler().WriteToken(token),
83         role = roles.FirstOrDefault(),
84         displayName = user.DisplayName,
85         userId = user.Id,
86         city = user.City,
87         province = user.Province,
88         postalCode = user.PostalCode,
89         latitude = user.Latitude,
90         longitude = user.Longitude,
91         profileComplete =
92             !string.IsNullOrWhiteSpace(user.DisplayName) &&
93             !string.IsNullOrWhiteSpace(user.City) &&
94             !string.IsNullOrWhiteSpace(user.Province) &&
95             !string.IsNullOrWhiteSpace(user.PostalCode) &&
96             user.Latitude.HasValue &&
97             user.Longitude.HasValue
98     });
99 }
100 }
101
```



- Program.cs

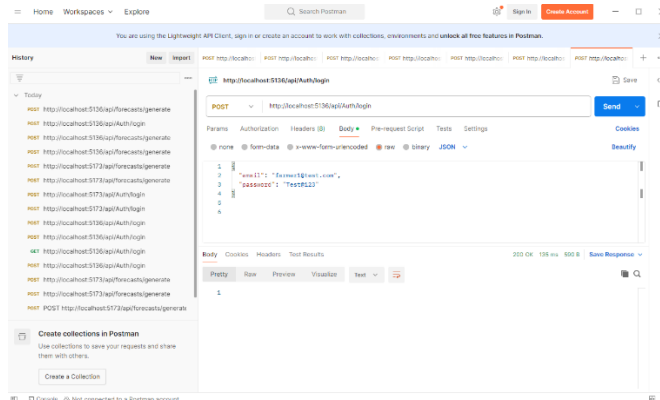
```
Program.cs X
V26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Program.cs

1  using FarmVendor.Api.Data;
2  using FarmVendor.Api.Models;
3  using Microsoft.AspNetCore.Authentication.JwtBearer;
4  using Microsoft.AspNetCore.Identity;
5  using Microsoft.EntityFrameworkCore;
6  using Microsoft.IdentityModel.Tokens;
7  using FarmVendor.Api.Services;
8  using System.Text;
9  using FarmVendor.Api.Hubs;
10
11  var builder = WebApplication.CreateBuilder(args);
12
13  builder.Services.AddControllers();
14
15  // Swagger + JWT Authorize button
16  builder.Services.AddEndpointsApiExplorer();
17  builder.Services.AddSwaggerGen();
18
19  // DB
20  builder.Services.AddDbContext<AppDbContext>(options =>
21  |   options.UseSqlServer(builder.Configuration.GetConnectionString("Default")));
22
23  // Identity
24  builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
25  |  {
26  |      options.Password.RequiredLength = 6;
27  |      options.User.RequireUniqueEmail = true;
28  |  })
29  |  .AddEntityFrameworkStores<AppDbContext>()
30  |  .AddDefaultTokenProviders();
31
32  // JWT
33  var jwtKey = builder.Configuration["Jwt:Key"]!;
34  var jwtIssuer = builder.Configuration["Jwt:Issuer"]!;
35  var jwtAudience = builder.Configuration["Jwt:Audience"]!;
36
37  builder.Services.AddAuthentication(options =>
38  |  {
39  |      options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
40  |      options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
41  |  })
42  |  .AddJwtBearer(options =>
43  |  |  {
44  |  |      options.TokenValidationParameters = new TokenValidationParameters
45  |  |      |  {
46  |  |  |      ValidateIssuer = true,
47  |  |  |      ValidateAudience = true,
48  |  |  |      ValidateLifetime = true,
49  |  |  |      ValidateIssuerSigningKey = true,
50  |  |  |      ValidIssuer = jwtIssuer,
51  |  |  |      ValidAudience = jwtAudience,
52  |  |  |      IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey))
53  |  |  |  };
54  |  |  });
55
56  builder.Services.AddAuthorization();
```

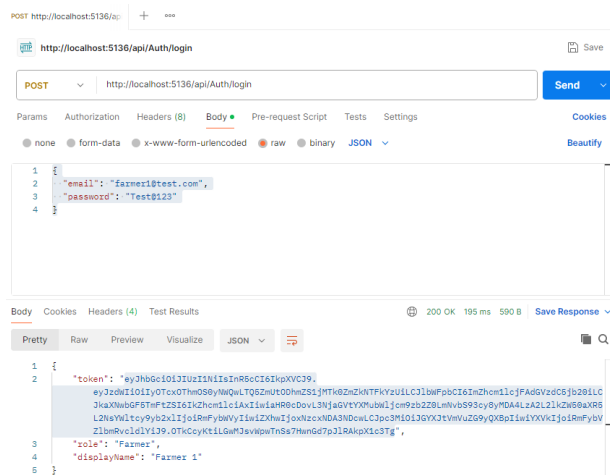
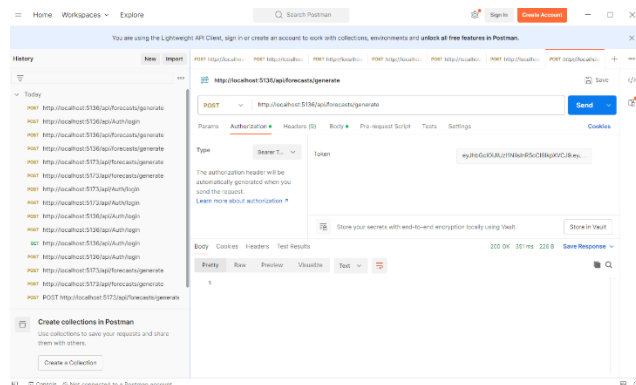
```
Program.cs X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Program.cs
38 {
40     options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
41 }
42 .AddJwtBearer(options =>
43 {
44     options.TokenValidationParameters = new TokenValidationParameters
45     {
46         ValidateIssuer = true,
47         ValidateAudience = true,
48         ValidateLifetime = true,
49         ValidateIssuerSigningKey = true,
50         ValidIssuer = jwtIssuer,
51         ValidAudience = jwtAudience,
52         IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey))
53     };
54 });
55
56 builder.Services.AddAuthorization();
57
58 // CORS (React + SignalR)
59 builder.Services.AddCors(options =>
60 {
61     options.AddPolicy("ReactApp", policy =>
62     {
63         policy.WithOrigins("http://localhost:5173")
64             .AllowAnyHeader()
65             .AllowAnyMethod()
66             .AllowCredentials();
67     });
68
69 // register services
70 builder.Services.AddScoped<DemandForecastService>();
71 builder.Services.AddScoped<DispatchOptimizationService>();
72 builder.Services.AddScoped<RelationshipScoreService>();
73 builder.Services.AddScoped<ChatService>();
74 builder.Services.AddSignalR();
75 builder.Services.AddScoped<FarmerRecommendationService>();
76
77 var app = builder.Build();
78
79 app.UseCors("ReactApp");
80
81 if (app.Environment.IsDevelopment())
82 {
83     app.UseSwagger();
84     app.UseSwaggerUI();
85 }
86
87 // app.UseHttpsRedirection();
88
89 app.UseAuthentication();
90 app.UseAuthorization();
91
92 app.MapControllers();
93 app.MapHub<ChatHub>("/hubs/chat");
94
95 // seed
96 await SeedData.InitializeAsync(app.Services);
97
98 app.Run();
```

- Postman: JWT token generation logic using issuer, audience, and signing key

POST : http://localhost:5136/api/Auth/login



Bearer token



Database

- AspNetUsers Table

WOL4803_S3_Manual > Reports\sqlDocuments > sql_queries > userQuery.sql

```
-- USE [FarmVendorDB];
-- GO
SELECT Id,Email, Username
FROM AspNetUsers;
SELECT
FROM AspNetUsers;
SELECT COLUMN_NAME
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'AspNetUsers';
```

Results (2) Messages

Id	Display Name	Username	Normalized Username	Email	Normalized Email	Email Confirmed	Password Hash	Security Stamp	Concurrency Stamp
1	0913075e-b644-465f-b4db-77993c5dbb72	Vendor 13	vendor13@test.com	vendor13@test.com	VENDOR13@TEST.COM	0	AQAAAAAAtagAAAAE8Dv85BKO/p3agZad3vskM9hu5Gba...	CYAR5OPVNN5Y1N8NWWA2QALMICKG	28045146-86f7-4647-d8c1-d09
2	09463d1c-b786-4258-b52f-d3af6e096c0	Vendor 6	vendor6@test.com	vendor6@test.com	VENDOR6@TEST.COM	0	AQAAAAAAtagAAAAEPUGYbRRCM7ypn62liq+c2B2JO7e...	6VTRTGT3DWU54FMBUSJGALCC753ANL	ed5fbcad-6eaa-404f-962e-d04b
3	19dbda90-a25b-4643-b495-3948b4279b63	Farmer 9	farmer9@test.com	farmer9@test.com	FARMER9@TEST.COM	0	AQAAAAAAtagAAAAEPDor7JInJPQQL4wTx+AwMPHdbt8Vp...	LVGHFXGBH4K85PEZ2JTKORRCEQ5ORZ	6cc79d5b-b415-4993-9209-fca0
4	2b0d536e-b91e-436a-b496-9fc19946f09	Vendor 14	vendor14@test.com	vendor14@test.com	VENDOR14@TEST.COM	0	AQAAAAAAtagAAAAE8v7xb2Ge43f1Hb6AHak3NlyMgp+Y...	BM6QM458ZBRDSV75XM6YHWGFSH7K3	97158ad2-4409-43b2-43ba-058
5	3b49b493-43ea-4001-47ee-af49ffead9ff	Farmer 2	farmer2@test.com	farmer2@test.com	FARMER2@TEST.COM	0	AQAAAAAAtagAAAAK6g4N4Bf5Unib4/TnAt77X0attwG2qg...	ILW3UN2XCDG7BP5GYG3DZHR2SCQW	7ae53ed3-513b-4ee8-883a-10e
6	4ee27b1c-7529-4a2c-89e8-8a6d4a29f9ee	Vendor 2	vendor2@test.com	vendor2@test.com	VENDOR2@TEST.COM	0	AQAAAAAAtagAAAAJ1JALcwWfBBR2057H5Hf4CZmmsL...	FMJZG3PUBA233H0DGHFNK3DCTD6XJ	95371a65-427e-4099-508f-4ee
7	58ba99c-30f4-4831-8a8f-5d4f43dbb1b	Vendor 8	vendor8@test.com	vendor8@test.com	VENDOR8@TEST.COM	0	AQAAAAAAtagAAAAAPYBRLX3JPFNPW963d6m5d6eaR2c...	Q2M4B8B8NHN4H76GPGDFG0TAWJABVEZ	5453bae5-9a5b-414a-908a-d64
8	59eb885-523a-4103-9777-8a53d84f1168	Vendor 4	vendor4@test.com	vendor4@test.com	VENDOR4@TEST.COM	0	AQAAAAAAtagAAAAK03tag58Bf8vty7H1bdu2yAtfawWZL...	TKTC7DKDC4V62OK6G8HMHMHS9B8C	a71ebbaa-af48-486a-8809-a29
9	609c5ac-4114-4714-9391-4540854ca3a0	Vendor 15	vendor15@test.com	vendor15@test.com	VENDOR15@TEST.COM	0	AQAAAAAAtagAAAAE8hyf8p4ndQ3d3B168f+Y5Q2cwvQ...	Q628B8Q2KTW07T1U1XKJ2B80364RYGGS	c3d34a4b-4a8b-408f-b3d8-962b
10	7d141797-613b-40f7-bc05-58675e42d25c	Farmer 6	farmer6@test.com	farmer6@test.com	FARMER6@TEST.COM	0	AQAAAAAAtagAAAAE8v4S3PwYmG6MP77y4HQ7n7G2J8L...	UNZTSQZSQH8K27AT8Q2Y23WYVCN	536a8f18-405b-4822-aad7-bacc
11	7ca81195-44ba-4a5a-a064-b2894335c556	Vendor 10	vendor10@test.com	vendor10@test.com	VENDOR10@TEST.COM	0	AQAAAAAAtagAAAAE8v+mR8Ea8A1cunYc7WU67EKOIMfSdb5...	Z3RLUWKKY3LXKSYDUR04RLCY3K78P	c28a4b5-13bb-4021-b0ab-8751
12	815031b-cbbd-49f4-4963-43b03e75664	Farmer 7	farmer7@test.com	farmer7@test.com	FARMER7@TEST.COM	0	AQAAAAAAtagAAAAE8vDUD1aEambYKwmCypr+J70FMJL...	VGETA15MWUWSPSLSZGY2UX6WRP4PU	879721c-2b4d-4be4-b85a-e013
13	83a6ae9-7b24-4a7d-a273-58c877bdaa3	Farmer 3	farmer3@test.com	farmer3@test.com	FARMER3@TEST.COM	0	AQAAAAAAtagAAAAE8vJcGBLORNa5ZCzKMGFLUWDoJwy2...	38FAV3OSSSHRYMQMG4KQ58X6WNRT3S	314e5f3d-b2a5-4372-8bc0-14fa
14	84c6262-3a74-46d0-94c-9bade91d1fe	Vendor 7	vendor7@test.com	vendor7@test.com	VENDOR7@TEST.COM	0	AQAAAAAAtagAAAAE8vFawmO8ZgizmG1v5vRhawwQ8B...	HLFQ4AMNQ8BH1O3D7F5EELUXSWZODK	6315027d-01e7-433c-a161-b4d
15	86c778b-055d-4130-a702-c78237b7cde2	Farmer 1	farmer1@test.com	farmer1@test.com	FARMER1@TEST.COM	0	AQAAAAAAtagAAAAEMT5vskW5wgZ5X1XJHgmfcL8nf...	3GSAH7GEUELEYTGVO75ECEU5YOLZ	2fa0435-c6f9-433e-a196-8b0a

COLUMN_NAME

1	Id
2	DisplayName
3	Username
4	NormalizedUsername
5	Email
6	NormalizedEmail
7	EmailConfirmed
8	PasswordHash
9	SecurityStamp
10	ConcurrencyStamp
11	PhoneNumber
12	PhoneNumberConfirmed
13	TwoFactorEnabled
14	LockoutEnd
15	LockoutEnabled
16	AccessFailedCount

Feature 5: Farmer Inventory Management

This feature enables farmers to add, manage, and monitor product inventory lots with quantity and expiry tracking. It supports operational planning by providing clear visibility into available stock.

In the final implementation, this module was significantly enhanced by integrating product image selection using the Unsplash API, improving usability and visual interaction for non-technical users.

This feature includes:

Add product lot form

Inventory table view

Expiry date tracking

Expiry status badge logic

Product image selection using Unsplash API

Visual product identification instead of text-only selection

The design consists of:

Modal-based inventory lot entry

Real-time table updates after submission

Status labels such as *Fresh / Near Expiry / Expired*

Image-based product selection modal

Preview of selected product image before saving

Implementation

Frontend

- FarmerDashboard.jsx
- Add Product Lot modal
- Inventory table UI

- Dynamic status badge rendering
- Image selection modal integrated with Unsplash API
- Image preview component for selected product

Backend

- InventoryLotsController.cs
- ProductsController.cs
- CreateInventoryLotDto.cs
- InventoryLot.cs
- Validation for:
 - Quantity
 - Expiry date
 - Product selection

Database

InventoryLot table storing:

- FarmerId
- ProductId
- QuantityAvailable
- Unit
- ExpiryDate
- CreatedAt

Product table enhanced with image fields

- ImageUrl
- ImageThumbUrl
- ImageSource
- PhotographerName
- PhotographerProfile

App

- FarmerDashboard

Upcoming Vendor Requests			Refresh
Product	Qty	Needed By	
 Apple	60 kg	2026-04-09	
 Tomatoes	48 kg	2026-04-10	
 Potatoes	22 kg	2026-04-10	
 Milk	32 L	2026-04-10	
 Onions	50 kg	2026-04-10	
 Potatoes	10 kg	2026-04-11	
 Tomatoes	24 kg	2026-04-12	
 Milk	56 L	2026-04-12	
 Onions	38 kg	2026-04-13	
 Onions	25 kg	2026-04-13	
Add Product Lot Create Dispatch			Requests loaded: 10

Welcome, Farmer 1

Overview of your stock, requests, and vendor recommendations

Available Products3

Expiring Soon1

Upcoming Requests

Current Stock

Refresh

Upcoming Vendor Requests

Product	Qty	Expiry	Status	Product	Qty	Needed By
 Onions	84 kg	2026-04-11	Near Expiry	 Apple	60 kg	2026-04-09
 Milk	97 L	2026-04-16	Fresh	 Tomatoes	48 kg	2026-04-10
 Eggs	21 dozen	2026-04-23	Fresh	 Potatoes	22 kg	2026-04-10
					32 L	2026-04-10
					50 kg	2026-04-10
					10 kg	2026-04-11
					24 kg	2026-04-12
					56 L	2026-04-12
					38 kg	2026-04-13
					25 kg	2026-04-13

Add Product Lot

Create Dispatch

Requests loaded: 10

Top Vendors for Dispatching

Add Product Lot

Product

-- Select --

+

Close

New Product Name

e.g., Carrots

Category (optional)

e.g., Vegetables

Default Unit

e.g., kg / L / dozen

Find Images

Save Product

Cancel

Quantity

e.g., 50

Unit

kg / L / dozen

Expiry Date (optional)

yyyy-mm-dd

Cancel

Save Lot

Distance (km)

Welcome, Farmer 1

Overview of your stock, requests, and vendor recommendations

Role Farmer

Logout

Available Products

3

Current Stock

Product	Qty	Expiry
Onions	84 kg	2026-04-11
Milk	97 L	2026-04-16
Eggs	21 dozen	2026-04-23

Top Vendors for Dispatching

Vendor	Products	Market Qty
Vendor 15	Onions, Milk	63
Vendor 1	Milk	11.69
Vendor 2	Onions	38.55
Vendor 3	Onions	58.57
Vendor 4	Onions	25.53

Add Product Lot

Product

-- Select --

+

Close

New Product Name

Bell pepper

Category (optional)

e.g., Vegetables

Default unit

e.g., kg / L / dozens

Find Images

Select

Select

Select

Select

Select

Select

Save Product

Cancel

Quantity

e.g., 50

Unit

kg / L / dozen

Expiry Date (optional)

yyyy-mm-dd

Cancel

Save Lot

Upcoming Requests

1

for Requests

Qty	Needed By
80 kg	2026-04-09
40 kg	2026-04-10
20 kg	2026-04-10
32 L	2026-04-10
50 kg	2026-04-10
10 kg	2026-04-11
24 kg	2026-04-12
56 L	2026-04-12
30 kg	2026-04-13
25 kg	2026-04-13

Create Dispatch

Refresh

Address

Installation

Score

Surrey, BC, V2T5A1	50	79.13
Surrey, BC, V2T5A1	50	76.5
Surrey, BC, V2T5A1	50	76.47
Surrey, BC, V2T5A1	50	76.45
Surrey, BC, V2T5A1	50	76.42

Add Product Lot

!

Please check this form

×

A product with this name already exists.

Product

-- Select --

+ Close

New Product Name

Bell pepper

Category (optional)

Vegetable

Default Unit

Kg

Find Images

Selected image

Photo by Rens D on Unsplash



Select



Select



Select



Select



Select



Select



Select



Select

Save Product

Cancel

Quantity

e.g., 50

Unit

kg / L / dozen

Expiry Date (optional)

yyyy-mm-dd

Calendar icon

Cancel

Save Lot

Add Product Lot

Product

-- Select --

+ Close

New Product Name

Green Onion

Category (optional)

Vegetable

Default Unit

Kg

Find Images

Selected image

Photo by Christopher Previte on Unsplash

Select

Select

Select

Select

Select

Select

Select

Select

Save Product

Cancel

Quantity

e.g., 50

Unit

kg / L / dozen

Expiry Date (optional)

yyyy-mm-dd

Cancel

Save Lot

Page | 96

Add Product Lot



Green Onion

Kg • Vegetable

+ New Product

Quantity

69

Unit

Kg

Expiry Date (optional)

2026-04-16

Cancel

Save Lot

Add Product Lot

Requests loaded: 10

Current Stock				Refresh
Product	Qty	Expiry	Status	
 Onions	84 kg	2026-04-11	Near Expiry	
 Milk	97 L	2026-04-16	Fresh	
 Green Onion	69 Kg	2026-04-16	Fresh	
 Eggs	21 dozen	2026-04-23	Fresh	

- Product interface

The screenshot displays the 'Products' section of the FarmVendor application. The interface is divided into a sidebar, a main content area, and a right-hand panel.

Sidebar: Contains navigation links: Dashboard, Products (active), Requests, Dispatch, Forecast, Chat, Profile, and Logout.

Main Content Area:

- Products Header:** 'Products' with a subtitle 'Manage your inventory, lists and view product catalog'. It includes a 'Refunds' button and a user status 'Welcome, Farmer 1' with a 'Logout' button.
- Active Product Catalog:** A table listing various products with columns for Name, Default Unit, and Status (Action).
- My Inventory List:** A table showing inventory items with columns for Product, Qty, Expiry, and Actions (Edit, Delete). It includes a 'Refunds' button and an 'Add Item' button.

Name	Default Unit	Status
Apple	kg	Action
Banana	Bunch	Action
Bell Pepper	kg	Action
Broccoli	kg	Action
Carrot	kg	Action
Cauliflower	kg	Action
Cherry	kg	Action
Clementine	Bunch	Action
Cucumber	kg	Action
Egg	dozen	Action
Garlic	kg	Action
Grape	kg	Action
Green Onion	kg	Action
Milk	L	Action
Mushroom	kg	Action
Onion	kg	Action
Orange	kg	Action
Parsley	Bunch	Action
Pine	kg	Action
Potatoes	kg	Action
Sprouts	kg	Action
Strawberry	kg	Action
Tomatoes	kg	Action

Product	Qty	Expiry	Actions
Green Onion	45 kg	2020-04-16	Edit Delete
Onions	54 kg	2020-04-11	Edit Delete
Milk	31 L	2020-04-16	Edit Delete
Eggs	21 dozen	2020-04-23	Edit Delete

Frontend

- FarmerDashboard.jsx

```
FarmerDashboard.jsx
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > FarmerDashboard

11  export default function FarmerDashboard() {
746    requests.map((row, idx) => {
827      <div>
828        <span className="request-hover-label">Address:</span>{ " " }
829        [
830          hoveredRequest.vendorCity,
831          hoveredRequest.vendorProvince,
832          hoveredRequest.vendorPostalCode,
833        ]
834        .filter(Boolean)
835        .join(", ") || "N/A"
836      </div>
837
838      <div>
839        <span className="request-hover-label">Needed By:</span>{ " " }
840        {new Date(hoveredRequest.neededBy).toISOString().slice(0, 10)}
841      </div>
842    </div>
843  </div>
844 </div>
845 </td>
846 </tr>
847 }}
848 </React.Fragment>
849 ))
850 })
851 </tbody>
852 </table>
853
854 <div style={{ marginTop: 12, display: "flex", gap: 10, flexWrap: "wrap" }}>
855   <button className="btn btn-primary" onClick={openAddLot}>
856     Add Product Lot
857   </button>
858   <button className="btn btn-secondary" onClick={openCreateDispatch}>
859     Create Dispatch
860   </button>
861 </div>
862
863 <div style={{ marginTop: 8, fontSize: 12, color: "#6b7280" }}>
864   Requests loaded: <b>{requests.length}</b>
865 </div>
866 </div>
867 </div>
868 </section>
869
870 <section style={{ marginTop: 14 }}>
871   <div className="card">
872     <div className="card-header">
873       <h2>Top Vendors for Dispatching</h2>
874       <button
875         className="btn btn-secondary"
876         onClick={loadRecommendedVendors}
877         disabled={loadingRecommendations}
878       >
879         {loadingRecommendations ? "Loading..." : "Refresh"}
880       </button>
881     </div>
882
883     <div className="card-body">
884       <table className="table">
885         <thead>
886           <tr>
887             <th>Vendor</th>
```

```

FarmerDashboard.jsx M X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx >
11 export default function FarmerDashboard() {
12     //add product
13     const openAddLot = async () => {
14         setPageError("");
15         setLotError("");
16         setShowAddLot(true);
17         setShowNewProductFields(false);
18
19         setNewProductForm({
20             name: "",
21             category: "",
22             defaultUnit: "",
23         });
24
25         setLotForm({
26             productId: "",
27             quantityAvailable: "",
28             unit: "",
29             expiryDate: "",
30         });
31
32         setImageSearchResults([]);
33         setSelectedImage(null);
34
35         if (products.length === 0) {
36             try {
37                 await loadProducts();
38             } catch (e) {
39                 setLotError(e?.message || "Failed to load products for lot form");
40             }
41         }
42     };
43
44     const submitNewProduct = async () => {
45         setLotError("");
46
47         if (!newProductForm.name.trim()) {
48             return setLotError("Product name is required.");
49         }
50
51         if (!newProductForm.defaultUnit.trim()) {
52             return setLotError("Default unit is required.");
53         }
54
55         setSavingProduct(true);
56
57         try {
58             const res = await fetch(`${API_BASE}/api/products`, {
59                 method: "POST",
60                 headers: authHeaders,
61                 body: JSON.stringify({
62                     name: newProductForm.name.trim(),
63                     category: newProductForm.category.trim()
64                     ? newProductForm.category.trim()
65                     : null,
66                     defaultUnit: newProductForm.defaultUnit.trim(),
67                     imageUrl: selectedImage?.imageUrl || null,
68                     imageThumbUrl: selectedImage?.thumbnailUrl || null,
69                     imageSource: selectedImage?.source || null,
70                     photographerName: selectedImage?.photographerName || null,
71                     photographerProfile: selectedImage?.photographerProfile || null,
72                 }),
73             });
74         }
75     };
76 }

```



```
FarmerDashboard.jsx M X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashbo
11 export default function FarmerDashboard() {
289
290   const submitNewProduct = async () => {
291     setLotError("");
292
293     if (!newProductForm.name.trim()) {
294       return setLotError("Product name is required.");
295     }
296
297     if (!newProductForm.defaultUnit.trim()) {
298       return setLotError("Default unit is required.");
299     }
300
301     setSavingProduct(true);
302
303     try {
304       const res = await fetch(`${API_BASE}/api/products`, {
305         method: "POST",
306         headers: authHeaders,
307         body: JSON.stringify({
308           name: newProductForm.name.trim(),
309           category: newProductForm.category.trim()
310             ? newProductForm.category.trim()
311             : null,
312           defaultUnit: newProductForm.defaultUnit.trim(),
313           imageUrl: selectedImage?.imageUrl || null,
314           imageThumbUrl: selectedImage?.thumbnailUrl || null,
315           imageSource: selectedImage?.source || null,
316           photographerName: selectedImage?.photographerName || null,
317           photographerProfile: selectedImage?.photographerProfile || null,
318         }),
319     });
320
321     const text = await res.text();
322     if (!res.ok) throw new Error(text || "Failed to create product.");
323
324     const createdProduct = JSON.parse(text);
325
326     await loadProducts();
327
328     setLotForm((prev) => ({
329       ...prev,
330       productId: String(createdProduct.productId),
331       unit: createdProduct.defaultUnit || prev.unit,
332     }));
333
334     setNewProductForm({
335       name: "",
336       category: "",
337       defaultUnit: "",
338     });
339
340     setImageSearchResults([]);
341     setSelectedImage(null);
342     setShowNewProductFields(false);
343     setLotError("");
344   } catch (err) {
345     setLotError(err?.message || "Failed to create product.");
346   } finally {
347     setSavingProduct(false);
348   }
349 };
350
```

```

FarmerDashboard.jsx M X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx
11  export default function FarmerDashboard() {
290  const submitNewProduct = async () => {
349  };
350
351  const submitLot = async (e) => {
352    e.preventDefault();
353    setLotError("");
354
355    if (!lotForm.productId) return setLotError("Please select a product.");
356    if (!lotForm.quantityAvailable || Number(lotForm.quantityAvailable) <= 0) {
357      return setLotError("Quantity must be greater than 0.");
358    }
359
360    setSavingLot(true);
361    try {
362      const payload = {
363        productId: Number(lotForm.productId),
364        quantityAvailable: Number(lotForm.quantityAvailable),
365        unit: lotForm.unit?.trim() ? lotForm.unit.trim() : null,
366        expiryDate: lotForm.expiryDate
367          ? new Date(lotForm.expiryDate).toISOString()
368          : null,
369      };
370
371      const res = await fetch(`${API_BASE}/api/InventoryLots`, {
372        method: "POST",
373        headers: authHeaders,
374        body: JSON.stringify(payload),
375      });
376
377      const text = await res.text();
378      if (!res.ok) throw new Error(text || "Failed to create inventory lot.");
379
380      setShowAddLot(false);
381      setLotError("");
382      setLotForm({
383        productId: "",
384        quantityAvailable: "",
385        unit: "",
386        expiryDate: "",
387      });
388
389      await loadDashboard();
390      await loadRecommendedVendors();
391    } catch (err) {
392      setLotError(err?.message || "Failed to save inventory lot");
393    } finally {
394      setSavingLot(false);
395    }
396  };
397
398  const openCreateDispatch = async () => {
399    setPageError("");
400    setDispatchError("");
401
402    try {
403      if (requests.length === 0) {
404        await loadDashboard();
405      }
406    } catch (e) {
407      setDispatchError(e?.message || "Failed to load requests");
408    }
409  }

```

```

FarmerDashboard.jsx M X
V26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > Fa
11 export default function FarmerDashboard() {
928
929     {showAddLot && (
930         <div className="modal-backdrop" onClick={() => setShowAddLot(false)}>
931             <div className="modal" onClick={(e) => e.stopPropagation()}>
932                 <div className="modal-header">
933                     <h3>Add Product Lot</h3>
934                     <button className="icon-btn" onClick={() => setShowAddLot(false)}>
935                         X
936                     </button>
937                 </div>
938
939                 <form onSubmit={submitLot} className="modal-body">
940                     {lotError && (
941                         <div className="modal-notice modal-notice-error" role="alert">
942                             <div className="modal-notice-icon">!</div>
943                             <div className="modal-notice-content">
944                                 <div className="modal-notice-title">Please check this form</div>
945                                 <div className="modal-notice-text">{lotError}</div>
946                             </div>
947                             <button
948                                 type="button"
949                                 className="modal-notice-close"
950                                 onClick={() => setLotError("")}
951                                 aria-label="Dismiss error"
952                             >
953                                 ✖
954                             </button>
955                         </div>
956                     )}
957
958                     <div className="product-picker-wrap">
959                         <label className="field">
960                             <span>Product</span>
961
962                             <Select
963                                 className="product-image-select"
964                                 classNamePrefix="product-image-select"
965                                 options={productOptions}
966                                 value={selectedProductOption}
967                                 onChange={(selected) => {
968                                     setLotForm((prev) => ({
969                                         ...prev,
970                                         productId: selected ? String(selected.value) : "",
971                                         unit: selected?.defaultUnit ?? "",
972                                     }));
973
974                                     if (lotError) setLotError("");
975                                 })}
976                                 placeholder="-- Select --"
977                                 isClearable
978                                 menuPortalTarget={document.body}
979                                 styles={selectStyles}
980                                 formatOptionLabel={renderProductOptionLabel}
981                                 noOptionsMessage={() => "No products found"}
982                             />
983                         </label>
984
985                         <button
986                             type="button"
987                             className="fab-add-product"
988                             onClick={() => {
989                                 setShowNewProductFields((prev) => !prev);

```

```

FarmerDashboard.jsx M X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx
11  export default function FarmerDashboard() {
983  </label>
984
985  <button
986    type="button"
987    className="fab-add-product"
988    onClick={() => {
989      setShowNewProductFields((prev) => !prev);
990      setLotError("");
991    }}
992    title="Add a new product"
993  >
994    <span className="fab-plus">+</span>
995    <span>{showNewProductFields ? "Close" : "New Product"}</span>
996  </button>
997 </div>
998
999 <div className="new-product-section">
1000   {showNewProductFields && (
1001     <div className="new-product-card">
1002       <label className="field">
1003         <span>New Product Name</span>
1004         <input
1005           value={newProductForm.name}
1006           onChange={(e) =>
1007             setNewProductForm((prev) => ({
1008               ...prev,
1009               name: e.target.value,
1010             })))
1011           placeholder="e.g., Carrots"
1012         />
1013       </label>
1014
1015       <label className="field">
1016         <span>Category (optional)</span>
1017         <input
1018           value={newProductForm.category}
1019           onChange={(e) =>
1020             setNewProductForm((prev) => ({
1021               ...prev,
1022               category: e.target.value,
1023             })))
1024           placeholder="e.g., Vegetables"
1025         />
1026       </label>
1027
1028       <label className="field">
1029         <span>Default Unit</span>
1030         <input
1031           value={newProductForm.defaultUnit}
1032           onChange={(e) =>
1033             setNewProductForm((prev) => ({
1034               ...prev,
1035               defaultUnit: e.target.value,
1036             })))
1037           placeholder="e.g., kg / L / dozen"
1038         />
1039       </label>
1040
1041       <div className="image-search-toolbar">
1042         <button

```

```

FarmerDashboard.jsx M X
W26.4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > FarmerDashboard.jsx
11 export default function FarmerDashboard() {
1044     <div className="image-search-toolbar">
1045         <button
1046             type="button"
1047             className="btn btn-secondary"
1048             onClick={searchProductImages}
1049             disabled={searchingImages}
1050         >
1051             {searchingImages ? "Searching..." : "Find Images"}
1052         </button>
1053     </div>
1054
1055     {selectedImage && (
1056         <div className="selected-image-preview">
1057             <img
1058                 src={selectedImage.thumbnailUrl || selectedImage.imageUrl}
1059                 alt={selectedImage.alt || newProductForm.name}
1060             />
1061             <div>
1062                 <div className="selected-image-title">Selected image</div>
1063                 <div className="selected-image-meta">
1064                     {selectedImage.photographerName
1065                         ? `Photo by ${selectedImage.photographerName} on Unsplash`
1066                         : "Unsplash image selected"}
1067                 </div>
1068             </div>
1069         </div>
1070     )}
1071
1072     {imageSearchResults.length > 0 && (
1073         <div className="image-picker-grid">
1074             {imageSearchResults.map((img, idx) => (
1075                 <button
1076                     key={` ${img.imageUrl}-${idx}`}
1077                     type="button"
1078                     className={`image-card ${
1079                         selectedImage?.imageUrl === img.imageUrl ? "selected" : ""
1080                     }`}
1081                     onClick={() => setSelectedImage(img)}
1082                 >
1083                     <img
1084                         src={img.thumbnailUrl || img.imageUrl}
1085                         alt={img.alt || "product"}
1086                     />
1087                     <span>Select</span>
1088                 </button>
1089             ))}
1090         </div>
1091     )}
1092
1093     <div className="new-product-actions">
1094         <button
1095             type="button"
1096             className="btn btn-primary"
1097             onClick={submitNewProduct}
1098             disabled={savingProduct}
1099         >
1100             {savingProduct ? "Saving..." : "Save Product"}
1101         </button>
1102
1103         <button
1104             type="button"
1105             className="btn btn-secondary"

```

```

11  export default function FarmerDashboard() {
1091  }
1092
1093      <div className="new-product-actions">
1094          <button
1095              type="button"
1096              className="btn btn-primary"
1097              onClick={submitNewProduct}
1098              disabled={savingProduct}
1099          >
1100              {savingProduct ? "Saving..." : "Save Product"}
1101          </button>
1102
1103          <button
1104              type="button"
1105              className="btn btn-secondary"
1106              onClick={() => {
1107                  setShowNewProductFields(false);
1108                  setLotError("");
1109                  setImageSearchResults([]);
1110                  setSelectedImage(null);
1111              }}
1112          >
1113              Cancel
1114          </button>
1115      </div>
1116  </div>
1117  )}
1118 </div>
1119
1120  <div className="row">
1121      <label className="field">
1122          <span>Quantity</span>
1123          <input
1124              type="number"
1125              step="0.01"
1126              value={lotForm.quantityAvailable}
1127              onChange={(e) => {
1128                  setLotForm((prev) => ({ ...prev, quantityAvailable: e.target.value }));
1129              }}
1130              placeholder="e.g., 50"
1131          />
1132      </label>
1133
1134      <label className="field">
1135          <span>Unit</span>
1136          <input
1137              value={lotForm.unit}
1138              onChange={(e) => {
1139                  setLotForm((prev) => ({ ...prev, unit: e.target.value }));
1140              }}
1141              placeholder="kg / L / dozen"
1142          />
1143      </label>
1144  </div>
1145
1146  <label className="field">
1147      <span>Expiry Date (optional)</span>
1148      <input
1149          type="date"
1150          value={lotForm.expiryDate}
1151          onChange={(e) => {
1152              setLotForm((prev) => ({ ...prev, expiryDate: e.target.value }));

```

```
FarmerDashboard.jsx M X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDashboard.jsx > FarmerDashboard
11  export default function FarmerDashboard() {
1118  //
1119  </div>
1120  <div className="row">
1121    <label className="field">
1122      <span>Quantity</span>
1123      <input
1124        type="number"
1125        step="0.01"
1126        value={lotForm.quantityAvailable}
1127        onChange={(e) =>
1128          setLotForm((prev) => ({ ...prev, quantityAvailable: e.target.value }))
1129        }
1130        placeholder="e.g., 50"
1131      />
1132    </label>
1133
1134    <label className="field">
1135      <span>Unit</span>
1136      <input
1137        value={lotForm.unit}
1138        onChange={(e) =>
1139          setLotForm((prev) => ({ ...prev, unit: e.target.value }))
1140        }
1141        placeholder="kg / L / dozen"
1142      />
1143    </label>
1144  </div>
1145
1146  <label className="field">
1147    <span>Expiry Date (optional)</span>
1148    <input
1149      type="date"
1150      value={lotForm.expiryDate}
1151      onChange={(e) =>
1152        setLotForm((prev) => ({ ...prev, expiryDate: e.target.value }))
1153      }
1154    />
1155  </label>
1156
1157  <div className="modal-actions">
1158    <button
1159      type="button"
1160      className="btn btn-secondary"
1161      onClick={() => {
1162        setShowAddLot(false);
1163        setLotError("");
1164        setImageSearchResults([]);
1165        setSelectedImage(null);
1166      }}
1167    >
1168    Cancel
1169  </button>
1170  <button type="submit" className="btn btn-primary" disabled={savingLot}>
1171    {savingLot ? "Saving..." : "Save Lot"}
1172  </button>
1173  </div>
1174  </form>
1175  </div>
1176  </div>
1177  )}
1178
1179  {showCreateDispatch && (
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

- FarmerProduct.jsx

```

FarmerDashboard.jsx M  FarmerProducts.jsx X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerProducts.jsx > ...
1  import React, { useEffect, useMemo, useState } from "react";
2  import { useNavigate } from "react-router-dom";
3  import "../styles/dashboard.css";
4  import FarmerSidebar from "../assets/components/FarmerSidebar";
5
6  const API_BASE = import.meta.env.VITE_API_URL; // e.g., http://localhost:5136
7
8  export default function FarmerProducts() {
9    const navigate = useNavigate();
10    const token = localStorage.getItem("token");
11    const displayName = localStorage.getItem("displayName") || "Farmer";
12
13    const authHeaders = useMemo(() => {
14      return {
15        "Content-Type": "application/json",
16        Authorization: `Bearer ${token}`,
17      };
18    }, [token]);
19
20    const [loading, setLoading] = useState(true);
21    const [error, setError] = useState("");
22
23    // Catalog
24    const [products, setProducts] = useState([]);
25
26    // My lots
27    const [lots, setLots] = useState([]);
28
29    // Add Lot modal
30    const [showAddLot, setShowAddLot] = useState(false);
31    const [lotForm, setLotForm] = useState({
32      productId: "",
33      quantityAvailable: "",
34      unit: "",
35      expiryDate: "",
36    });
37    const [savingLot, setSavingLot] = useState(false);
38
39    // Edit Lot modal
40    const [showEditLot, setShowEditLot] = useState(false);
41    const [editLot, setEditLot] = useState(null);
42    const [savingEdit, setSavingEdit] = useState(false);
43
44    const logout = () => {
45      localStorage.clear();
46      navigate("/login");
47    };
48
49    const loadCatalog = async () => {
50      const res = await fetch(`${API_BASE}/api/products/active`, { headers: authHeaders });
51      const text = await res.text();
52      if (!res.ok) throw new Error(text || "Failed to load products");
53      setProducts(JSON.parse(text));
54    };
55
56    const loadMyLots = async () => {
57      const res = await fetch(`${API_BASE}/api/farmer/inventorylots`, { headers: authHeaders });
58      const text = await res.text();
59      if (!res.ok) throw new Error(text || "Failed to load inventory lots");
60      setLots(JSON.parse(text));
61    };
62
63    const loadAll = async () => {

```


Backend

- ProductController.cs

```
FarmerDashboard.jsx  ProductsController.cs x
V26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > ProductsController.cs

1  using System.Net.Http.Headers;
2  using System.Text.Json;
3  using FarmVendor.Api.Data;
4  using FarmVendor.Api.Models;
5  using FarmVendor.Api.Models.DTOs;
6  using Microsoft.AspNetCore.Authorization;
7  using Microsoft.AspNetCore.Mvc;
8  using Microsoft.EntityFrameworkCore;
9
10 namespace FarmVendor.Api.Controllers
11 {
12     [ApiController]
13     [Route("api/[controller]")]
14     [Authorize]
15     public class ProductsController : ControllerBase
16     {
17         private readonly AppDbContext _db;
18         private readonly IConfiguration _config;
19
20         public ProductsController(AppDbContext db, IConfiguration config)
21         {
22             _db = db;
23             _config = config;
24         }
25
26         [HttpGet("active")]
27         public async Task<IActionResult> GetActive()
28         {
29             var items = await _db.Product
30                 .Where(p => p.IsActive)
31                 .OrderBy(p => p.Name)
32                 .Select(p => new
33                 {
34                     p.ProductId,
35                     p.Name,
36                     p.Category,
37                     p.DefaultUnit,
38                     p.ImageUrl,
39                     p.ImageThumbUrl,
40                     p.ImageSource,
41                     p.PhotographerName,
42                     p.PhotographerProfile
43                 })
44                 .ToListAsync();
45
46             return Ok(items);
47         }
48
49         [HttpGet("search-images")]
50         public async Task<IActionResult> SearchImages([FromQuery] string query)
51         {
52             if (string.IsNullOrEmpty(query))
53                 return BadRequest("Query is required.");
54
55             var accessKey = _config["Unsplash:AccessKey"];
56             if (string.IsNullOrEmpty(accessKey))
57                 return StatusCode(500, "Unsplash access key is not configured.");
58
59             using var httpClient = new HttpClient();
60             httpClient.DefaultRequestHeaders.Authorization =
61                 new AuthenticationHeaderValue("Client-ID", accessKey);
62             httpClient.DefaultRequestHeaders.Add("Accept", "json");
63         }
64     }
65 }
```

- CreateProductDTO.cs

```
FarmerDashboard.jsx X CreateProductDto.cs X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models >
1 namespace FarmVendor.Api.Models.DTOs
2 {
3     public class CreateProductDto
4     {
5         public string Name { get; set; } = "";
6         public string? Category { get; set; }
7         public string DefaultUnit { get; set; } = "kg";
8
9         public string? ImageUrl { get; set; }
10        public string? ImageThumbUrl { get; set; }
11        public string? ImageSource { get; set; }
12        public string? PhotographerName { get; set; }
13        public string? PhotographerProfile { get; set; }
14    }
15 }
```

- InventoryLotDTO.cs

```
FarmerDashboard.jsx X InventoryLotRowDto.cs X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Mod
1 namespace FarmVendor.Api.Models.DTOs;
2
3 public class InventoryLotRowDto
4 {
5     public int InventoryLotId { get; set; }
6     public int ProductId { get; set; }
7     public string ProductName { get; set; } = "";
8     public decimal QuantityAvailable { get; set; }
9     public string Unit { get; set; } = "kg";
10    public DateTime? ExpiryDate { get; set; }
11    public DateTime CreatedAt { get; set; }
12 }
```

- UpdateInventoryLotDto.cs

```
1 namespace FarmVendor.Api.Models.DTOs;
2
3 public class UpdateInventoryLotDto
4 {
5     public decimal QuantityAvailable { get; set; }
6     public string? Unit { get; set; }
7     public DateTime? ExpiryDate { get; set; }
8 }
```

- InventoryLot.cs

```
1 namespace FarmVendor.Api.Models;
2
3 public class InventoryLot
4 {
5     public int InventoryLotId { get; set; }
6
7     public string FarmerId { get; set; } = "";
8     public ApplicationUser Farmer { get; set; } = null!;
9
10    public int ProductId { get; set; }
11    public Product Product { get; set; } = null!;
12
13    public decimal QuantityAvailable { get; set; }
14
15    public string Unit { get; set; } = "kg";
16
17    public DateTime? ExpiryDate { get; set; }
18
19    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
20 }
21
```

Database

- Product table

FarmerDashboard.js productLog

```
W06.4495.53_Maneeshaf > ReportsAndDocuments > sql queries > productLog
[localhost@MS2C02008] C:\FarmVendorDb
1 USE FarmVendorDb;
2 select * from Product;
3
4 SELECT COLUMN_NAME
5 FROM INFORMATION_SCHEMA.COLUMNS
6 WHERE TABLE_NAME = 'Product';
7
8
```

Results (2) Messages

ProductId	Name	Category	DefaultUnit	IsActive	ImageSource	ImageThumbUrl	ImageId	PhotographerName	PhotographerProfile
1	Tomatoes	Vegetables	kg	1	Unsplash	https://images.unsplash.com/photo-156730626418-280f6dc...	https://images.unsplash.com/photo-156730626418-280f6dc...	Mekus Spike	https://unsplash.com/@mekusspike
2	Onions	Vegetables	kg	1	Unsplash	https://images.unsplash.com/photo-1664975367131-4c7ac34f...	https://images.unsplash.com/photo-1664975367131-4c7ac34f...	Katrina Holmes	https://unsplash.com/@katrinaholmes
3	Peas	Vegetables	kg	1	Unsplash	https://images.unsplash.com/photo-151897764601-85382a...	https://images.unsplash.com/photo-151897764601-85382a...	Johette	https://unsplash.com/@johette
4	Milk	Dairy	L	1	Unsplash	https://images.unsplash.com/photo-1550551724-826924b9b1...	https://images.unsplash.com/photo-1550551724-826924b9b1...	Elle Asaron	https://unsplash.com/@elleanaron
5	Eggs	Dairy	dozen	1	Unsplash	https://images.unsplash.com/photo-15063765307-8732ab5...	https://images.unsplash.com/photo-15063765307-8732ab5...	Jakub Kaponak	https://unsplash.com/@jakubkaponak
6	Carrot	Vegetable	kg	1	Unsplash	https://images.unsplash.com/photo-144526276818-728615c...	https://images.unsplash.com/photo-144526276818-728615c...	NicNBRAND	https://unsplash.com/@nicnbrand
7	Cucumber	Vegetable	kg	1	Unsplash	https://images.unsplash.com/photo-1604977042946-1eacc30f...	https://images.unsplash.com/photo-1604977042946-1eacc30f...	Mockup Graphics	https://unsplash.com/@mockupgraphics
8	Spinach	Vegetable	kg	1	Unsplash	https://images.unsplash.com/photo-157604057995-58f58ff...	https://images.unsplash.com/photo-157604057995-58f58ff...	Louis Hansel	https://unsplash.com/@louis-hansel
9	Red Pepper	Vegetable	kg	1	Unsplash	https://images.unsplash.com/photo-154366235-5f8f8d6d8...	https://images.unsplash.com/photo-154366235-5f8f8d6d8...	Stephanie Stuber	https://unsplash.com/@stephanie-stuber
10	Broccoli	Vegetable	kg	1	Unsplash	https://images.unsplash.com/photo-1493411621433-762937f...	https://images.unsplash.com/photo-1493411621433-762937f...	Dan Gold	https://unsplash.com/@dangoldphoto
11	Cauliflower	Vegetable	kg	1	Unsplash	https://images.unsplash.com/photo-1510627486534-c7d9002...	https://images.unsplash.com/photo-1510627486534-c7d9002...	Monika Grabkowska	https://unsplash.com/@monika
12	Strawberry	Fruit	kg	1	Unsplash	https://images.unsplash.com/photo-1464963911881-746a0d6...	https://images.unsplash.com/photo-1464963911881-746a0d6...	Lufkin Hunt	https://unsplash.com/@lufkinhunt180
13	Apple	Fruit	kg	1	Unsplash	https://images.unsplash.com/photo-150806087-1e4c0b6cb...	https://images.unsplash.com/photo-150806087-1e4c0b6cb...	Rachel Park	https://unsplash.com/@rachelpark
14	Pear	Fruit	kg	1	Unsplash	https://images.unsplash.com/photo-161540925673-5a278d6...	https://images.unsplash.com/photo-161540925673-5a278d6...	Mekus Spike	https://unsplash.com/@mekusspike
15	Grape	Fruit	kg	1	Unsplash	https://images.unsplash.com/photo-151577876754-39c2d31...	https://images.unsplash.com/photo-151577876754-39c2d31...	Sonya Langford	https://unsplash.com/@sonyalangford
16	Orange	Fruit	kg	1	Unsplash	https://images.unsplash.com/photo-1545314301-43382107b...	https://images.unsplash.com/photo-1545314301-43382107b...	Steno Nascimento	https://unsplash.com/@steno_nascimento
17	Yogurt	Dairy	L	1	Unsplash	https://images.unsplash.com/photo-1571212515415-f8f164...	https://images.unsplash.com/photo-1571212515415-f8f164...	Sara Cervera	https://unsplash.com/@sara-cervera
18	Cheese	Dairy	kg	1	Unsplash	https://images.unsplash.com/photo-1486297678162-8b2a19b...	https://images.unsplash.com/photo-1486297678162-8b2a19b...	Alex Munsell	https://unsplash.com/@alexmunsell

COLUMN_NAME

ProductId
Name
Category
DefaultUnit
IsActive
ImageSource
ImageThumbUrl
ImageId
PhotographerName
PhotographerProfile

- InventoryLot table

FarmerDashboard.js productLog inventoryLotLog

```
W06.4495.53_Maneeshaf > ReportsAndDocuments > sql queries > inventoryLotLog
[localhost@MS2C02008] C:\FarmVendorDb
1 USE FarmVendorDb;
2 select * from InventoryLot;
3
4 SELECT COLUMN_NAME
5 FROM INFORMATION_SCHEMA.COLUMNS
6 WHERE TABLE_NAME = 'InventoryLot';
7
8
```

Results (2) Messages

InventoryLotId	FarmerId	ProductId	QuantityAvailable	Unit	ExpiryDate	CreatedAt
1	8c3c778b-055d-4130-a7f2-c78237b7e9e2	5	21.00	dozen	2026-04-22 18:02:46.6355362	2026-04-07 18:02:46.6355639
2	8c3c778b-055d-4130-a7f2-c78237b7e9e2	4	97.00	L	2026-04-15 18:02:46.6491699	2026-04-07 18:02:46.6493172
3	8c3c778b-055d-4130-a7f2-c78237b7e9e2	2	84.00	kg	2026-04-10 18:02:46.6492419	2026-04-07 18:02:46.6493421
4	384f9e93-43ea-4001-87ee-abd9ffead9ff	5	14.00	dozen	2026-04-15 18:02:46.6492736	2026-04-07 18:02:46.6493237
5	384f9e93-43ea-4001-87ee-abd9ffead9ff	4	52.00	L	2026-04-20 18:02:46.6493013	2026-04-07 18:02:46.6493014
6	384f9e93-43ea-4001-87ee-abd9ffead9ff	2	43.00	kg	2026-04-13 18:02:46.6493260	2026-04-07 18:02:46.6493261
7	83eabee9-7824-4a7d-a273-5f6c8f7b8aa3	3	31.00	kg	2026-04-18 18:02:46.6493594	2026-04-07 18:02:46.6493595
8	83eabee9-7824-4a7d-a273-5f6c8f7b8aa3	5	79.00	dozen	2026-04-26 18:02:46.6493850	2026-04-07 18:02:46.6493850
9	83eabee9-7824-4a7d-a273-5f6c8f7b8aa3	1	86.00	kg	2026-04-14 18:02:46.6494132	2026-04-07 18:02:46.6494133
10	dafad88c-add8-42e3-8514-8a0681a36680	2	53.00	kg	2026-04-15 18:02:46.6494368	2026-04-07 18:02:46.6494368
11	dafad88c-add8-42e3-8514-8a0681a36680	5	50.00	dozen	2026-04-24 18:02:46.6494671	2026-04-07 18:02:46.6494672
12	dafad88c-add8-42e3-8514-8a0681a36680	2	32.00	kg	2026-04-22 18:02:46.6494955	2026-04-07 18:02:46.6494956
13	cab52a33-4ca7-44f0-b35c-b63e24d1b1a1	1	51.00	kg	2026-04-12 18:02:46.6495186	2026-04-07 18:02:46.6495187
14	cab52a33-4ca7-44f0-b35c-b63e24d1b1a1	1	13.00	kg	2026-04-11 18:02:46.6495450	2026-04-07 18:02:46.6495451
15	cab52a33-4ca7-44f0-b35c-b63e24d1b1a1	2	12.00	kg	2026-04-12 18:02:46.6495681	2026-04-07 18:02:46.6495686
16	7d141797-613b-48f7-bcd5-5867f5a42d5c	2	79.00	kg	2026-04-24 18:02:46.6495975	2026-04-07 18:02:46.6495976
17	7d141797-613b-48f7-bcd5-5867f5a42d5c	1	63.00	kg	2026-04-26 18:02:46.6496203	2026-04-07 18:02:46.6496204
18	7d141797-613b-48f7-bcd5-5867f5a42d5c	3	83.00	kg	2026-04-26 18:02:46.6497086	2026-04-07 18:02:46.6497089
19	81f5031b-cbbd-49ff-b963-458c03e75664	5	27.00	dozen	2026-04-16 18:02:46.6497627	2026-04-07 18:02:46.6497629

COLUMN_NAME

InventoryLotId
FarmerId
ProductId
QuantityAvailable
Unit
ExpiryDate
CreatedAt

Feature 6: Vendor Demand Request System

This feature allows vendors to create demand requests for products when stock is low. It supports structured communication of vendor needs and acts as a core input to forecasting and dispatch planning.

This feature includes:

- Low stock alert table
- Demand request creation
- Open / Fulfilled / Cancelled lifecycle tracking
- Product-based request creation

The design consists of:

- Low stock visibility
- Demand request creation form
- Request status lifecycle management

Implementation

Frontend

- VendorDashboard.jsx
- Create demand request button
- Low stock alert UI
- Open request list

Backend

- VendorDemandRequestsController.cs
- DemandRequest.cs
- CreateDemandRequestDto.cs
- Request status handling logic

Database

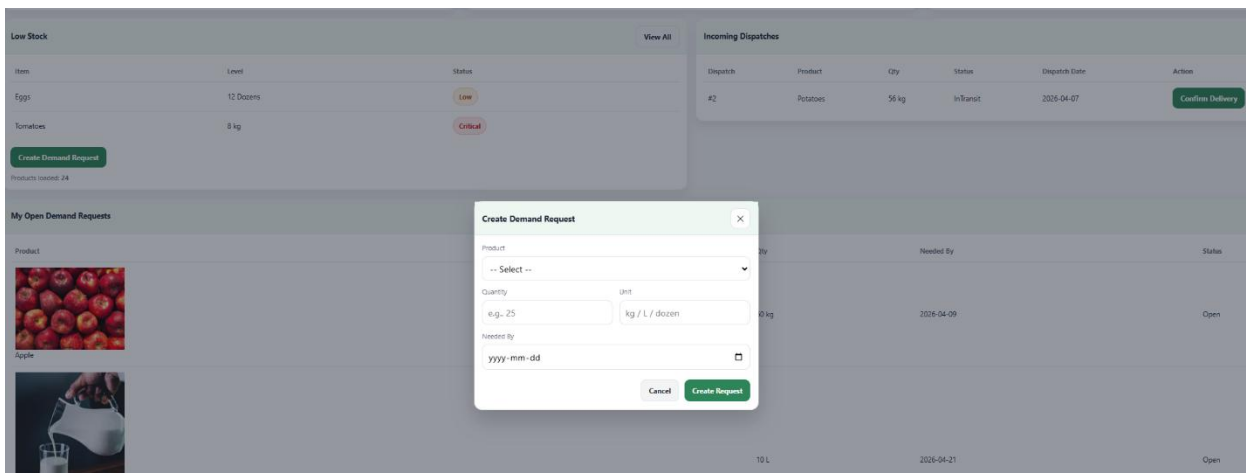
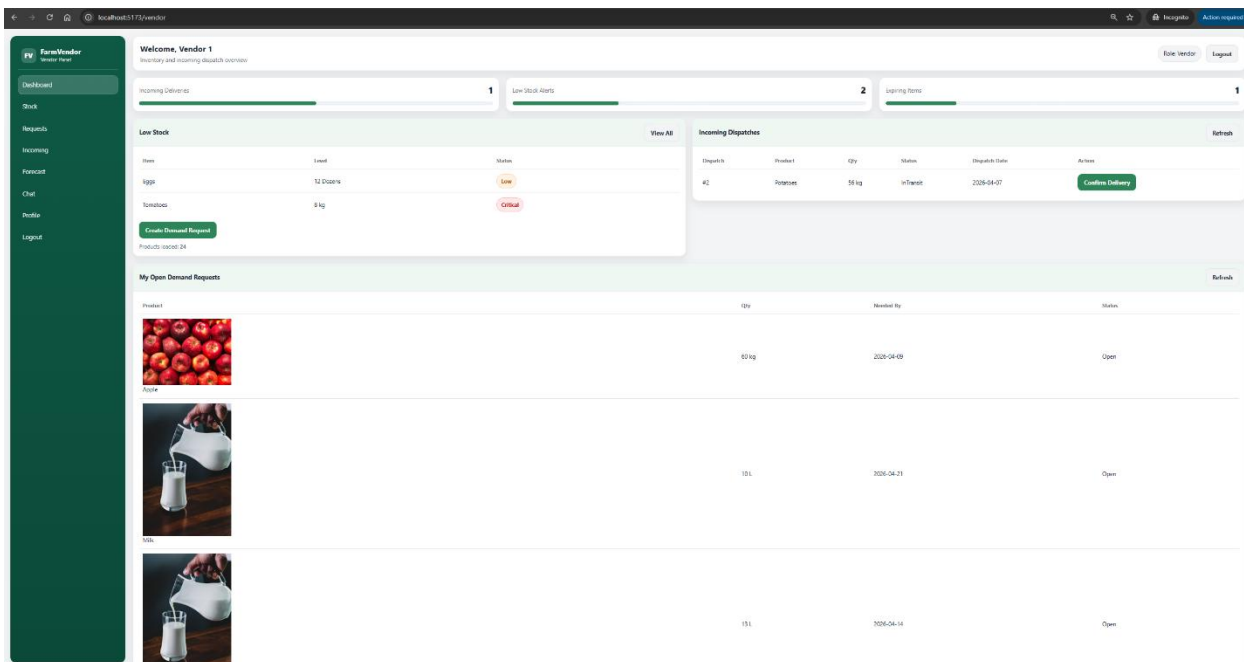
DemandRequest table storing:

- VendorId
- ProductId
- QuantityRequested
- Unit

- NeededBy
- Status
- CreatedAt

App

- VendorDashboard



Frontend

- VendorDashboard.jsx

```
FarmerDashboard.jsx | InventoryLot.sql U | VendorDashboard.jsx X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > VendorDashboard.jsx > Vendor_Dashboard
8   export default function Vendor_Dashboard() {
160  const renderProductOptionLabel = (option) => (
175  <div className={dispatchSelectMeta ? option.getDefaultMeta() : ''}>
176  </div>
177  </div>
178  );
179
180
181
182  return (
183    <div className="dashboard-page">
184      <div className="dashboard-layout">
185        <VendorSidebar />
186        <main className="main">
187          <div className="main-inner">
188            <div className="topbar">
189              <div><h1>Welcome, {displayName}</h1><div className="sub">Inventory and incoming dispatch overview</div></div>
190              <div className="topbar-right">
191                <span className="pill">Role: Vendor</span>
192                <button className="btn btn-secondary" onClick={logout}>Logout</button>
193              </div>
194            </div>
195
196            {error && <div className="auth-alert error">{error}</div>}
197
198            {/* Stats */}
199            <section className="stats">
200              <div className="stat-card"><div className="stat-top"><div className="stat-title">Incoming Deliveries</div><div className="stat-value">{incomingDispatches}</div></div><div className="stat-card"><div className="stat-top"><div className="stat-title">Low Stock Alerts</div><div className="stat-value">2</div></div><div className="stat-card"><div className="stat-top"><div className="stat-title">Expiring Items</div><div className="stat-value">1</div></div></div>
201
202            {/* Low Stock Table + Create Demand */}
203            <section className="grid-sections">
204              <div className="card">
205                <div className="card-header"><h2>Low Stock</h2><button className="btn btn-secondary">View All</button></div>
206                <div className="card-body">
207                  <table className="table">
208                    <thead><tr><th>Item</th><th>Level</th><th>Status</th></tr></thead>
209                    <tbody>
210                      <tr><td>Eggs</td><td>12 Dozens</td><td><span className="badge badge-orange">Low</span></td></tr>
211                      <tr><td>Tomatoes</td><td>8 kg</td><td><span className="badge badge-red">Critical</span></td></tr>
212                    </tbody>
213                  </table>
214                  <div style={{ marginTop: 12 }}><button className="btn btn-primary" onClick={openCreateDemand}>Create Demand Request</button></div>
215                  <div style={{ marginTop: 10, fontSize: 12, color: "#6b7280" }}>Products loaded: <b>{products.length}</b></div>
216                </div>
217              </div>
218            </section>
219          </div>
220        </main>
221      </div>
222    </div>
223  );
224  }
```

```
FarmerDashboard.jsx | inventoryLots.sql U | VendorDashboard.jsx X
5.4495_S3_Maneeshaf > implementations > FarmVendor > farmvendor-web > src > pages > VendorDashboard.jsx > Vendor_Dashboard > openCreateDemand

export default function Vendor_Dashboard() {
  const [error, setError] = useState("");

  // Products for dropdown
  const [products, setProducts] = useState([]);

  const [myRequests, setMyRequests] = useState([]);
  const [incomingDispatches, setIncomingDispatches] = useState([]);
  const [confirmingId, setConfirmingId] = useState(null);

  const [showCreateDemand, setShowCreateDemand] = useState(false);
  const [demandForm, setDemandForm] = useState({
    productId: "",
    quantityRequested: "",
    unit: "",
    neededBy: "",
  });
  const [savingDemand, setSavingDemand] = useState(false);

  const authHeaders = useMemo(() => ({
    "Content-Type": "application/json",
    Authorization: `Bearer ${token}`,
  }), [token]);

  const logout = () => {
    localStorage.removeItem("token");
    localStorage.removeItem("role");
    localStorage.removeItem("displayName");
    navigate("/login");
  };

  // ----- API LOADERS -----
  const loadProducts = async () => {
    const res = await fetch(`${API_BASE}/api/products/active`, { headers: authHeaders });
    const text = await res.text();
    if (!res.ok) throw new Error(text || "Failed to load products");
    setProducts(JSON.parse(text));
  };

  const loadMyRequests = async () => {
    const res = await fetch(`${API_BASE}/api/vendor/demandrequests?status=Open`, { headers: authHeaders });
    const text = await res.text();
    if (!res.ok) throw new Error(text || "Failed to load vendor requests");
    setMyRequests(JSON.parse(text));
  };

  const loadIncomingDispatches = async () => {
    const res = await fetch(`${API_BASE}/api/vendor/dispatches/incoming`, { headers: authHeaders });
    const text = await res.text();
    if (!res.ok) throw new Error(text || "Failed to load incoming dispatches");
    setIncomingDispatches(JSON.parse(text));
  };

  const loadDashboard = async () => {
    await Promise.all([loadProducts(), loadMyRequests(), loadIncomingDispatches()]);
  };

  const openCreateDemand = async () => {
    setError("");
    try { if (products.length === 0) await loadProducts(); } catch (e) { setError(e?.message || "Failed to load products"); return; }
    setDemandForm({ productId: "", quantityRequested: "", unit: "", neededBy: "" });
    setShowCreateDemand(true);
  };
}
```


FarmerDashboard.jsx InventoryLot.sql U **VendorDashboard.jsx** X

6_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > VendorDashboard.jsx > Vendor_Dashboard > submitDemandRequest

```

8  export default function Vendor_Dashboard() {
9    const loadMyRequests = async () => {
10     if (!res.ok) throw new Error(text || "Failed to load vendor requests");
11     setMyRequests(JSON.parse(text));
12   };
13
14   const loadIncomingDispatches = async () => {
15     const res = await fetch(`${API_BASE}/api/vendor/dispatches/incoming`, { headers: authHeaders });
16     const text = await res.text();
17     if (!res.ok) throw new Error(text || "Failed to load incoming dispatches");
18     setIncomingDispatches(JSON.parse(text));
19   };
20
21   const loadDashboard = async () => {
22     await Promise.all([loadProducts(), loadMyRequests(), loadIncomingDispatches()]);
23   };
24
25   const openCreateDemand = async () => {
26     setError("");
27     try { if (products.length === 0) await loadProducts(); } catch (e) { setError(e?.message || "Failed to load products"); return; }
28     setDemandForm({ productId: "", quantityRequested: "", unit: "", neededBy: "" });
29     setShowCreateDemand(true);
30   };
31
32   const submitDemandRequest = async (e) => {
33     e.preventDefault();
34     setError("");
35
36     if (!demandForm.productId) return setError("Please select a product.");
37     if (!demandForm.quantityRequested || Number(demandForm.quantityRequested) <= 0)
38       return setError("Quantity must be greater than 0.");
39     if (!demandForm.neededBy) return setError("Please select Needed By date.");
40
41     setSavingDemand(true);
42     try {
43       const payload = {
44         productId: Number(demandForm.productId),
45         quantityRequested: Number(demandForm.quantityRequested),
46         unit: demandForm.unit?.trim() || null,
47         neededBy: new Date(demandForm.neededBy).toISOString(),
48       };
49
50       const res = await fetch(`${API_BASE}/api/vendor/demandrequests`, {
51         method: "POST",
52         headers: authHeaders,
53         body: JSON.stringify(payload),
54       });
55
56       const text = await res.text();
57       if (!res.ok) throw new Error(text || "Failed to create demand request.");
58
59       setShowCreateDemand(false);
60       await loadMyRequests();
61     } catch (err) {
62       setError(err?.message || "Failed to create demand request.");
63     } finally { setSavingDemand(false); }
64   };
65
66   const confirmDelivery = async (dispatchId) => {

```

Backend

- VendorDemandRequestController.cs

```
FarmerDashboard.jsx  InventoryLot.sql U  VendorDemandRequestsController.cs X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > VendorDemandRequestsController.cs
1  using FarmVendor.Api.Data;
2  using FarmVendor.Api.Models;
3  using FarmVendor.Api.Models.DTOs;
4  using Microsoft.AspNetCore.Authorization;
5  using Microsoft.AspNetCore.Mvc;
6  using Microsoft.EntityFrameworkCore;
7  using System.Security.Claims;
8
9  namespace FarmVendor.Api.Controllers;
10
11  [ApiController]
12  [Route("api/vendor/demandrequests")]
13  [Authorize] // JWT required
14  public class VendorDemandRequestsController : ControllerBase
15  {
16      private readonly AppDbContext _db;
17
18      public VendorDemandRequestsController(AppDbContext db)
19      {
20          _db = db;
21      }
22
23      private string GetUserId()
24      => User.FindFirstValue(ClaimTypes.NameIdentifier) ?? "";
25
26      // GET: api/vendor/demandrequests?status=Open
27      [HttpGet]
28      public async Task<ActionResult> GetMyDemandRequests([FromQuery] string? status = null)
29      {
30          var vendorId = GetUserId();
31          if (string.IsNullOrEmpty(vendorId)) return Unauthorized();
32
33          var q = _db.DemandRequest
34              .AsNoTracking()
35              .Where(r => r.VendorId == vendorId)
36              .Include(r => r.Product)
37              .Include(r => r.Vendor)
38              .AsQueryable();
39
40          if (!string.IsNullOrEmpty(status))
41              q = q.Where(r => r.Status == status);
42
43          var rows = await q
44              .OrderByDescending(r => r.CreatedAt)
45              .Take(100)
46              .Select(r => new FarmerDemandRequestRowDto
47              {
48                  DemandRequestId = r.DemandRequestId,
49                  ProductId = r.ProductId,
50                  Product = r.Product.Name,
51                  Qty = r.QuantityRequested,
52                  Unit = r.Unit,
53                  NeededBy = r.NeededBy,
54                  Status = r.Status,
55                  VendorId = r.VendorId,
56                  VendorName = r.Vendor.UserName,
57                  VendorEmail = r.Vendor.Email,
58                  ImageUrl = r.Product.ImageUrl,
59                  ImageThumbUrl = r.Product.ImageThumbUrl
60              })
61              .ToListAsync();
62
63          return Ok(rows);
64      }
65  }
```

```

FarmerDashboard.jsx | InventoryLots.sql U | VendorDemandRequestsController.cs X
26.4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > VendorDemandRequestsController.cs
5  {
29  {
33      var rows = await q
37      {
38          imageUrl = r.Product.ImageUrl,
39          ImageThumbUrl = r.Product.ImageThumbUrl
40      })
41      .ToListAsync();
42
43      return Ok(rows);
44  }
45
46  // POST: api/vendor/demandrequests
47  [HttpPost]
48  public async Task<IActionResult> CreateDemandRequest([FromBody] CreateDemandRequestDto dto)
49  {
50      var vendorId = GetUserId();
51      if (string.IsNullOrEmpty(vendorId)) return Unauthorized();
52
53      if (dto.ProductId <= 0) return BadRequest("ProductId is required.");
54      if (dto.QuantityRequested <= 0) return BadRequest("QuantityRequested must be > 0.");
55      if (dto.NeededBy == default) return BadRequest("NeededBy is required.");
56
57      var product = await _db.Product
58          .FirstOrDefaultAsync(p => p.ProductId == dto.ProductId && p.IsActive);
59
60      if (product == null) return NotFound("Product not found or inactive.");
61
62      var unit = !string.IsNullOrEmpty(dto.Unit) ? dto.Unit.Trim() : (product.DefaultUnit ?? "kg");
63
64      var req = new DemandRequest
65      {
66          VendorId = vendorId,
67          ProductId = dto.ProductId,
68          QuantityRequested = dto.QuantityRequested,
69          Unit = unit,
70          NeededBy = dto.NeededBy,
71          Status = "Open",
72          CreatedAt = DateTime.UtcNow
73      };
74
75      _db.DemandRequest.Add(req);
76      await _db.SaveChangesAsync();
77
78      // Return a row (good for UI immediately)
79      return Ok(new FarmerDemandRequestRowDto
80      {
81          DemandRequestId = req.DemandRequestId,
82          ProductId = req.ProductId,
83          Product = product.Name,
84          Qty = req.QuantityRequested,
85          Unit = req.Unit,
86          NeededBy = req.NeededBy,
87          Status = req.Status
88      });
89  }
90  }

```

- DemandRequest.cs

```
FarmerDashboard.jsx | InventoryLot.sql U | DemandRequest.cs X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > DemandRequest.cs
1 namespace FarmVendor.Api.Models;
2
3 public class DemandRequest
4 {
5     public int DemandRequestId { get; set; }
6
7     public string VendorId { get; set; } = "";
8     public ApplicationUser Vendor { get; set; } = null!;
9
10    public int ProductId { get; set; }
11    public Product Product { get; set; } = null!;
12
13    public decimal QuantityRequested { get; set; }
14
15    public string Unit { get; set; } = "kg";
16
17    public DateTime NeededBy { get; set; }
18
19    public string Status { get; set; } = "Open";
20    // Open, Accepted, Fulfilled, Cancelled
21
22    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
23 }
24
```

- CreateDemandRequestDto.cs

```
FarmerDashboard.jsx | InventoryLot.sql U | CreateDemandRequestDto.cs X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > DTOs > CreateDemandRequestDto.cs
1 namespace FarmVendor.Api.Models.DTOs;
2
3 public class CreateDemandRequestDto
4 {
5     public int ProductId { get; set; }
6     public decimal QuantityRequested { get; set; }
7     public string? Unit { get; set; } // optional; can default from product
8     public DateTime NeededBy { get; set; } // required
9 }
```

Feature 6: Dispatch Management Module

This feature enables farmers to create dispatches based on vendor demand requests and allows vendors to confirm delivery. It supports operational coordination between both roles.

This feature includes:

- Create Dispatch action from Farmer dashboard
- Request selection for dispatch
- Incoming dispatch list on Vendor dashboard
- Delivery confirmation
- Dispatch status tracking

The design consists of:

- Request-to-dispatch workflow
- Vendor-side delivery confirmation
- Status updates such as Planned / InTransit / Delivered

Implementation

Frontend

- Create Dispatch action in FarmerDashboard.jsx
- Incoming dispatch list in VendorDashboard.jsx
- Confirm Delivery button on vendor side
- Product image shown with requests in dropdown

Backend

- FarmerDispatchController.cs
- VendorDispatchesController.cs
- CreateDispatchDto.cs
- VendorConfirmDeliveryDto.cs
- Delivery status update logic

Database

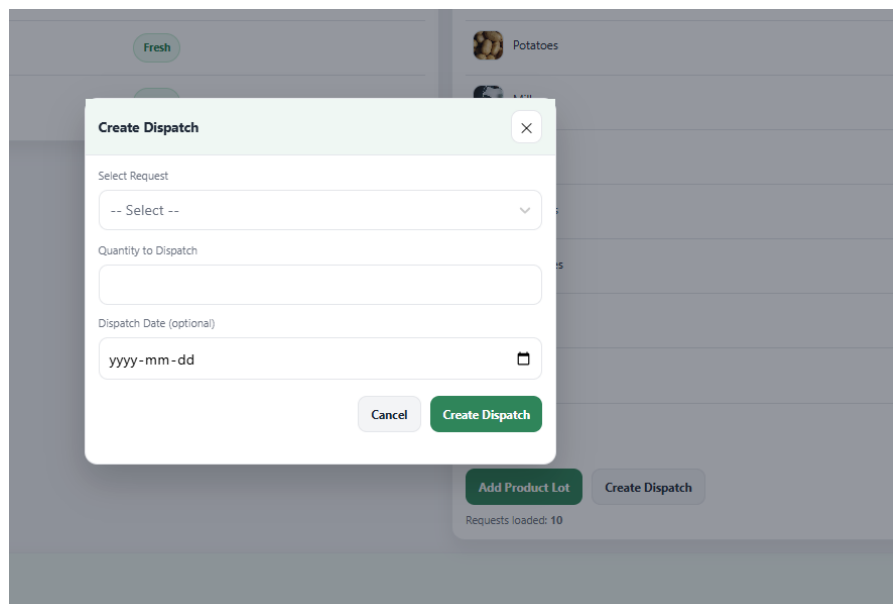
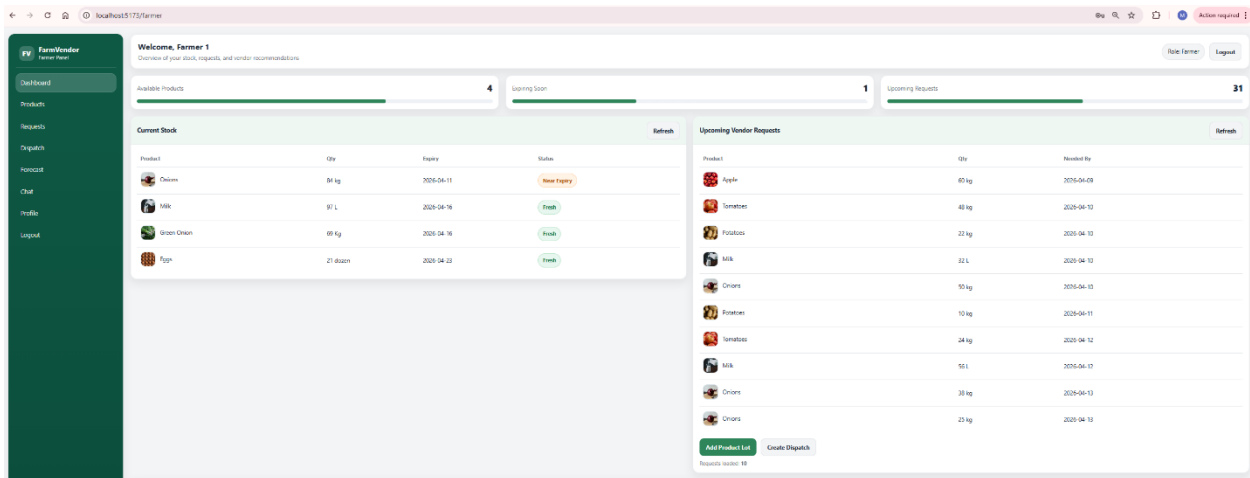
Dispatch table storing:

- FarmerId
- VendorId
- ProductId
- DemandRequestId

- QuantityDispatched
- Unit
- DispatchDate
- DeliveryStatus
- CreatedAt

App

- FarmerDashboard



Create Dispatch ×

Select Request

-- Select -- ▼

-  **Apple**
60 kg • Needed: 2026-04-09
-  **Tomatoes**
48 kg • Needed: 2026-04-10
-  **Potatoes**
22 kg • Needed: 2026-04-10
-  **Milk**
32 L • Needed: 2026-04-10
-  **Onions**
50 kg • Needed: 2026-04-10
-  **Potatoes**

Requests loaded: 10

Create Dispatch ×

 **Please check this form** ×
Not enough stock. Available: 0.00, Requested dispatch: 48

Select Request

 **Tomatoes** × ▼
48 kg • Needed: 2026-04-10

Quantity to Dispatch

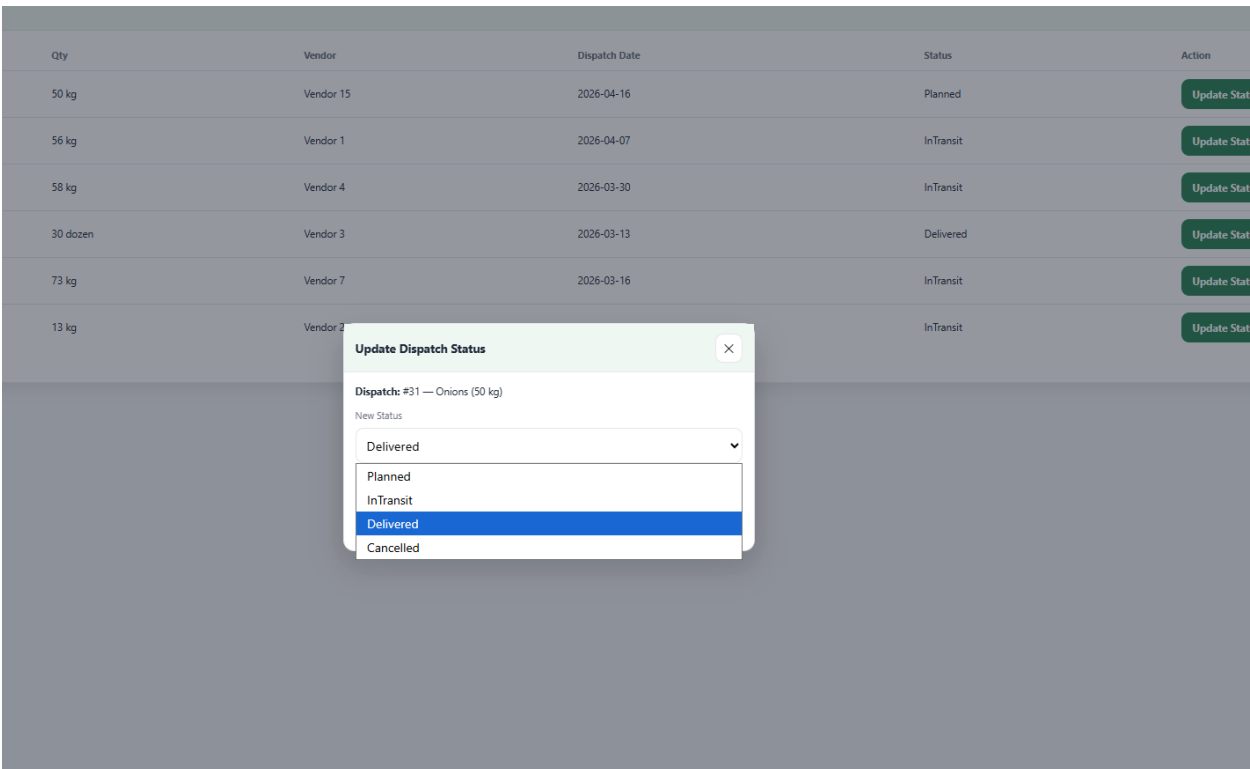
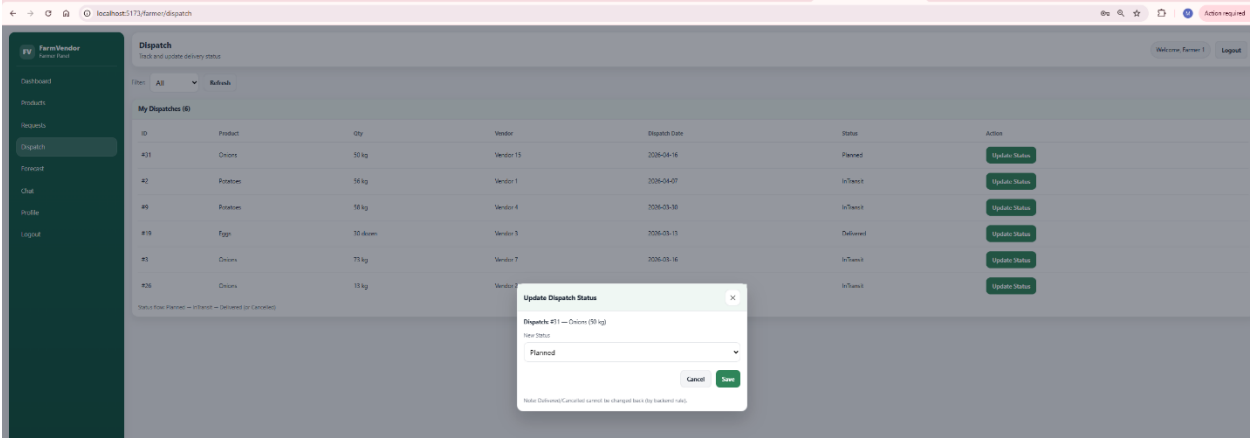
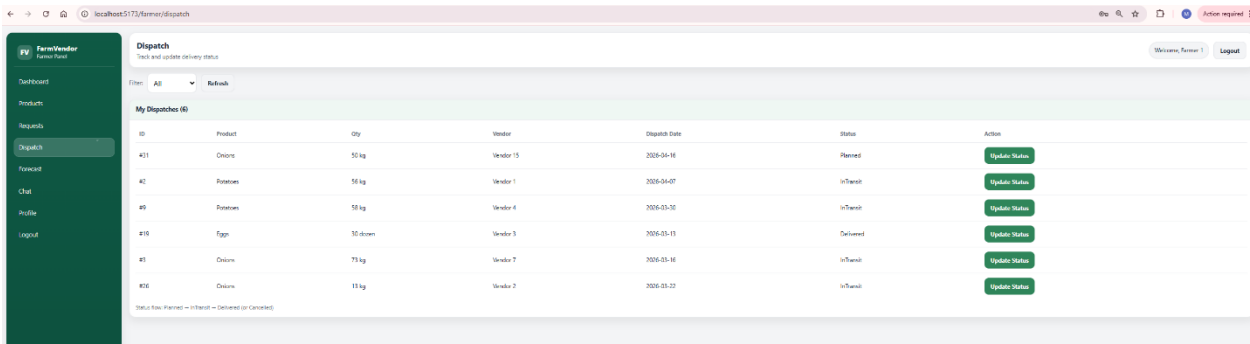
48

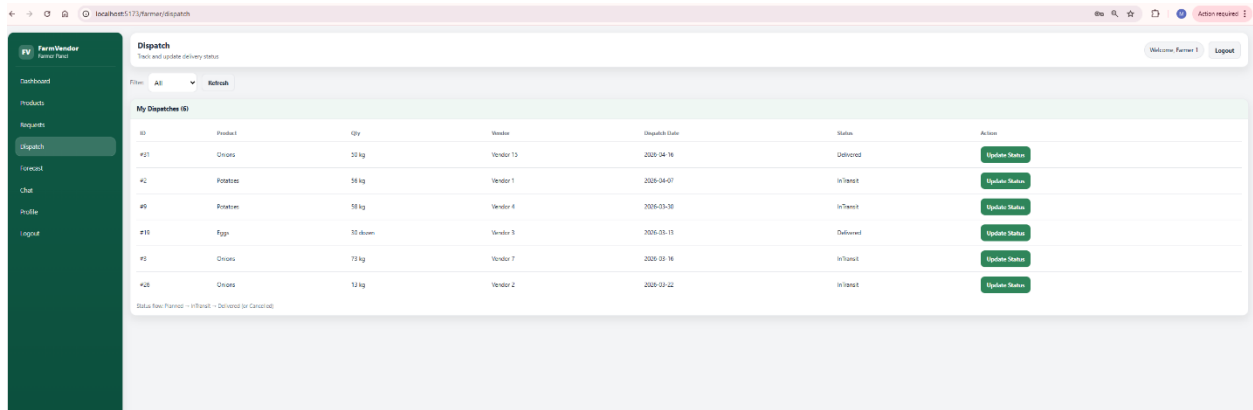
Dispatch Date (optional)

2026-04-16 

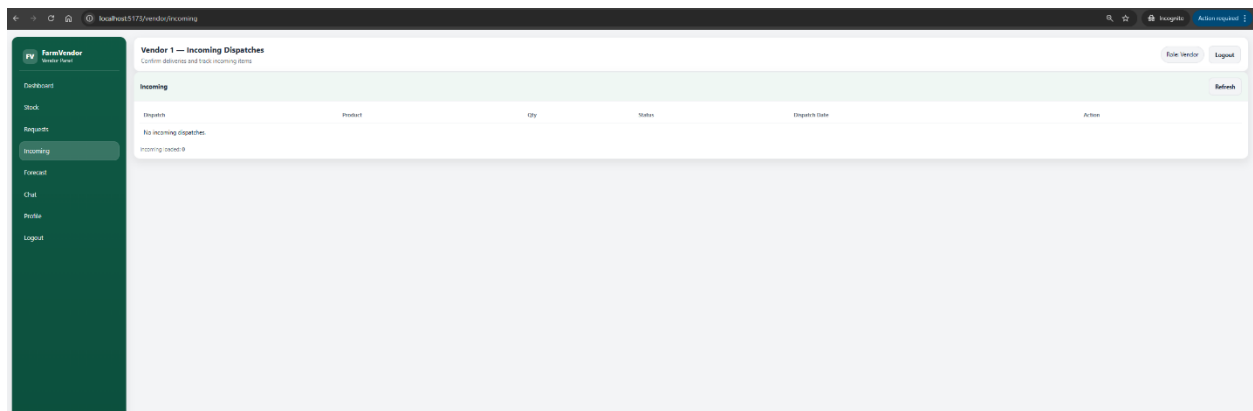
Cancel Create Dispatch

Requests loaded: 10





Vendor-dispatch incoming



Frontend

```
FarmerDashboard.jsx  InventoryLots.jsx  FarmerDispatch.jsx X
W26_4495_S3_ManeeshaE > implementations > FarmVendor > farmvendor-web > src > pages > FarmerDispatch.jsx > ...

1  import React, { useEffect, useMemo, useState } from "react";
2  import { useNavigate } from "react-router-dom";
3  import "../styles/dashboard.css";
4  import FarmerSidebar from "../assets/components/FarmerSidebar";
5
6  const API_BASE = import.meta.env.VITE_API_URL;
7
8  const STATUS_OPTIONS = ["All", "Planned", "InTransit", "Delivered", "Cancelled"];
9
10 export default function FarmerDispatch() {
11   const navigate = useNavigate();
12   const token = localStorage.getItem("token");
13   const displayName = localStorage.getItem("displayName") || "Farmer";
14
15   const authHeaders = useMemo(
16     () => ({
17       "Content-Type": "application/json",
18       Authorization: `Bearer ${token}`,
19     }),
20     [token]
21   );
22
23   const logout = () => {
24     localStorage.clear();
25     navigate("/login");
26   };
27
28   const [statusFilter, setStatusFilter] = useState("All");
29   const [dispatches, setDispatches] = useState([]);
30   const [loading, setLoading] = useState(true);
31   const [error, setError] = useState("");
32
33   // Status modal
34   const [showStatusModal, setShowStatusModal] = useState(false);
35   const [selectedDispatch, setSelectedDispatch] = useState(null);
36   const [newStatus, setNewStatus] = useState("Planned");
37   const [saving, setSaving] = useState(false);
38
39   const loadDispatches = async () => {
40     setLoading(true);
41     setError("");
42     try {
43       const qs = statusFilter === "All" ? "" : `?status=${encodeURIComponent(statusFilter)}`;
44       const res = await fetch(`${API_BASE}/api/farmer/dispatches${qs}`, { headers: authHeaders });
45       const text = await res.text();
46       if (!res.ok) throw new Error(text || "Failed to load dispatches");
47       setDispatches(JSON.parse(text));
48     } catch (e) {
49       setError(e?.message || "Failed to load dispatches");
50     } finally {
51       setLoading(false);
52     }
53   };
54
55   const openStatusModal = (d) => {
56     setError("");
57     setSelectedDispatch(d);
58     setNewStatus(d.deliveryStatus || "Planned");
59     setShowStatusModal(true);
60   };
61
62   const saveStatus = async (e) => {
63     e.preventDefault();
64   };
65 }
```

Backend

```
FarmerDashboard.jsx | InventoryLots.js | DispatchesController.cs X
6.4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Controllers > DispatchesController.cs
1 using FarmVendor.Api.Data;
2 using FarmVendor.Api.Models;
3 using FarmVendor.Api.Models.DTOS;
4 using Microsoft.AspNetCore.Authorization;
5 using Microsoft.AspNetCore.Mvc;
6 using Microsoft.EntityFrameworkCore;
7 using System.Security.Claims;
8
9 namespace FarmVendor.Api.Controllers;
10
11 [ApiController]
12 [Route("api/dispatches")]
13 [Authorize]
14 public class DispatchesController : ControllerBase
15 {
16     private readonly AppDbContext _db;
17
18     public DispatchesController(AppDbContext db)
19     {
20         _db = db;
21     }
22
23     private string GetUserId()
24     => User.FindFirstValue(ClaimTypes.NameIdentifier) ?? "";
25
26     [HttpPost]
27     public async Task<IActionResult> CreateDispatch([FromBody] CreateDispatchDto dto)
28     {
29         var farmerId = GetUserId();
30         if (string.IsNullOrEmpty(farmerId)) return Unauthorized();
31
32         if (dto.DemandRequestId <= 0) return BadRequest("DemandRequestId is required.");
33         if (dto.QuantityDispatched <= 0) return BadRequest("QuantityDispatched must be greater than 0.");
34
35         // Load demand request + product + vendor
36         var req = await _db.DemandRequest
37             .Include(r => r.Product)
38             .FirstOrDefaultAsync(r => r.DemandRequestId == dto.DemandRequestId);
39
40         if (req == null) return NotFound("Demand request not found.");
41
42         if (req.Status != "Open")
43             return BadRequest($"Demand request is not Open. Current status: {req.Status}");
44
45         // (Optional but recommended) ensure farmer has inventory for this product
46         var availableQty = await _db.InventoryLot
47             .Where(l => l.FarmerId == farmerId && l.ProductId == req.ProductId)
48             .SumAsync(l => l.QuantityAvailable);
49
50         if (availableQty < dto.QuantityDispatched)
51             return BadRequest($"Not enough stock. Available: {availableQty}, Requested dispatch: {dto.QuantityDispatched}");
52
53         var dispatch = new Dispatch
54         {
55             DemandRequestId = req.DemandRequestId,
56             FarmerId = farmerId,
57             VendorId = req.VendorId,
58             ProductId = req.ProductId,
59             QuantityDispatched = dto.QuantityDispatched,
60             Unit = string.IsNullOrEmpty(req.Unit) ? "kg" : req.Unit,
61             DispatchDate = dto.DispatchDate ?? DateTime.UtcNow,
62             DeliveryStatus = "Planned",
63             CreatedAt = DateTime.UtcNow
64         }
65     }
66 }
```

PROBLEMS 1 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | QUERY RESULTS

```
FarmerDashboard.jsx × InventoryLot.sql U CreateDispatchDto.cs ×
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > DTOs > CreateDispatchDto.cs
1 namespace FarmVendor.Api.Models.DTOs;
2
3 public class CreateDispatchDto
4 {
5     public int DemandRequestId { get; set; }
6     public decimal QuantityDispatched { get; set; }
7     public DateTime? DispatchDate { get; set; } // this is optional
8 }
```

```
FarmerDashboard.jsx × InventoryLot.sql U UpdateDispatchStatusDto.cs ×
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > DTOs > UpdateDispatchStatusDto.cs
1 namespace FarmVendor.Api.Models.DTOs;
2
3 public class UpdateDispatchStatusDto
4 {
5     public string DeliveryStatus { get; set; } = "Planned";
6 }
```

```
FarmerDashboard.jsx × InventoryLot.sql U VendorConfirmDeliveryDto.cs ×
W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > DTOs > VendorConfirmDeliveryDto.cs
1 namespace FarmVendor.Api.Models.DTOs;
2
3 public class VendorConfirmDeliveryDto
4 {
5     public int DispatchId { get; set; }
6 }
```

FarmerDashboard.jsxInventoryLot.sql UDispatch.cs X

W26_4495_S3_ManeeshaE > implementations > FarmVendor > FarmVendor.Api > Models > Dispatch.cs

```
1 namespace FarmVendor.Api.Models;
2
3 public class Dispatch
4 {
5     public int DispatchId { get; set; }
6
7     public int? DemandRequestId { get; set; }
8     public DemandRequest? DemandRequest { get; set; }
9
10    public string FarmerId { get; set; } = "";
11    public ApplicationUser Farmer { get; set; } = null!;
12
13    public string VendorId { get; set; } = "";
14    public ApplicationUser Vendor { get; set; } = null!;
15
16    public int ProductId { get; set; }
17    public Product Product { get; set; } = null!;
18
19    public decimal QuantityDispatched { get; set; }
20
21    public string Unit { get; set; } = "kg";
22
23    public DateTime DispatchDate { get; set; }
24
25    public string DeliveryStatus { get; set; } = "Planned";
26    // Planned, InTransit, Delivered, Cancelled
27
28    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
29 }
30
```


Feature 7: Dashboard Analytics and Status Badge System

This feature improves usability by showing summarized operational insights and visual status indicators for both farmer and vendor dashboards.

Design includes:

- Analytics summary cards
- Expiry status badges
- Low stock alerts
- Dynamic UI rendering based on live API data

Logic implemented using:

- Expiry date difference calculation
- Quantity threshold rules
- React conditional rendering
- API-driven dashboard refresh

Implementation

Frontend

- FarmerDashboard.jsx
- VendorDashboard.jsx
- dashboard.css
- Dynamic summary cards and badges

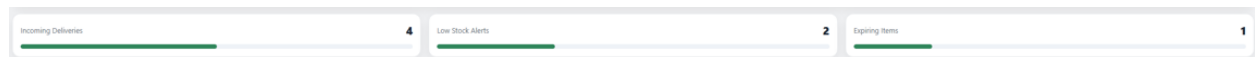
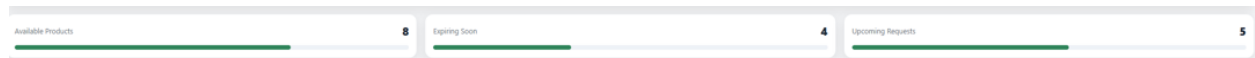
Backend

- FarmerController.cs
- Vendor dashboard API endpoints
- Aggregation logic for counts and summary metrics

Database

- Data derived from:
 - InventoryLot

- DemandRequest
- Dispatch



Status	Status
Low	Near Expiry
Critical	Near Expiry
	Near Expiry
	Fresh
	Fresh
	Fresh
	Fresh
	Fresh
	Fresh
	Fresh
	Fresh

6. Evaluation Technique

The FarmVendor system was evaluated using a combination of different algorithm comparison, functional and integration testing, scenario-based evaluation, and usability assessment. These evaluation techniques were selected to validate system correctness, decision quality, and practical usability in real-world agricultural supply-chain scenarios. The outcomes of these evaluations directly influenced several design and implementation decisions in the final system.

Forecasting model evaluation

The demand forecasting component was evaluated by comparing multiple approaches before finalizing the forecasting algorithm. Initially, simple logic-based methods, including basic averages and Moving Average techniques, were considered due to their ease of implementation. However, preliminary testing revealed that these approaches were unable to capture temporal trends, seasonal variations, and fluctuations present in historical demand request data.

As an alternative, the ML.NET Singular Spectrum Analysis (SSA) time-series forecasting algorithm was implemented and evaluated using multi-period historical demand data. SSA demonstrated superior performance in identifying trends and generating smoother, forward-looking demand predictions compared to traditional statistical methods.

Design Decision Outcome:

Based on this evaluation, the logic-based forecasting approach was replaced with ML.NET SSA. Forecasting results were fully integrated into the system and used as a core input for vendor recommendation and dispatch planning, enabling proactive and data-driven decision-making.

Comparison between ML.NET SSA model and Moving Average model

To evaluate the effectiveness of the demand forecasting component in the FarmVendor system, two forecasting approaches were implemented and compared: a Moving Average model and a machine-learning-based ML.NET Singular Spectrum Analysis (SSA) model. Both models were applied to the same historical demand dataset across multiple vendors and products, allowing a direct comparison of forecast behavior and suitability for decision support.

A	B	C	D	E	F	G
Vendor	Product	Forecast Date	Qty	Model	Lookback	Created At
Vendor 8	Eggs	2026-04-08	27.57	MLNET_SSA		2026-04-09
Vendor 6	Milk	2026-04-08	7.4	MLNET_SSA		2026-04-09
Vendor 6	Tomatoes	2026-04-08	50.59	MLNET_SSA		2026-04-09
Vendor 12	Eggs	2026-04-08	30.58	MLNET_SSA		2026-04-09
Vendor 13	Milk	2026-04-08	25.57	MLNET_SSA		2026-04-09
Vendor 1	Tomatoes	2026-04-08	14.52	MLNET_SSA		2026-04-09
Vendor 5	Milk	2026-04-08	39.59	MLNET_SSA		2026-04-09
Vendor 5	Tomatoes	2026-04-08	29.57	MLNET_SSA		2026-04-09
Vendor 3	Milk	2026-04-08	42.26	MLNET_SSA		2026-04-09
Vendor 9	Milk	2026-04-08	36.58	MLNET_SSA		2026-04-09
Vendor 9	Tomatoes	2026-04-08	26.57	MLNET_SSA		2026-04-09
Vendor 7	Eggs	2026-04-08	52.86	MLNET_SSA		2026-04-09
Vendor 2	Tomatoes	2026-04-08	21.56	MLNET_SSA		2026-04-09
Vendor 10	Tomatoes	2026-04-08	36.55	MLNET_SSA		2026-04-09
Vendor 4	Eggs	2026-04-08	34.58	MLNET_SSA		2026-04-09
Vendor 15	Eggs	2026-04-08	17.54	MLNET_SSA		2026-04-09
Vendor 11	Tomatoes	2026-04-08	56.6	MLNET_SSA		2026-04-09
Vendor 14	Tomatoes	2026-04-08	30.31	MLNET_SSA		2026-04-09

Vendor	Product	Forecast Date	Qty	Model	Lookback	Created At
Vendor 8	Eggs	2026-04-08	28	MovingAverage	3	2026-04-09
Vendor 6	Milk	2026-04-08	8	MovingAverage	3	2026-04-09
Vendor 6	Tomatoes	2026-04-08	51	MovingAverage	3	2026-04-09
Vendor 12	Eggs	2026-04-08	31	MovingAverage	3	2026-04-09
Vendor 13	Milk	2026-04-08	26	MovingAverage	3	2026-04-09
Vendor 1	Tomatoes	2026-04-08	15	MovingAverage	3	2026-04-09
Vendor 5	Milk	2026-04-08	40	MovingAverage	3	2026-04-09
Vendor 5	Tomatoes	2026-04-08	30	MovingAverage	3	2026-04-09
Vendor 3	Milk	2026-04-08	42.67	MovingAverage	3	2026-04-09
Vendor 9	Milk	2026-04-08	37	MovingAverage	3	2026-04-09
Vendor 9	Tomatoes	2026-04-08	27	MovingAverage	3	2026-04-09
Vendor 7	Eggs	2026-04-08	35.67	MovingAverage	3	2026-04-09
Vendor 2	Tomatoes	2026-04-08	22	MovingAverage	3	2026-04-09
Vendor 10	Tomatoes	2026-04-08	37	MovingAverage	3	2026-04-09
Vendor 4	Eggs	2026-04-08	35	MovingAverage	3	2026-04-09
Vendor 15	Eggs	2026-04-08	18	MovingAverage	3	2026-04-09
Vendor 11	Tomatoes	2026-04-08	57	MovingAverage	3	2026-04-09
Vendor 14	Tomatoes	2026-04-08	30.67	MovingAverage	3	2026-04-09

Moving Average Model Evaluation

The Moving Average model was implemented using a fixed lookback window of **three periods**, producing forecasts based on recent historical demand values. As shown in the dataset, the Moving Average model generated reasonable baseline estimates for short-term demand. For example, Vendor 6's milk demand was forecast at **8 units**, while Vendor 11's tomato demand was forecast at **57 units**.

However, the Moving Average model exhibited several limitations. Because it relies strictly on recent averages, it is **highly sensitive to short-term fluctuations** and does not capture longer-term trends or demand patterns. In cases where historical demand varied significantly, the model produced abrupt changes or plateau-like forecasts. Additionally, the Moving Average approach treats all past observations equally within the lookback window, limiting its ability to adapt to evolving demand behavior.

ML.NET SSA Model Evaluation

The ML.NET SSA model was implemented to address the limitations of simple statistical forecasting. SSA is a time-series decomposition technique that identifies underlying trends

and patterns in historical data. When applied to the same dataset, the SSA model produced forecasts that were smoother and more trend-aware.

For instance, Vendor 6's milk demand forecast shifted from 8 units (Moving Average) to 7.4 units (SSA), reflecting a slight adjustment based on learned patterns rather than a direct average. Similarly, Vendor 7's egg demand increased from 35.67 units under the Moving Average model to 52.86 units under the SSA model, indicating that SSA detected an upward demand trend that the Moving Average approach failed to capture.

Across most vendor-product combinations, SSA forecasts differed moderately from Moving Average values rather than drastically, suggesting that SSA refines forecasts by accounting for trend continuity rather than simply smoothing past values.

Quantitative and Behavioral Comparison

Overall, both models produced forecasts within a comparable numerical range; however, their forecasting behavior differed significantly:

- The Moving Average model provided quick, interpretable baseline forecasts but struggled with trend detection and adaptability.
- The ML.NET SSA model produced more stable and forward-looking forecasts by learning from historical patterns, reducing sensitivity to short-term noise.

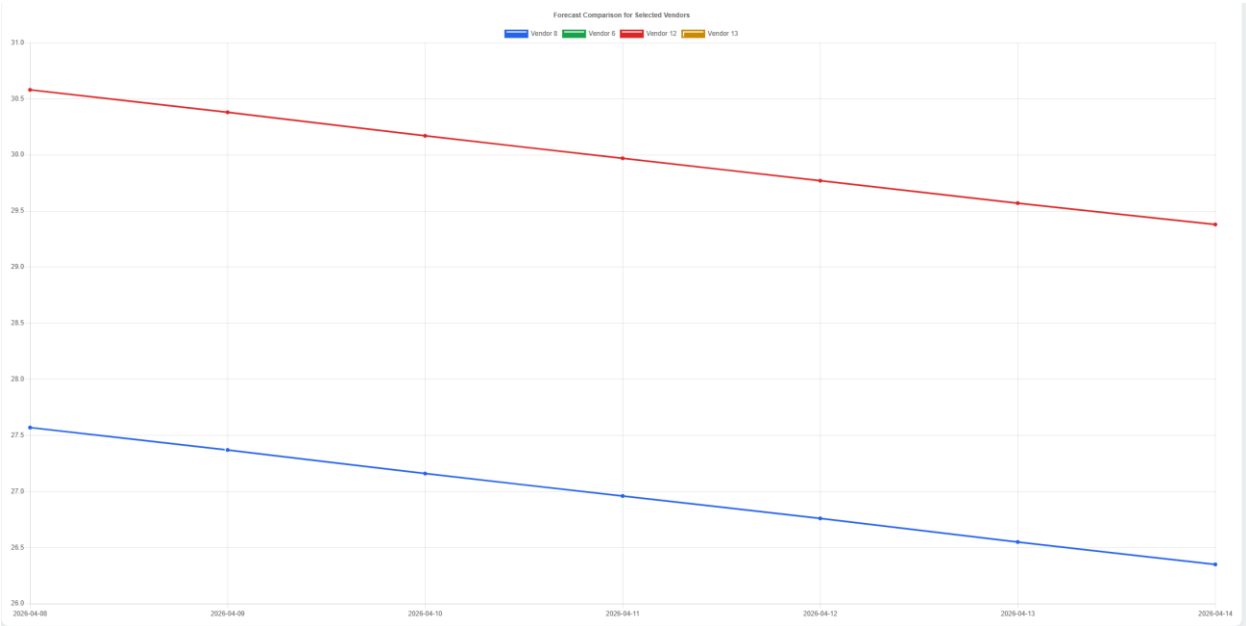
SSA forecasts also demonstrated greater consistency across similar products and vendors, which is particularly important for downstream processes such as vendor recommendation and dispatch planning.

Based on this evaluation, the Moving Average model was retained as a baseline comparison, while the ML.NET SSA model was selected as the primary forecasting mechanism for the FarmVendor platform. The SSA model's ability to capture trends and produce smoother predictions made it more suitable for integration into the vendor recommendation system and overall decision-support workflow.

By incorporating SSA-based forecasts, the system shifted from reactive forecasting toward proactive demand planning, enabling farmers to make better-informed inventory and dispatch decisions.

Aspect	Moving Average	ML.NET SSA
Model Type	Statistical	Machine-Learning (Time-Series)
Trend Detection	Limited	Strong
Sensitivity to Noise	High	Lower
Forecast Smoothness	Moderate	High
Interpretability	High	Moderate
Suitability for Decision Support	Limited	High

- Graph generated for **ML.NET SSA** Model



Recommendation Logic Evaluation

The vendor recommendation feature was evaluated by comparing different decision-making strategies. An optimization-based approach was initially explored; however, early experimentation showed that such models were difficult to interpret and explain to non-technical users, limiting their practical usability.

To address this, a weighted scoring-based recommendation system was implemented and evaluated through scenario-based testing. Multiple test scenarios were executed by varying key inputs such as forecasted demand, available inventory, geographic distance (calculated using the Haversine formula), and historical relationship scores. The resulting rankings were reviewed to ensure consistency, transparency, and logical behavior.

Design Decision Outcome:

The optimization-based approach was replaced with a scoring-based model. This change improved interpretability, transparency, and user trust, making the recommendation system more suitable for real-world agricultural users.

Functional and Integration Testing

Functional testing was conducted across all major system modules, including:

- Role-based authentication and authorization
- Farmer inventory management
- Vendor demand request lifecycle
- Dispatch creation and delivery confirmation
- Real-time chat communication

Integration testing focused on validating seamless interaction between the frontend (React) and backend (ASP.NET Core APIs), as well as ensuring proper enforcement of JWT-based security. SignalR-based real-time messaging was tested to confirm message delivery, role isolation, and conversation persistence.

Design Decision Outcome:

Testing results led to improvements in API validation, error handling, and frontend state management, ensuring system stability and secure role-based access.

Usability Evaluation

Usability evaluation was performed through developer testing, instructor testing and informal peer testing, focusing on common user workflows such as adding inventory, submitting demand requests, creating dispatches, and initiating chat conversations, adding products. Observations were made regarding ease of navigation, clarity of information, and error frequency.

Based on this evaluation, several UI/UX enhancements were implemented, including searchable dropdowns, improved input validation, shared sidebar navigation, and clearer status indicators.

Design Decision Outcome:

These changes reduced manual input errors, improved task completion efficiency, and enhanced the overall user experience for both farmers and vendors.

Overall, the evaluation techniques confirmed that the FarmVendor system functions correctly, produces meaningful decision-support outputs, and remains usable for non-technical users. The evaluation process directly informed key design choices, including the adoption of machine-learning-based forecasting, the use of a transparent scoring-based recommendation model, and multiple usability improvements. This iterative evaluation-driven approach ensured alignment between the project’s research objectives and the final implemented solution.

Evaluation Method	What Was Evaluated	Outcome
Model comparison	Moving Average vs ML.NET SSA	SSA selected as primary model
Functional testing	End-to-end workflows	Main features worked correctly
Usability evaluation	Dashboard clarity and ease of use	UI improved through redesign

7. Reflections/Discussions

The development of FarmVendor provided valuable insights into full-stack application design, machine learning integration, and user-centered system development. The project involved combining ASP.NET Core, React, SQL Server, ML.NET, SignalR, and external APIs into one integrated platform.

Challenges Faced

One of the most significant challenges was implementing machine learning forecasting in a realistic way. The ML.NET SSA model required sufficient historical demand data, which meant that data preparation and simulation became an essential part of the project. Other challenges included synchronizing frontend and backend changes, handling JWT and SignalR authentication issues, and ensuring that dashboard visualizations correctly reflected saved forecast data. Integrating product images and updating the schema without breaking existing workflows was also challenging.

Lessons Learned

This project emphasized the importance of iterative development, testing, and refinement. I learned that the success of a machine learning feature depends not only on the algorithm, but also on the quality and structure of the historical data. I also learned that usability is a major part of system effectiveness; visual improvements such as product images, cleaner dashboards, and better interaction patterns can significantly improve how users understand and use a system.

Most Satisfying Parts

The most satisfying aspect of the project was successfully integrating multiple advanced features into one working system. In particular, generating ML-based forecasts and visualizing them in charts was highly rewarding. It was also satisfying to enhance the dashboard with image-based product identification and real-time chat communication, making the application feel practical and complete.

Future Improvements

Future work could include deploying the system to a cloud platform, collecting real agricultural data for more accurate model training, and performing formal user testing with farmers and vendors. Additional enhancements could include push notifications, mobile

access, more advanced recommendation models, and integration with external logistics or weather data sources.

8. AI Use Section

AI Tool Name	Version / Account Type	Specific Feature for Which the AI Tool Was Used	Value Addition (What I Added Beyond AI Output)
ChatGPT	Free Version	• Research on existing farm–vendor management tools • Identification of gaps and missing features in existing solutions • High-level backend system design guidance	I conducted a critical review of existing platforms and refined the project idea by identifying missing functionalities. I independently designed and implemented all backend workflows, APIs, and database schemas specific to the FarmVendor system.
ChatGPT	Premium Version (GPT-4)	• Guidance on ASP.NET Core Identity and JWT authentication • Debugging JWT authorization and Postman API testing issues • React component structure and dashboard UI design suggestions	I reviewed and selectively applied AI suggestions, manually implemented authentication and authorization logic, resolved environment-specific issues independently, and validated functionality through end-to-end testing and debugging.
ChatGPT	Premium Version (GPT-4)	• Backend query debugging and EF Core issues • DTO structure design guidance • Dispatch status update flow guidance	I adapted suggestions to match my project’s namespace, DbSet naming, Identity setup, and business rules. All DTOs, controllers, and service logic were designed and implemented manually and tested against real scenarios.

ChatGPT	Premium Version (GPT-4)	• Research brainstorming for forecasting and optimization modules • Comparison between Moving Average and ML-based forecasting approaches	I designed my own research novelty by implementing both baseline Moving Average and ML.NET SSA forecasting models, conducting comparisons using real dataset outputs, and integrating the chosen model into decision-support workflows.
ChatGPT	Premium Version (GPT-4)	• ML.NET SSA forecasting pipeline explanation and integration guidance	I independently implemented the ML.NET SSA forecasting engine, configured training windows and prediction horizons, integrated it with EF Core persistence, and validated forecasts using historical demand data.
ChatGPT	Premium Version (GPT-4)	• Optimization research and design guidance for dispatch planning	I designed and implemented a greedy allocation optimization technique to match farmer inventory with forecasted vendor demand, created custom services and APIs, and validated optimization results through scenario testing.
ChatGPT	Premium Version (GPT-4)	• Relationship scoring logic brainstorming	I implemented a custom relationship-scoring model based on fulfillment rate, on-time delivery, and cancellation rate. I integrated automatic score updates into dispatch confirmation workflows and exposed APIs for analytics.
ChatGPT	Premium Version (GPT-4)	• Full-stack integration and troubleshooting guidance	I resolved backend–frontend mismatches, corrected DTO binding errors, aligned API contracts, and stabilized forecasting, optimization, recommendation, and chat modules for final evaluation.

ChatGPT	Premium Version (GPT-4)	• Chat module design and SignalR integration guidance	I independently designed and implemented a real-time farmer–vendor chat system using SignalR, ensured secure JWT-based communication, fixed CORS and foreign-key issues, and tested full conversation workflows.
ChatGPT	Premium Version (GPT-4)	• UI/UX improvement suggestions (searchable dropdowns, navigation simplification)	I redesigned the frontend UX by replacing manual ID entry with searchable dropdowns, implemented a common navigation bar to reduce React code complexity, and improved responsiveness and usability across dashboards.
ChatGPT	Premium Version (GPT-4)	• Documentation, work-log structuring, and report organization guidance	I rewrote and refined all documentation using my actual implementation details, validated every section against the rubric, and ensured accuracy, consistency, and academic integrity throughout the final report.
ChatGPT	Premium Version (GPT-4)	• Guidance on structuring the final report sections, including Evaluation Techniques, Reflections, AI Use, and Work-Log consolidation • Assistance in refining academic language and ensuring rubric compliance	I independently authored all report content using my actual implementation details, validated technical accuracy against the codebase, ensured consistency across sections, and aligned the final document strictly with course requirements and grading criteria.
ChatGPT	Premium Version (GPT-4)	• Debugging support for forecast analytics charts not rendering correctly • Assistance identifying API endpoint mismatches and missing parameters	I manually traced backend APIs, verified database records, corrected frontend API calls, and fixed chart-loading logic. All fixes were implemented and tested independently to ensure accurate data visualization.

ChatGPT	Premium Version (GPT-4)	• Assistance in validating vendor recommendation logic, including forecast date alignment and inventory overlap issues	I analyzed recommendation outputs, refined scoring logic in FarmerRecommendationService.cs, validated results using real forecast data, and ensured recommendations change correctly based on farmer inventory and forecast inputs.
Mermaid (Live Editor)	Free Web Tool	• Creation of system architecture diagrams, database ERD, and workflow flowcharts for final report documentation	I manually defined the complete folder structure, entity relationships, and process flows based on the implemented system. All diagrams were verified against the actual codebase and database schema to ensure correctness and consistency.
ChatGPT	Premium Version (GPT-4)	• Assistance in generating Mermaid syntax for architecture, ERD, and flow diagrams	I reviewed and edited all Mermaid diagrams, ensured accurate mapping to controllers, services, entities, and frontend components, and adjusted diagram structure to match the real system design.
ChatGPT	Premium Version (GPT-4)	• Guidance on organizing work-log tables, consolidating Progress Reports 1–5, and ensuring realistic daily granularity	I reconstructed the full work-log using actual development activities, verified time distribution, mapped logs to implemented features, and ensured compliance with academic logging standards.
ChatGPT	Premium Version (GPT-4)	• Support in validating database architecture documentation and ERD explanations	I independently reviewed all entities, relationships, and foreign-key mappings, ensured normalization, and confirmed that the ERD accurately represents the implemented SQL Server schema.

Summary of AI use

Artificial Intelligence tools, primarily ChatGPT (Free and GPT-4 Premium) and Mermaid Live Editor, were utilized throughout the development of the FarmVendor system to support research, design, implementation, debugging, and documentation activities.

ChatGPT was initially used to explore existing farm–vendor management systems and identify gaps, which helped refine the project scope. It provided high-level guidance on system architecture, authentication (ASP.NET Identity with JWT), frontend design, and API structuring. Additionally, AI-assisted suggestions were used for troubleshooting issues related to backend queries, EF Core, API integration, and frontend rendering.

For advanced features, AI supported conceptual understanding of forecasting and optimization techniques. However, the Moving Average and ML.NET SSA forecasting models were independently implemented, configured, and validated using real historical data. Similarly, optimization logic, relationship scoring models, and recommendation systems were designed and developed manually based on domain-specific requirements.

In full-stack development, AI contributed to resolving integration issues, but all critical implementations—including DTO design, service-layer logic, API contracts, and database schema—were independently built, tested, and refined. The real-time chat module using SignalR was also fully implemented with manual handling of authentication, CORS, and communication workflows.

AI tools also supported UI/UX improvements and documentation structuring. However, all final designs, usability enhancements, and report content were carefully reviewed, rewritten, and aligned with actual implementation details to ensure accuracy and academic integrity.

Mermaid Live Editor was used for generating system diagrams, but all architectural flows, ERDs, and diagrams were manually defined and verified against the implemented codebase and database schema.

Overall, AI served as a supportive assistant for ideation and guidance, while the core system design, development, debugging, evaluation, and documentation were independently executed, ensuring originality, correctness, and practical applicability of the FarmVendor system

Appendix – Prompt History

Appendix X.1: Backend Development & Debugging Prompts

Prompt ID	AI Prompt Description
P-B1	Fix IQueryable implicit conversion error in EF Core Include query
P-B2	Why am I getting 400 error when posting dispatch
P-B3	How to design DTO for dispatch status update
P-B4	How to return demandRequestId in projection query
P-B5	Fix alignment issue in styling auth.css
P-B6	Fix dependency injection error when using RelationshipScoreService in ASP.NET Core controller

Appendix X.2: Forecasting, Optimization & Research Design Prompts

Prompt ID	AI Prompt Description
P-F1	How to implement a moving-average demand forecasting module in ASP.NET Core using EF Core data
P-F2	Design the DemandForecast model and API endpoints for generating and retrieving forecast results
P-F3	How to implement time-series forecasting using ML.NET SSA algorithm
P-F4	Create ML.NET pipeline for predicting future demand values from historical data
P-F5	How to integrate ML.NET forecasting into ASP.NET Core service layer

P-F6	Store forecast results generated by ML.NET in SQL Server using Entity Framework Core
P-F7	Design forecasting comparison between Moving Average and ML.NET SSA models
P-F8	How to export database data to CSV for research experiments and evaluation metrics

Appendix X.3: Frontend Analytics & Visualization Prompts

Prompt ID	AI Prompt Description
P-UI1	Create React chart component to display demand history and forecast predictions
P-UI2	Integrate forecast visualization charts into React analytics dashboard
P-UI3	Create React frontend views to display forecast results, optimization plans, and relationship statistics
P-UI4	How to integrate backend analytics endpoints with React dashboard pages using secure API calls
P-UI5	Check the endpoint error for chart forecast error that is not populating in forecastApi.js

Appendix X.4: Optimization & Recommendation System Prompts

Prompt ID	AI Prompt Description
P-O1	How to create a greedy optimization algorithm to allocate farmer inventory to vendor demand requests

P-O2	How to design RecommendedDispatchPlan table and DispatchOptimizationService for dispatch recommendation logic
P-O3	Optimization research and design for demand-driven allocation strategies
P-O4	Design recommendation system to suggest best vendors for farmers based on forecast and inventory data
P-O5	Implement ML-based or rule-based vendor recommendation logic using forecast demand and available inventory
P-O6	Create API endpoint to return top recommended vendors for a farmer dashboard
P-O7	Integrate vendor recommendation API into React dashboard and display results in a table UI

Appendix X.5: Real-Time Chat & System Integration Prompts

Prompt ID	AI Prompt Description
P-C1	How to implement real-time chat functionality using SignalR in ASP.NET Core with React frontend
P-C2	Fix foreign key constraint error when creating chat conversation between two users in EF Core
P-C3	How to correctly store and retrieve authenticated userId from JWT/localStorage in React application
P-C4	Resolve SignalR connection issues including CORS and WebSocket negotiation errors in ASP.NET Core
P-C5	Design chat system with conversation creation, message storage, and real-time updates using SignalR

P-C6	Replace manual user ID input with searchable dropdown using name/email in React UI
P-C7	Create backend API to search users (vendors/farmers) by name or email using Entity Framework Core
P-C8	Fix React module import/export errors and ensure proper API integration structure
P-C9	Create API endpoints for vendor and product lookup with search functionality in ASP.NET Core
P-C10	How to handle validation errors and payload structure issues in React form submission to backend APIs

Appendix X.6: Routing, Navigation & System Refinement Prompts

Prompt ID	AI Prompt Description
P-R1	How to implement nested routes with Outlet in React Router v6
P-R2	Fix routing and authentication issues causing unexpected redirects to login page in React Router
P-R3	Refactor React navigation into reusable sidebar components to reduce code duplication
P-R4	Improve overall system integration between frontend and backend by aligning API contracts and data flow

9. Work Date/Hours logs for student

Date	No. of Hours	Description of Work Done
13-Jan	2	Finalized the research idea for the FarmVendor application by clearly defining the problem statement and project scope.
13-Jan	1	Installed basic development tools including .NET SDK, Visual Studio Code, and SQL Server LocalDB.
14-Jan	4	Created a local database and initial React web application. Tested application execution in the browser and resolved multiple dependency installation issues.
15-Jan	4	Integrated ASP.NET Core Web API (.NET 8.0). Configured SQL Server LocalDB (FarmVendorDb), implemented ASP.NET Identity, defined Farmer and Vendor roles, and set up JWT authentication.
17-Jan	2	Tested authentication endpoints using Postman, including POST /api/auth/register and POST /api/auth/login.
20-Jan	0.5	Attended project idea re-evaluation meeting with instructor (Madam Priya) and refined the research direction.
21-Jan	2	Conducted initial research on existing mobile and web-based agricultural supply-chain applications.
22-Jan	2	Performed background research and literature review related to demand forecasting and supply-chain systems.
24-Jan	2	Conducted research on React integration with ASP.NET backend architecture.
25-Jan	2	Continued research proposal development, focusing on feature scope and system design.
26-Jan	1	Finalized the research proposal and prepared it for submission.
27-Jan	3	Reviewed submitted proposal feedback with instructor and made modifications to the data-collection and forecasting strategy.
29-Jan	4	Implemented JWT authorization configuration and secured API endpoints with role-based access control.

02-Feb	2	Moved GitHub repository to the root directory and cleaned unused project artifacts.
04-Feb	3	Created initial frontend sketches for the Farmer Dashboard and planned React component structure. Downloaded and configured required React dependencies.
05-Feb	2	Resolved installation and configuration issues related to Node.js and npm.
08-Feb	3	Completed login and registration UI design. Added dashboard CSS styling and modified application routing for Farmer and Vendor dashboards.
09-Feb	3	Uploaded modified proposal to GitHub. Updated progress report 1, linked working code screenshots, and verified GitHub repository structure.
23-Feb	8	Implemented Add Product Lot feature (React form + API POST)
		Created API endpoint POST /api/farmer/inventorylots using CreateInventoryLotDto and InventoryLotsController
		Final Midterm video recording
		Final Midterm report completion and submission
24-Feb	5	Implemented API endpoint POST /api/farmer/inventorylots using CreateInventoryLotDto and InventoryLotsController
28-Feb	3	Developed Dispatch module (frontend) and updated UI styles (FarmerDashboard.jsx, dashboard.css)
01-Mar	5	Implemented Dispatch module (backend) (DispatchController.cs, CreateDispatchDto.cs)
02-Mar	6	Fixed Create Dispatch - select request issue by aligning backend dispatch initialization and frontend handling
		Fixed Dispatched items update issue by adding/returning demandRequestId correctly in FarmerController.cs
		Implemented nested routing support by adding Outlet in RequireAuth and RequireRole.jsx
		Completed Farmer Products page frontend + backend (FarmerProducts.jsx, FarmerController.cs / ProductsController.cs, InventoryLotDto.cs, UpdateInventoryLotDto.cs)
		Completed Farmer Requests page frontend + backend (FarmerController.cs, FarmerDemandRequestRowDto.cs, FarmerRequests.jsx)
		Completed Farmer Dispatch page frontend + backend (DispatchRowDto.cs, UpdateDispatchStatusDto.cs, FarmerDispatchController.cs, FarmerDispatch.jsx)

03-Mar	6	Implemented inventory lot creation and stock validation logic to support dispatch planning from farmer inventory against vendor demand requests.
		Developed Vendor Dashboard functionality for Create Demand Request and integrated frontend actions with backend API endpoints.
		Implemented Incoming Dispatches view and Confirm Delivery workflow, including updates to dispatch delivery status and related demand request status.
05-Mar	5	Resolved full-stack integration issues involving DTO mismatches, dropdown binding problems, missing response fields, and EF Core query typing issues.
		Updated VendorDashboard.jsx and integrated modal-based demand request creation with VendorDemandRequestsController.cs.
		Added backend support for CSV/data extraction to prepare for forecasting and experimental analysis.
07-Mar	6	Began implementation of a moving-average demand forecasting baseline in .NET as the first forecasting approach for the research module.
		Created the DemandForecast model, supporting DTOs, and required database updates for forecast storage.
		Completed baseline forecasting workflow and tested forecast generation successfully through API endpoints.
		Designed the optimization module structure by defining the optimization result table and dispatch recommendation flow.
		Implemented DispatchOptimizationService.cs using greedy baseline allocation logic based on forecast demand and available farmer inventory.
		Created optimization API endpoints for generating and retrieving recommended dispatch plans.
08-Mar	3	Conducted research and design for the optimization component, focusing on demand-driven farmer-to-vendor dispatch decision logic using forecast and inventory data.
		Refined the optimization formulation to support the research novelty component of the project.
09-Mar	7	Implemented the optimization module and connected it with forecast and inventory data to generate recommended dispatch plans.
		Implemented relationship score tracking between Farmer and Vendor pairs based on dispatch outcomes.

		Added automatic score updates for Delivered and Cancelled dispatch events.
		Computed relationship score using measurable factors including on-time delivery rate, fulfillment rate, and cancellation rate.
		Exposed backend API endpoints for viewing relationship statistics.
		Added and updated files required for relationship scoring, including RelationshipStat, RelationshipScoreRowDto, RelationshipScoreService, and RelationshipStatsController.
		Integrated score update logic into VendorDispatchesController confirm delivery flow and outlined remaining frontend analytics views for forecast, optimization, and relationship endpoints.
		Developed Progress report3
10-Mar	6	Individual checking
		changing research baseline model to ML related model which focus on ML.NET's forecasting task
		Researched machine learning-based forecasting approaches and identified ML.NET SSA (Singular Spectrum Analysis) time series forecasting as a more suitable ML-driven model for demand prediction.
		Updated research direction by replacing the baseline-only approach with an ML-focused forecasting implementation using ML.NET TimeSeries API and refactored the forecasting workflow in DemandForecastService.cs
		Studied ML.NET forecasting pipeline structure including SSA training window, horizon prediction, model retraining, and demand sequence preparation from historical demand request data, preparing the implementation structure for MLDemandForecastEngine.cs
		Studied ML.NET forecasting pipeline structure including SSA training window, horizon prediction, model retraining, and demand sequence preparation from historical demand request data.

		Refactored forecasting architecture to support multiple forecasting models (Moving Average + ML.NET SSA) and updated related models including DemandForecast.cs, GenerateForecastDto.cs, and DemandForecastRowDto.cs
11-Mar	6	Implemented the ML.NET forecasting engine inside the backend service layer.
		Created MLDemandForecastEngine.cs to train ML.NET models using historical demand request quantities stored in the database.
		Implemented time-series training pipeline using MLContext and SSA forecasting trainer to predict future vendor demand quantities inside MLDemandForecastEngine.cs
		Integrated ML forecast generation into the existing forecasting workflow by updating DemandForecastService.cs so prediction results can be stored in the DemandForecast table
12-Mar	5	Tested the ML.NET forecasting pipeline using historical demand request data inserted into the database and validated forecasting output.
		Verified training data extraction, forecast horizon configuration, and prediction output structure in MLDemandForecastEngine.cs
		Added model identification field (ModelName) to distinguish forecast outputs from MovingAverage vs MLNET_SSA models and updated related structures in DemandForecast.cs and DemandForecastRowDto.cs
13-Mar	4	Developed backend API endpoints to support ML forecasting execution and retrieval through the forecasting module
		Implemented API logic in ForecastController.cs for generating ML forecasts and storing prediction results in the database.
		Ensured compatibility between backend forecasting APIs and frontend analytics components by updating request/response DTOs such as GenerateForecastDto.cs and DemandForecastRowDto.cs

		Developed frontend analytics dashboard for displaying forecasting results and model comparison functionality.
16-Mar	6	Implemented Vendor Forecast Analytics page in VendorForecast.jsx including forecast generation form and model comparison module between Moving Average and ML.NET models.
		Integrated forecasting APIs with React frontend components using secure token-based authentication through forecastApi.js and existing route protection logic
		Added forecast visualization charts to the analytics dashboard to represent demand history and forecast trends visually.
		Implemented chart component ForecastLineChart.jsx to display historical demand data alongside forecast predictions.
		Integrated chart component into both analytics pages VendorForecast.jsx and FarmerForecast.jsx to provide graphical interpretation of forecasting results.
		Began implementation of a real-time communication feature between farmers and vendors within the FarmVendor system.
		Designed the chat module architecture and developed frontend messaging components FarmerChat.jsx and VendorChat.jsx.
		Updated authentication logic in Login.jsx to store userId in localStorage to support message sender identification for chat communication.
		Prepared frontend system structure for integrating backend chat APIs and enabling future real-time messaging functionality between farmers and vendors.
		Completed Progress report 4 and uploaded to blackboard and checked in to github repository.
18-Mar	4	Implemented and tested the farmer–vendor chat module in the FarmVendor system. Added chat conversation creation and message flow between farmer and vendor users.
		Fixed login-related user identification issues by updating Login.jsx to store userId in localStorage correctly.
		Resolved chat database and foreign key issues by verifying valid farmer and vendor IDs from theAspNetUsers table and testing conversation creation successfully.

		Fixed SignalR/CORS connection issues by updating backend configuration in Program.cs and retesting real-time chat connection.
		Improved chat usability by replacing manual user ID entry with searchable selection of existing farmers and vendors using name/email search. Updated ChatController.cs, chatApi.js, FarmerChat.jsx, and VendorChat.jsx.
		Tested the full chat workflow from both farmer and vendor sides, including conversation opening, message display, and sending messages in a more user-friendly interface.
19-Mar	3	Enhanced user experience of forecasting interface by replacing manual userId inputs with searchable dropdowns for vendor and product selection in FarmerForecast.jsx. Developed backend APIs for vendor and product lookup with search functionality in DemandForecastController.cs (endpoints: /vendors, /products). Updated API integration in forecastApi.js to support dynamic search queries and data fetching. Improved validation handling and request payload structure in forecasting workflow to correctly include vendorId and prevent API errors.
		Updated the navigation bar by creating a separate common component file. Refactored FarmerSidebar.jsx, VendorSidebar.jsx, and App.jsx to import the shared component. All related .jsx files were updated to include the new import structure and integrate the sidebar elements.
20-Mar	3	Implemented ML-based vendor recommendation feature to identify top vendors for dispatching based on forecast demand, inventory availability, and historical request data. Developed backend logic in FarmerRecommendationService.cs and exposed API via FarmerController.cs (/recommended-vendors). Integrated frontend in FarmerDashboard.jsx to display top recommended vendors dynamically.
22-Mar	3	Populated the system with additional historical demand data to support forecasting experiments and verified that ML.NET SSA forecast rows were generated successfully in the DemandForecast table.

23-Mar	5	Debugged why recommended vendors and charts were not appearing by checking forecast dates, inventory overlap, API responses, and endpoint usage. Fixed forecasting chart integration by correcting the chart API endpoint in forecastApi.js, updating chart-loading logic to use the selected forecasting model, and refining DemandForecastService.cs chart data retrieval to return recent historical values together with matching forecast values from the selected date onward. Continued testing of farmer-specific recommended vendors and confirmed that recommendation results change based on the logged-in farmer's inventory and forecast matching data.
		Progress report 5 submission
24-Mar	5	Enhanced forecasting dashboard UI with advanced settings panel including vendor, product, date, model, and horizon selection.
		Implemented default parameter auto-loading for vendor, product, and forecast date to improve usability.
		Improved error handling for forecast generation and API failures in FarmerForecast.jsx.
25-Mar	4	Implemented CSV export functionality for forecast results.
		Added Export button to forecasting dashboard and generated downloadable CSV file including vendor name, product name, forecast date, quantity, model, and created date.
		Ensured removal of internal IDs from export and replaced with user-friendly names.
26-Mar	6	Upgraded forecast visualization to support multi-vendor comparison charts.
		Developed MultiVendorForecastLineChart.jsx to display up to 4 vendors in a single graph with distinct colors and legends.
		Integrated vendor selection logic allowing dynamic selection of top vendors from forecast results.
27-Mar	5	Implemented backend API enhancement for chart data retrieval per vendor and product.
		Added filtering logic in DemandForecastController.cs to support vendorId, productId, and forecastDate.
		Improved chart API response to include both historical demand and forecast data series.

28-Mar	4	Enhanced frontend chart rendering logic by fixing dataset mapping and handling missing/null values.
		Resolved chart data alignment issues between historical and forecast series.
		Improved chart readability with legend labels and consistent formatting.
29-Mar	6	Implemented vendor filtering logic based on farmer inventory.
		Restricted vendors to only those having demand requests matching farmer's available products.
		Updated backend logic in DemandForecastService.cs and FarmerController.cs to support real-world filtering conditions.
30-Mar	5	Improved forecasting data realism by generating additional historical demand data.
		Enhanced SQL scripts using CTE logic to simulate multi-period demand patterns.
		Validated ML.NET SSA forecasting output with expanded dataset.
31-Mar	4	Enhanced dashboard UX by displaying selected vendor name and product name directly on the chart section.
		Removed redundant UI elements such as "Available Model Names" for cleaner interface.
01-Apr	7	Improved layout alignment and readability.
		Integrated Unsplash API for product image selection in inventory management.
		Implemented image search and selection within Add Product Lot modal.
02-Apr	6	Stored image metadata (ImageUrl, ImageThumbUrl, PhotographerName) in Product entity and displayed images in UI.
		Enhanced product selection UX by displaying images alongside product names in dropdowns.
		Improved user experience by allowing visual identification of products.
		Added fallback handling for missing images.
03-Apr	8	Improved real-time chat module stability and authentication handling.

		Resolved SignalR WebSocket connection issues caused by missing JWT token.
		Updated ChatHub configuration and frontend connection logic to include access token.
04-Apr	6	Enhanced chat UI by replacing user IDs with display names in conversation lists.
		Implemented backend join queries in ChatController.cs to return vendor and farmer names.
		Improved conversation selection usability with searchable dropdowns.
05-Apr	7	Optimized API performance by reducing redundant API calls in forecasting and chat modules.
		Implemented caching and minimized repeated fetch requests in React components.
		Improved loading times for dashboard and analytics pages.
06-Apr	8	Performed full system integration testing across all modules including forecasting, dispatch, recommendation, chat, and inventory.
		Verified role-based access control for Farmer and Vendor workflows.
		Fixed minor bugs related to DTO mismatches and API responses.
07-Apr	7	Conducted final UI/UX refinements across the application.
		Improved dashboard layout to resemble Power BI style with better spacing and card alignment.
		Enhanced consistency of buttons, dropdowns, and tables.
		Final report modifications
08-Apr	8	Final testing and validation of ML.NET SSA forecasting accuracy and system outputs.
		Verified correctness of forecast results, recommendation outputs, and chart visualizations.
		Prepared final screenshots, documentation, and report content.
		Final report modifications
		Completed final report writing including architecture diagrams, implemented features, evaluation techniques, and AI usage section.

		Prepared appendix with prompt history and finalized all submission materials.
		Prepared presentation file

10. Concluding Remarks

The FarmVendor project successfully achieved its objective of designing and implementing a data-driven agricultural supply coordination system that connects farmers and vendors through an intelligent, user-friendly platform.

The system evolved from a basic CRUD-based application into a comprehensive decision-support system incorporating advanced features such as demand forecasting, vendor recommendation, real-time communication, and interactive analytics dashboards. A key contribution of this project is the integration of machine learning-based forecasting using ML.NET Singular Spectrum Analysis (SSA), which enables more accurate and future-oriented demand predictions compared to traditional statistical methods.

The project also demonstrates the practical application of full-stack development using ASP.NET Core, React, SQL Server, and modern APIs such as SignalR and Unsplash. The inclusion of role-based authentication, dynamic dashboards, and real-time chat further enhances usability and system effectiveness in real-world agricultural scenarios.

Through iterative development, testing, and evaluation, several design improvements were made, including enhanced data generation strategies, optimized API integration, and improved user experience through intuitive UI components and visualization tools. The implementation of multi-vendor comparison charts and exportable analytics further strengthens the system's analytical capabilities.

Overall, the FarmVendor system demonstrates a scalable and extensible architecture that can be further expanded to support real-world agricultural supply chain optimization. Future enhancements may include advanced machine learning models, mobile application support, and integration with external market data sources.

11.References

AgriWebb. (2024). *AgriWebb farm management software* [Mobile application]. <https://www.agriwebb.com>

FarmLogs. (2024). *FarmLogs: Farm management and record keeping* [Mobile application]. <https://farmlogs.com>

CropIn Technology Solutions. (2024). *CropIn smart farm management platform* [Mobile application]. <https://www.cropin.com>

AgriDigital. (2024). *AgriDigital grain supply chain platform* [Web-based platform]. <https://www.agridigital.io>

Microsoft. (2024). *ML.NET Documentation*. <https://learn.microsoft.com/en-us/dotnet/machine-learning/>

Microsoft. (2024). *ASP.NET Core Documentation*. <https://learn.microsoft.com/en-us/aspnet/core/>

React Documentation. (2024). <https://react.dev/>

Chart.js Documentation. (2024). <https://www.chartjs.org/docs/latest/>

Unsplash API Documentation. (2024). <https://unsplash.com/documentation>

Microsoft. (2024). *SignalR Documentation*. <https://learn.microsoft.com/en-us/aspnet/core/signalr/>

SQL Server LocalDB Documentation. (2024). <https://learn.microsoft.com/en-us/sql/database-engine/configure-windows/sql-server-express-localdb>

12. Appendix A: Installation Guide

This section provides step-by-step instructions to set up and run the FarmVendor application locally.

1. Clone the Repository

```
git clone https://github.com/maneeshaprs-rgb/W26_4495_S3_ManeeshaE.git
cd W26_4495_S3_ManeeshaE
```

2. Backend Setup (ASP.NET Core API)

```
cd FarmVendor/FarmVendor.Api
```

Install Dependencies

```
dotnet restore
```

Install EF Core Tool (if not installed)

```
dotnet tool install --global dotnet-ef
```

Install ML.NET Packages

```
dotnet add package Microsoft.ML
dotnet add package Microsoft.ML.TimeSeries
```

3. Database Setup

Run Migrations

```
dotnet ef migrations add AddProductImageFields
dotnet ef database update
```

(If migrations already exist, only run database update)

4. Run Backend Server

```
dotnet build
dotnet run
```

Backend will run at:

`http://localhost:5136`

5. Frontend Setup (React + Vite)

```
cd ../../farmvendor-web
```

```
npm install
```

Install Required Packages

```
npm install react-chartjs-2 chart.js
```

```
npm install @microsoft/signalr
```

```
npm install react-select
```

6. Configure Environment Variables

Create `.env` file inside `farmvendor-web`:

```
VITE_API_URL=http://localhost:5136
```

7. Run Frontend

```
npm run dev
```

Application will run at:

`http://localhost:5173/`

8. Database Connection (Optional View)

- Install SQL Server Extension in VS Code
- Connect using:
 - Server: `(localdb)\MSSQLLocalDB`
 - Authentication: Windows
 - Trust Certificate: Yes

9. Test Credentials

Farmer

- Email: farmer1@test.com
- Password: Test@123

Vendor

- Email: vendor1@test.com
- Password: Test@123

10. Sample Test Parameters (Forecasting)

- Vendor ID: 2660cd91-4a85-4083-b925-ceceab6b8529
- Product ID: 4
- Forecast Date: 2026-03-12

11. Run Sample SQL Scripts

Location:

ReportsAndDocuments/sql_queries/

Run:

- Product.sql
- Product2.sql

13. Appendix B: User Guide

This section explains how end users (Farmers and Vendors) can use the FarmVendor system.

1. Login / Registration

- Open: <http://localhost:5173>
- Register as:
- Farmer OR Vendor
- Login using credentials
- Register interface

The screenshot shows the FarmVendor Login interface. On the left, a dark green sidebar contains the FarmVendor logo (FV) and the text 'Farmer & Vendor Portal'. Below this, it says 'Welcome back' and 'Sign in to manage your stock, requests, and dispatch planning.' followed by three bullet points: 'Role-based access (Farmer / Vendor)', 'Track inventory and vendor demand', and 'Plan dispatch and deliveries'. On the right, the 'Login' section has a light gray box with the text 'Use your email and password to continue.' Below this are input fields for 'Email' (containing 'farmer1@test.com') and 'Password' (containing 'Your password'). A green 'Login' button is at the bottom of the form. Below the button, it says 'Don't have an account? [Register](#)'.

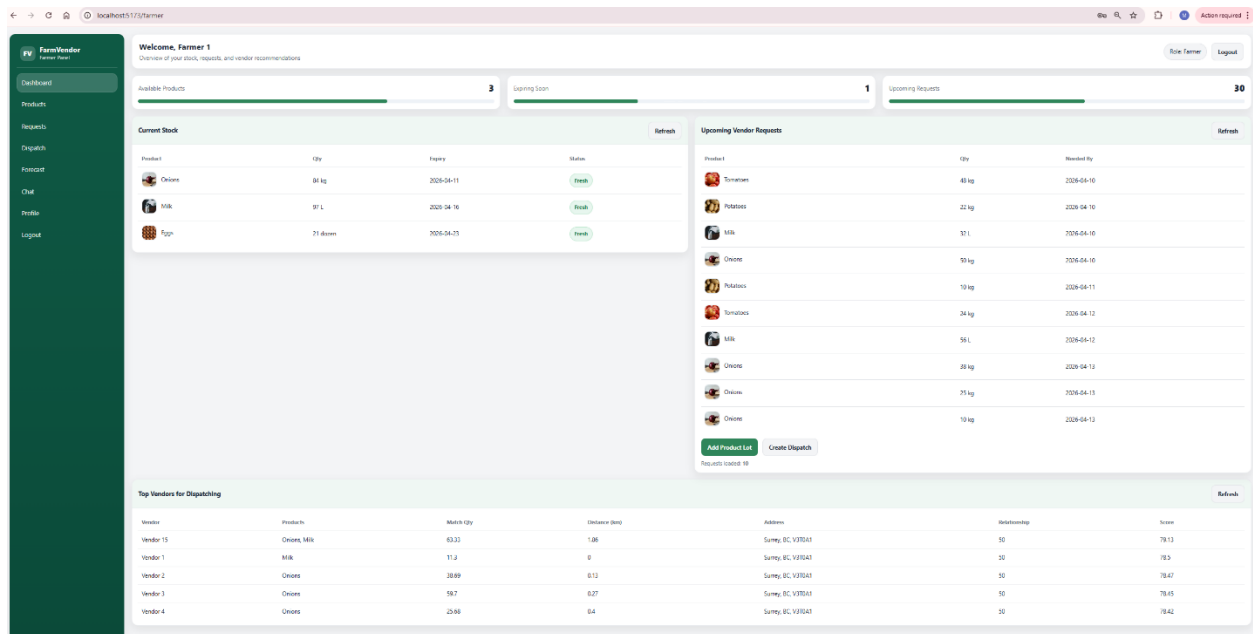
- Login interface

The screenshot shows the FarmVendor Register interface. On the left, a dark green sidebar contains the FarmVendor logo (FV) and the text 'Farmer & Vendor Portal'. Below this, it says 'Create your account' and 'Register as a Farmer or Vendor to start using the platform.' followed by three bullet points: 'Create demand requests (Vendor)', 'Add product lots and plan dispatch (Farmer)', and 'Secure login with role-based access'. On the right, the 'Register' section has a light gray box with the text 'Fill the form below to create an account.' Below this are input fields for 'Display Name' (containing 'Maneesha'), 'Email' (containing 'vendor1@test.com'), and 'Password' (containing 'Min 6 characters'). There is a 'Role' dropdown menu with 'Farmer' selected. A green 'Register' button is at the bottom of the form. Below the button, it says 'Already have an account? [Login](#)'. At the very bottom, there is a light gray box with the text 'Mock mode is OFF. (Turn it OFF when backend runs.)'.

2. Farmer Functionalities

Dashboard

- View inventory summary
 - View recommended vendors
-
- FarmerDashboard



Inventory Management

- Add product lot
- Select product (with image from Unsplash)
- Enter quantity and expiry date
- FarmerDashboard

Upcoming Vendor Requests			Refresh
Product	Qty	Needed By	
 Apple	60 kg	2026-04-09	
 Tomatoes	48 kg	2026-04-10	
 Potatoes	22 kg	2026-04-10	
 Milk	32 L	2026-04-10	
 Onions	50 kg	2026-04-10	
 Potatoes	10 kg	2026-04-11	
 Tomatoes	24 kg	2026-04-12	
 Milk	56 L	2026-04-12	
 Onions	38 kg	2026-04-13	
 Onions	25 kg	2026-04-13	
Add Product Lot Create Dispatch			Requests loaded: 10

Welcome, Farmer 1

Overview of your stock, requests, and vendor recommendations

Available Products

3

Spring Loan

1

Upcoming Requests

Current Stock

Refresh

Upcoming Vendor Requests

Product	Qty	Expiry	Status
 Onions	84 kg	2026-04-11	New Entry
 Milk	87 L	2026-04-16	Track
 Eggs	21 dozen	2026-04-25	Track

Add Product Lot

Product

-- Select --

New Product

Quantity

Unit

kg / L / dozen

Expiry Date (optional)

yyyy-mm-dd

Cancel

Save Lot

Product	Qty	Requested By
 Apple	80 kg	2026-04-08
 Tomatoes	40 kg	2026-04-10
 Avocados	22 kg	2026-04-10
	32 L	2026-04-10
	30 kg	2026-04-10
	19 kg	2026-04-11
	34 kg	2026-04-12
	55 L	2026-04-12
	38 kg	2026-04-13
	23 kg	2026-04-13

Add Product Lot

Create Dispatch

Requests loaded: 10

Add Product Lot

Product

-- Select --

+

Close

New Product Name

e.g., Carrots

Category (optional)

e.g., Vegetables

Default Unit

e.g., kg / L / dozen

Find Images

Save Product

Cancel

Quantity

e.g., 50

Unit

kg / L / dozen

Expiry Date (optional)

yyyy-mm-dd

Cancel

Save Lot

Distance (km)

Welcome, Farmer 1

Overview of your stock, requests, and vendor recommendations

Role Farmer

Logout

Available Products

3

Current Stock

Product	Qty	Expiry
Onions	84 kg	2026-04-11
Milk	97 L	2026-04-16
Eggs	21 dozen	2026-04-23

Upcoming Requests

1

31

Refresh

Top Vendors for Dispatching

Vendor	Products	Market Qty
Vendor 15	Onions, Milk	63
Vendor 1	Milk	11.69
Vendor 2	Onions	38.55
Vendor 3	Onions	58.57
Vendor 4	Onions	25.53

31

Refresh

Add Product Lot

Product

-- Select --

+

Close

New Product Name

Bell pepper

Category (optional)

e.g., Vegetables

Default unit

e.g., kg / L / dozens

Find Images

Select

Select

Select

Select

Select

Select

Save Product

Cancel

Quantity

e.g., 50

Unit

kg / L / dozen

Expiry Date (optional)

yyyy-mm-dd

Cancel

Save Lot

Add Product Lot

!

Please check this form

×

A product with this name already exists.

Product

-- Select --

+ Close

New Product Name

Bell pepper

Category (optional)

Vegetable

Default Unit

Kg

Find Images

Selected image

Photo by Rens D on Unsplash

Select

Select

Select

Select

Select

Select

Select

Select

Save Product

Cancel

Quantity

e.g., 50

Unit

kg / L / dozen

Expiry Date (optional)

yyyy-mm-dd

Cancel

Save Lot

Add Product Lot

Product

-- Select --

+ Close

New Product Name

Green Onion

Category (optional)

Vegetable

Default Unit

Kg

Find Images

Selected image

Photo by Christopher Previte on Unsplash

Select

Select

Select

Select

Select

Select

Select

Select

Save Product

Cancel

Quantity

e.g., 50

Unit

kg / L / dozen

Expiry Date (optional)

yyyy-mm-dd

Cancel

Save Lot

Page | 169

Add Product Lot



Green Onion

Kg • Vegetable

+ New Product

Quantity

69

Unit

Kg

Expiry Date (optional)

2026-04-16

Cancel

Save Lot

Add Product Lot

Requests loaded: 10

Current Stock				Refresh
Product	Qty	Expiry	Status	
 Onions	84 kg	2026-04-11	Near Expiry	
 Milk	97 L	2026-04-16	Fresh	
 Green Onion	69 Kg	2026-04-16	Fresh	
 Eggs	21 dozen	2026-04-23	Fresh	

- Product interface

The screenshot displays the 'Products' section of the FarmVendor application. The interface is divided into a sidebar, a main content area with two panels, and a top navigation bar.

Sidebar: Contains links for Dashboard, Products (active), Requests, Dispatch, Forecast, Chat, Profile, and Logout.

Top Bar: Shows 'Welcome, Farmer 1' and a 'Logout' button. A notification 'Action required!' is visible on the right.

Main Content Area:

- Left Panel (Active Product Catalog):** A table listing various products with columns for Name, Default Unit, and Status. Each row has an 'Action' button.

Name	Default Unit	Status
Apple	kg	Active
Banana	Bunch	Active
Bell Pepper	kg	Active
Broccoli	kg	Active
Carrot	kg	Active
Cauliflower	kg	Active
Cherry	kg	Active
Clementine	Bunch	Active
Cucumber	kg	Active
Egg	Dozen	Active
Garlic	kg	Active
Grape	kg	Active
Green Onion	kg	Active
Milk	L	Active
Mushroom	kg	Active
Onion	kg	Active
Orange	kg	Active
Parsley	Bunch	Active
Pine	kg	Active
Potatoes	kg	Active
Sprouts	kg	Active
Strawberry	kg	Active
Tomatoes	kg	Active
- Right Panel (My Inventory List):** A table showing inventory items with columns for Product, Qty, Expiry, and Actions. It includes 'Add Item' and 'Refunds' buttons.

Product	Qty	Expiry	Actions
Green Onion	45 kg	2020-04-16	<button>Left</button> <button>Delete</button>
Onions	54 kg	2020-04-11	<button>Left</button> <button>Delete</button>
Milk	31 L	2020-04-16	<button>Left</button> <button>Delete</button>
Eggs	21 Dozen	2020-04-23	<button>Left</button> <button>Delete</button>

Demand Requests

- View vendor requests
- Track status
- VendorDashboard

The screenshot shows the 'FarmVendor' dashboard for 'Vendor 1'. The dashboard is divided into several sections:

- Low Stock:** A table listing items with low stock levels. It includes columns for Item, Level, and Status. Items listed are Eggs (12 Dozens, Low) and Tomatoes (8 kg, Critical). A 'Create Demand Request' button is visible below the table.
- Incoming Deliveries:** A table showing incoming deliveries with columns for Dispatch, Product, Qty, Status, Dispatch Date, and Action. A delivery for Potatoes (56 kg, In Transit, 2026-04-07) is shown with a 'Confirm Delivery' button.
- My Open Demand Requests:** A table showing open demand requests with columns for Product, Qty, Needed By, and Status. Requests for Eggs (80 kg, 2026-04-09, Open), Milk (10 L, 2026-04-21, Open), and Milk (15 L, 2026-04-14, Open) are listed.

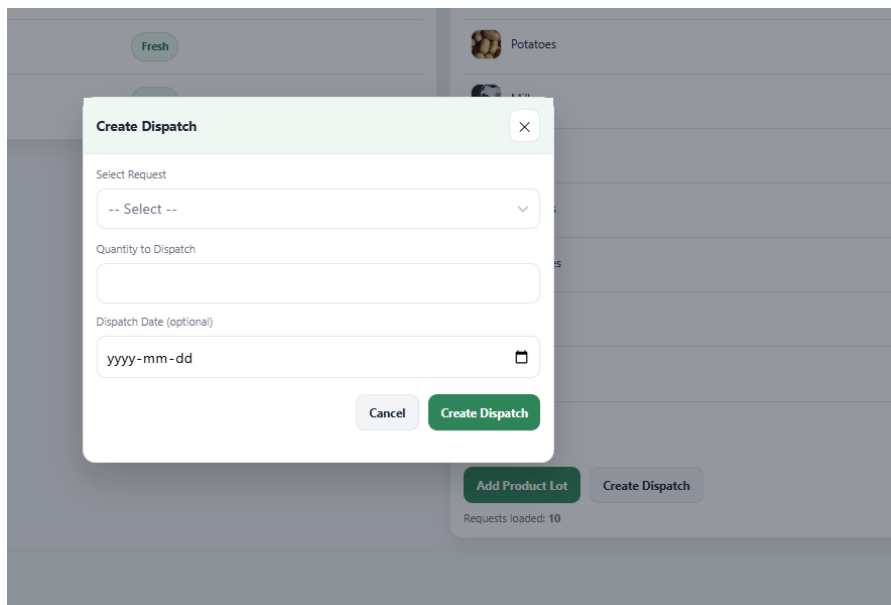
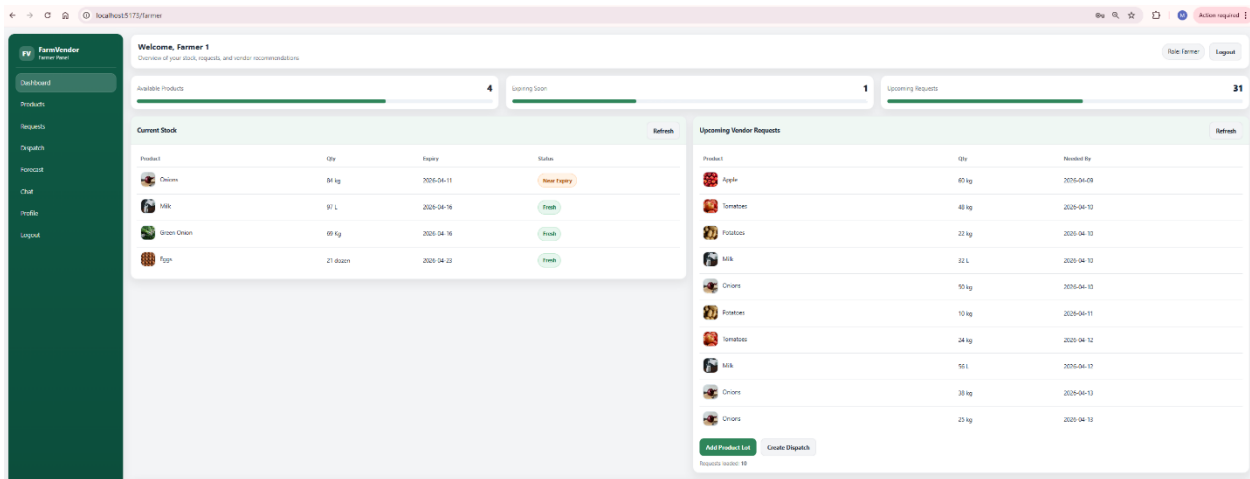
The screenshot shows the 'Create Demand Request' modal form overlaid on the dashboard. The form has the following fields:

- Product:** A dropdown menu with '-- Select --'.
- Quantity:** A text input field with 'e.g. 25'.
- Unit:** A dropdown menu with 'kg / L / dozen'.
- Needed By:** A date input field with the format 'yyyy-mm-dd'.

At the bottom of the modal, there are 'Cancel' and 'Create Request' buttons.

Dispatch Module

- Create dispatch for vendor requests
- Update delivery status
- FarmerDashboard



Create Dispatch ✕

Select Request

-- Select --

-  **Apple**
60 kg • Needed: 2026-04-09
-  **Tomatoes**
48 kg • Needed: 2026-04-10
-  **Potatoes**
22 kg • Needed: 2026-04-10
-  **Milk**
32 L • Needed: 2026-04-10
-  **Onions**
50 kg • Needed: 2026-04-10
-  **Potatoes**

Requests loaded: 10

Create Dispatch ✕

 **Please check this form** ✕
Not enough stock. Available: 0.00, Requested dispatch: 48

Select Request

 **Tomatoes**
48 kg • Needed: 2026-04-10 ✕ ▼

Quantity to Dispatch

48

Dispatch Date (optional)

2026-04-16 

Cancel Create Dispatch

Requests loaded: 10

- Dispatch interface

Dispatch
Track and update delivery status

Filter: All Refresh

My Dispatches (6)

ID	Product	Qty	Vendor	Dispatch Date	Status	Action
#21	Onions	50 kg	Vendor 15	2026-04-16	Planned	<button>Update Status</button>
#2	Potatoes	56 kg	Vendor 1	2026-04-07	In Transit	<button>Update Status</button>
#6	Potatoes	58 kg	Vendor 4	2026-03-30	In Transit	<button>Update Status</button>
#18	Eggs	30 dozen	Vendor 3	2026-03-13	Delivered	<button>Update Status</button>
#3	Onions	73 kg	Vendor 7	2026-03-16	In Transit	<button>Update Status</button>
#26	Onions	13 kg	Vendor 2	2026-03-22	In Transit	<button>Update Status</button>

Status flow: Planned → In Transit → Delivered (or Cancelled)

Dispatch
Track and update delivery status

Filter: All Refresh

My Dispatches (6)

ID	Product	Qty	Vendor	Dispatch Date	Status	Action
#21	Onions	50 kg	Vendor 15	2026-04-16	Planned	<button>Update Status</button>
#2	Potatoes	56 kg	Vendor 1	2026-04-07	In Transit	<button>Update Status</button>
#6	Potatoes	58 kg	Vendor 4	2026-03-30	In Transit	<button>Update Status</button>
#18	Eggs	30 dozen	Vendor 3	2026-03-13	Delivered	<button>Update Status</button>
#3	Onions	73 kg	Vendor 7	2026-03-16	In Transit	<button>Update Status</button>
#26	Onions	13 kg	Vendor 2	2026-03-22	In Transit	<button>Update Status</button>

Status flow: Planned → In Transit → Delivered (or Cancelled)

Update Dispatch Status

Dispatch: #21 — Onions (50 kg)

New Status

Planned

Cancel Save

Note: Delivered/Cancelled cannot be changed back by backend rules.

Qty	Vendor	Dispatch Date	Status	Action
50 kg	Vendor 15	2026-04-16	Planned	<button>Update Status</button>
56 kg	Vendor 1	2026-04-07	In Transit	<button>Update Status</button>
58 kg	Vendor 4	2026-03-30	In Transit	<button>Update Status</button>
30 dozen	Vendor 3	2026-03-13	Delivered	<button>Update Status</button>
73 kg	Vendor 7	2026-03-16	In Transit	<button>Update Status</button>
13 kg	Vendor 2		In Transit	<button>Update Status</button>

Update Dispatch Status

Dispatch: #31 — Onions (50 kg)

New Status

Delivered

Planned

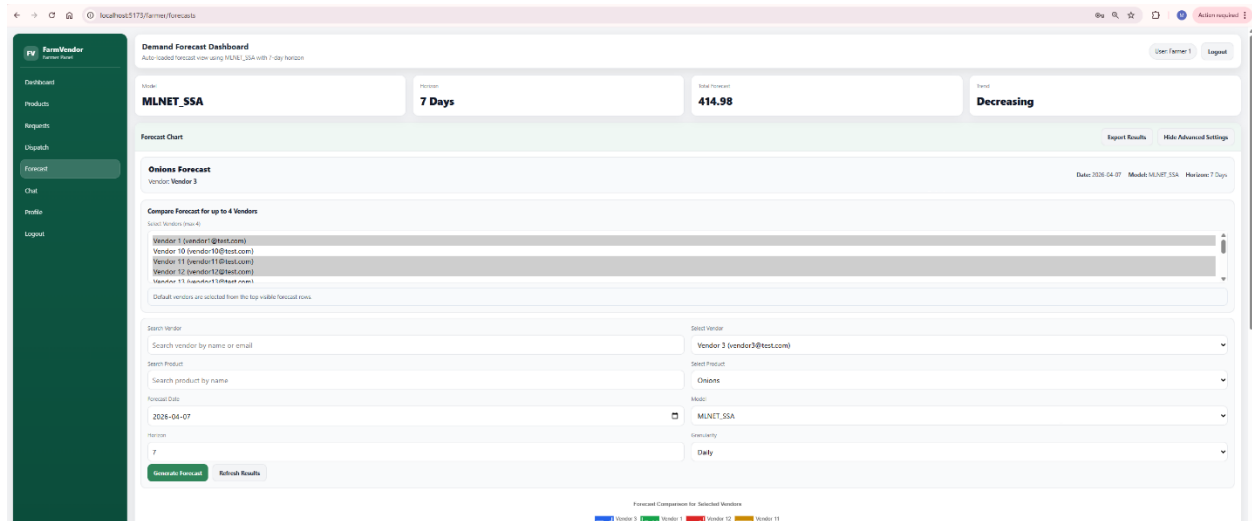
In Transit

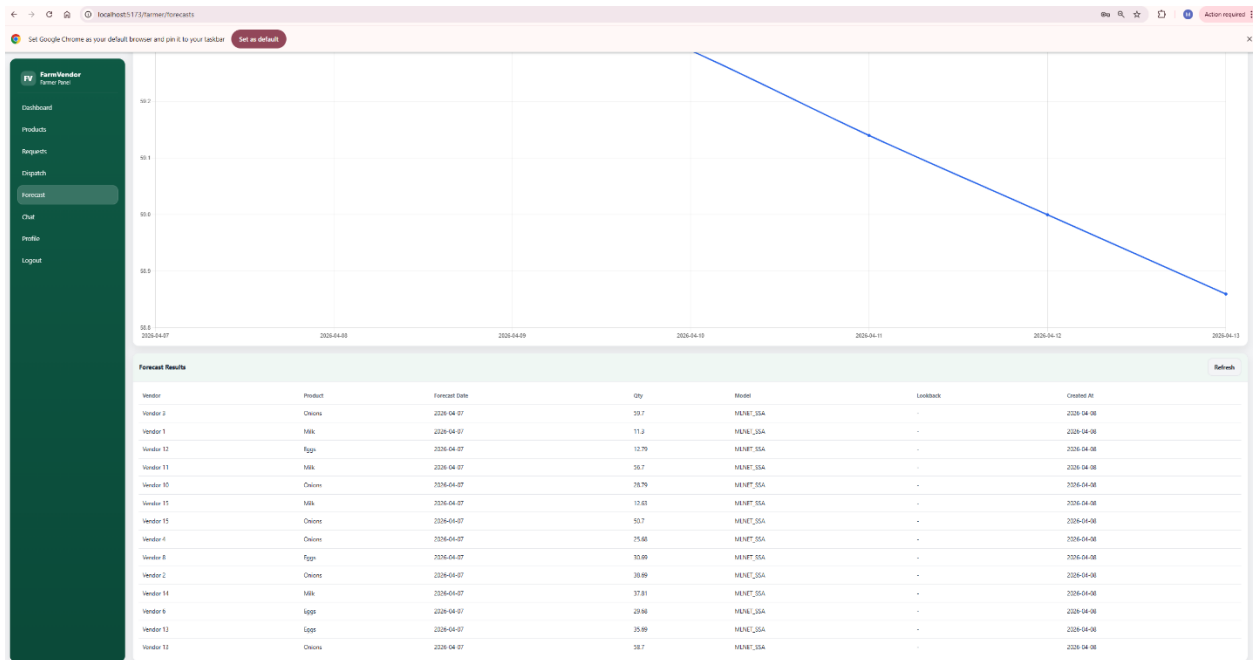
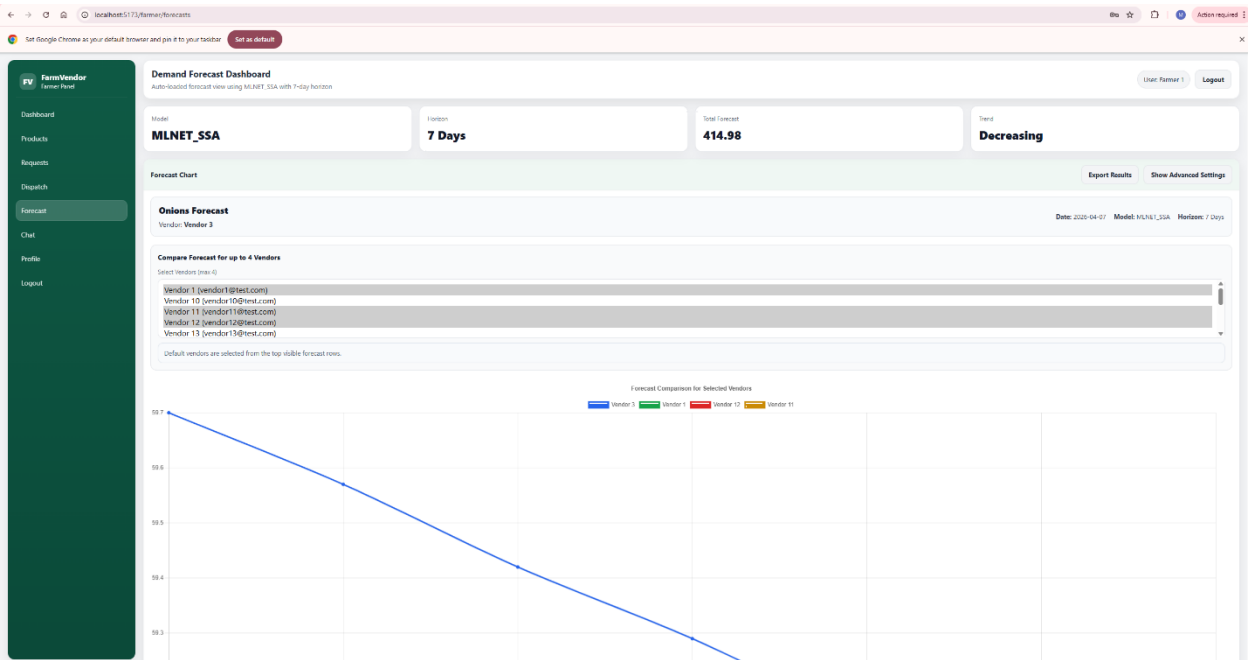
Delivered

Cancelled

Forecast Dashboard

- Select vendor, product, and date
- Choose model (Moving Average / ML.NET SSA)
- View:
 - Forecast chart
 - Multi-vendor comparison
- Export results as CSV





Chat Module

- Select vendor
- Start conversation
- Send and receive real-time messages

- farmerDashboard

Upcoming Vendor Requests

Refresh

Product	Qty	Needed By
 Tomatoes	48 kg	2026-04-10

Vendor Details

 **Tomatoes**
48 kg

Vendor: Vendor 3
Email: vendor3@test.com
Distance: 0.27 km
Address: Surrey, BC, V3T0A1
Needed By: 2026-04-10

Chat with Vendor 3

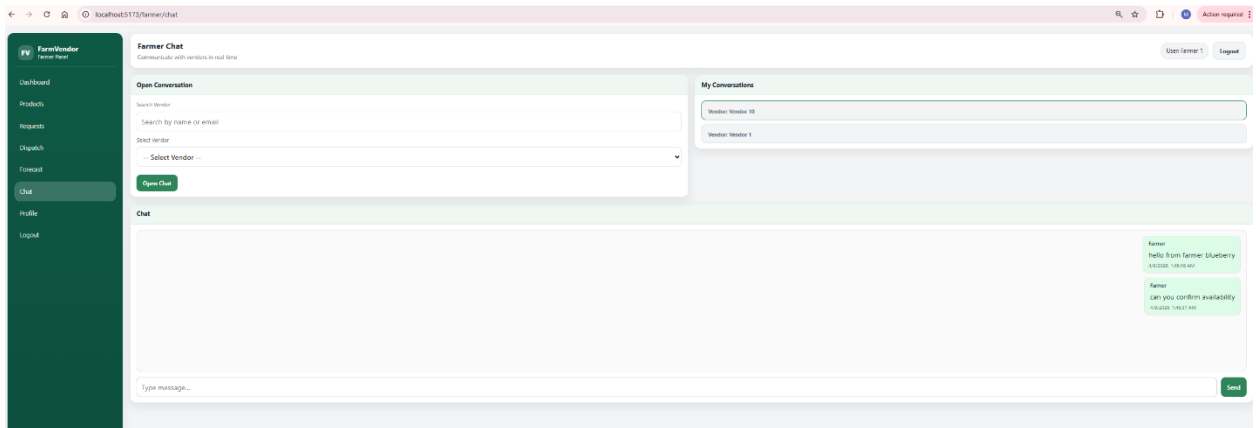
 Potatoes	22 kg	2026-04-10
 Milk	32 L	2026-04-10
 Onions	50 kg	2026-04-10
 Potatoes	10 kg	2026-04-11
 Tomatoes	24 kg	2026-04-12
 Milk	56 L	2026-04-12
 Onions	38 kg	2026-04-13
 Onions	25 kg	2026-04-13
 Onions	10 kg	2026-04-13

Add Product Lot

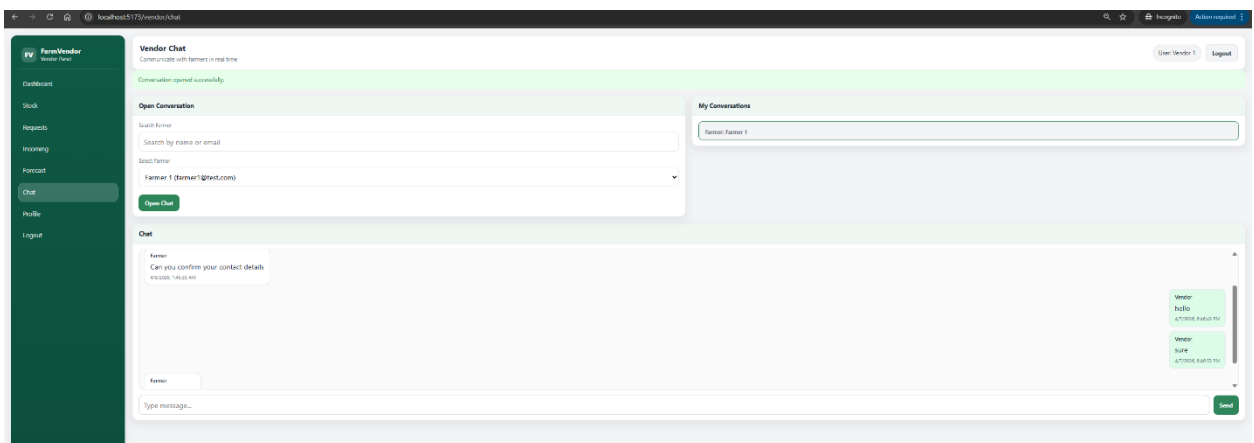
Create Dispatch

Requests loaded: 10

- Farmer chat panel



- Vendor chat panel

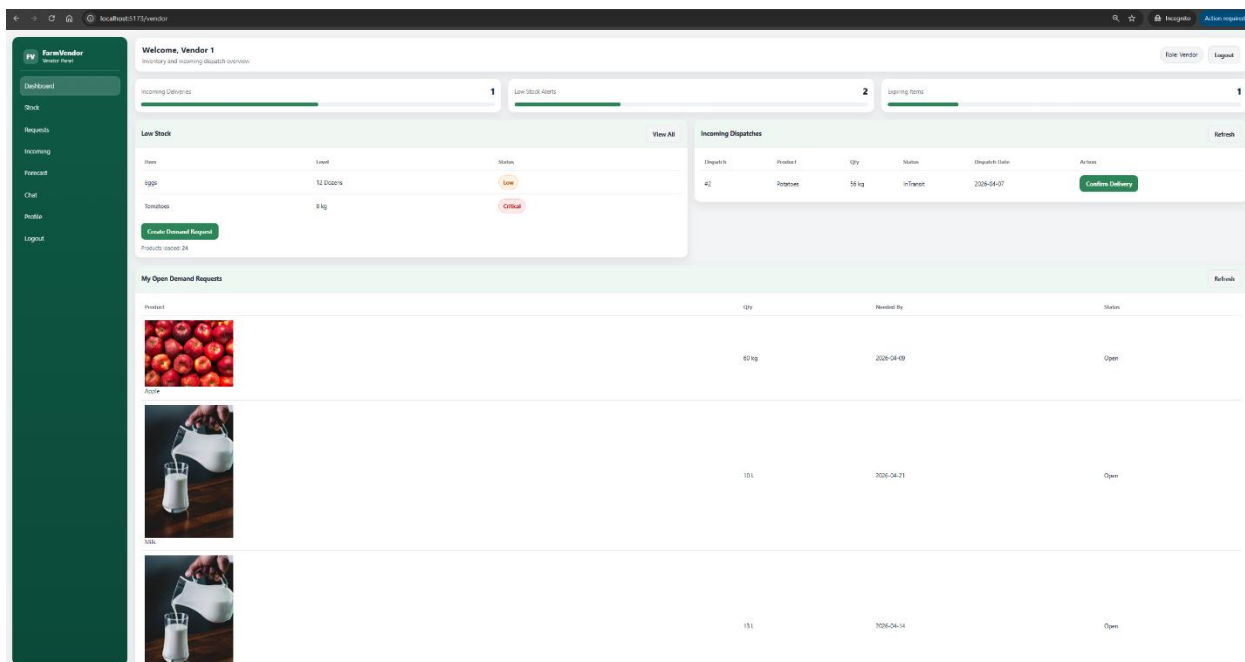


3. Vendor Functionalities

Dashboard

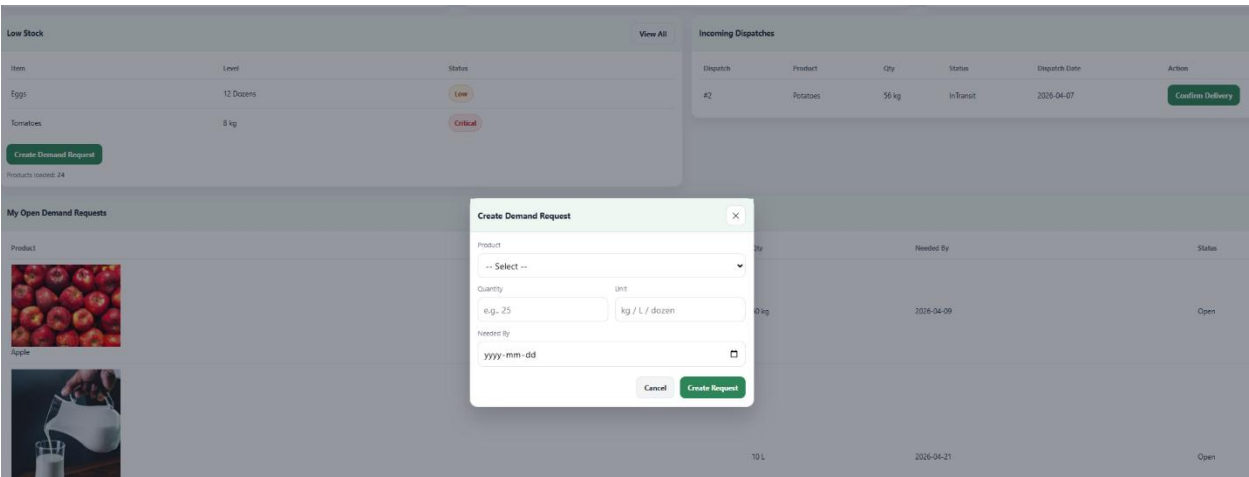
- View low stock alerts

- VendorDashboard



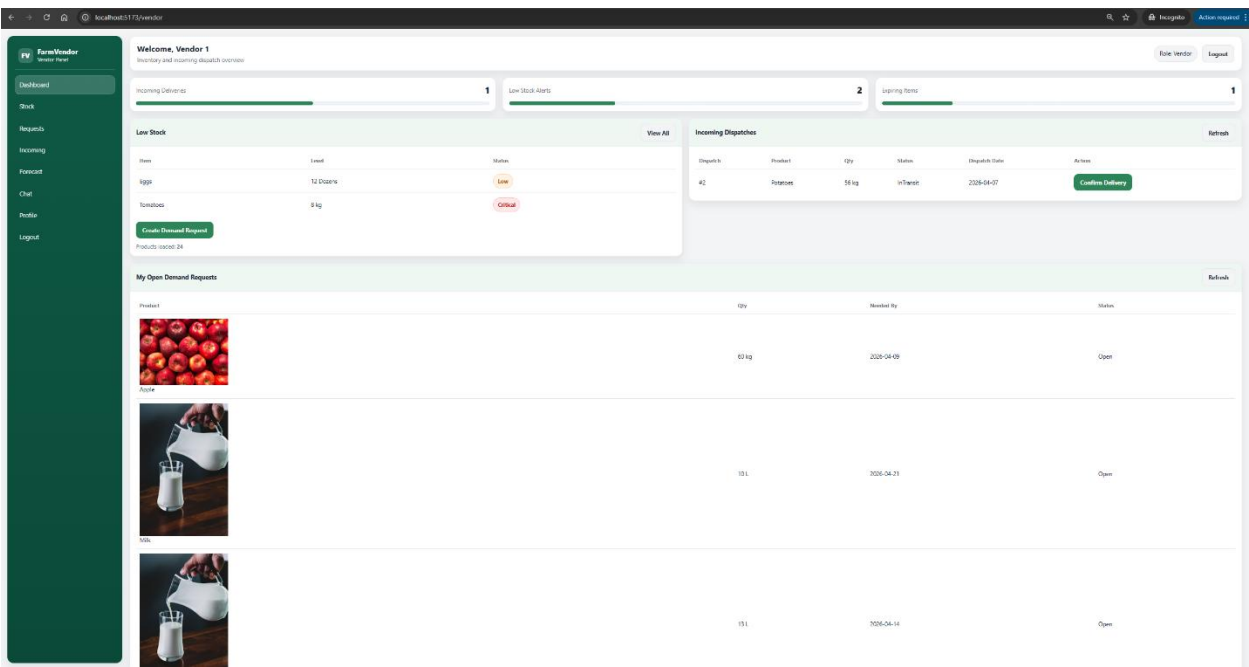
Demand Requests

- Create demand requests
- Track request status



Incoming Dispatches

- View deliveries from farmers
- Confirm delivery



Forecast Analytics

- View demand trends
- Compare forecasting models

Chat

- Communicate with farmers in real time
- Vendor chat panel

